Java Knowledge Boost

Category	Count
Java Conceptual Questions	106
Output Based Java Questions	67
Java Scenarios and Implementations	50
Stream Based Scenario Questions	50
Indirect Questions	366
Array coding Questions	25
String coding Questions	25
Experience wise Questions	
Java Cheat Sheet	

Interviewers usually focus on five major areas, and this document is designed to cover each one of them

1. Theory interview questions and Indirect questions.

2. Algorithm based interview questions.

3. Output based interview questions.

4. Scenario based interview questions.

5. Machine coding round.

Interview Tips (Technical):

1. Understand Core Java Concepts

Know the basics of Java well, including Object-Oriented Programming (inheritance, polymorphism, encapsulation, abstraction), data structures, algorithms, exception handling, multithreading, and the Collections Framework. Be ready to explain how these are used in your backend projects.

2. Learn Spring Framework Well

Be familiar with Spring and Spring Boot. Understand dependency injection, AOP, Spring Data JPA, Spring Security, and transaction management. Be able to explain how you used these features in your applications. Know the common Spring annotations.

3. Know RESTful API Design

Understand REST principles like statelessness, resource-based design, and proper use of HTTP methods (GET, POST, PUT, DELETE). Be ready to explain how to create secure, fast, and well-documented REST APIs using Spring.

4. Show Problem-Solving and System Design Skills

Be prepared to talk about tough technical problems you solved. Show that you can think about system design, performance, scalability, and fault tolerance. If you have experience with microservices or other architectures, explain it.

5. Be Good at Testing and Debugging

Explain how you test backend applications using unit tests, integration tests, or end-to-end tests. Mention tools like JUnit and Mockito. Show that you have written clean, maintainable, and well-tested code.

Interview Tips (General):

1. Discuss past projects using the STAR method

- Situation: Describe the context or challenge you faced.
- Task: State your specific responsibility in your project.
- Action: Explain the steps you took, decisions made, and collaboration involved to complete the task
- Result: Share the outcome and impact, preferably with metrics, e.g., reduced report generation time by 50%.

2. Communicate Clearly: Explain the concepts with examples or real time examples where you have used in your project

3. Highlight Collaboration Skills: Backend development involves working with frontend, DevOps, or QA teams. Share examples of teamwork and communication to show you fit in a collaborative environment.

4. Ask Smart Questions: Prepare 2–3 thoughtful questions about the team, architecture, or development practices. This shows interest and that you think beyond coding.

Click on any question to go straight to the answer.

Table of Contents

1 Why datatypes are important in Java?

Java is a **strongly-typed language**, which means we must tell the compiler what kind of data we are working with. Here is why data types matter:

1. Memory and Performance
Different data types use different sizes of memory.
Example: int uses less memory than long or double.
Using the right type helps our program run faster and use less memory.

2. To catch errors
Java checks the code at compile time.
If we try to assign a number to a String, it throws an error *before* the program runs.

3. Only Valid Operations Allowed
Each type supports specific actions.

- We can do math with numbers (int, double)
- We can use string methods with String

- We can't mix them up randomly
  This keeps the logic clean and predictable.

4. Easier to Read and Maintain

When we declare a variable like int age = 25; it is clear that age is meant to hold number.
That makes the code easier to read, understand, and maintain.

5. Helps Represent Data Correctly

Java gives us different types for different kinds of data:

- Whole numbers: byte, short, int, long
- Decimals: float, double
- Characters: char
- True/false: boolean
  Choosing the right one helps us represent and process the data properly.

Indirect Questions:

1. What would happen if you try to add a String and an int in Java? Why?

If we add a String and an int, Java converts the int to a String and adds.
"Age" + 25 gives "Age25".

2. Is it good use String isActive = "true"; instead of boolean isActive = true?

Using String isActive = "true" instead of a boolean is not ideal, it can lead to bugs since any string can be assigned, not just true or false.

3. Is Java strongly typed language?

Yes, Java is a strongly typed language. Variables must be declared with a specific data type**.** Before a variable can be used, its data type (e.g., int, String, double) must be explicitly specified during declaration.

2 What is reference type in java?

In Java, reference types (non-primitive types) store memory addresses (references) to objects rather than the actual data. Unlike primitive types, they can hold null, support methods, and are stored in the heap memory.

1. What's the default value of an object member in a class?

Default value of object is null.

2. Is String a primitive or object?

Object

3. What happens when you assign one object to another?

Both references now point to the same object.

3 In how many ways we can create the object?

1. Using new keyword

```
class Car {
    Car() {
                System.out.println("Car constructor called");
        }
}
Car car = new Car(); // Classic object creation
```

2. Using clone():

```
  class Car implements Cloneable {
        int speed = 100;
        public Object clone() throws CloneNotSupportedException {
                return super.clone();
        }
}
Car car1 = new Car();
Car car2 = (Car) car1.clone(); // Constructor NOT called
```

3. Using Deserialization (ObjectInputStream)

```
class Car implements Serializable {
        int speed = 100;
}
ObjectInputStream in = new ObjectInputStream(new FileInputStream("car.ser"));
Car car = (Car) in.readObject(); // Constructor NOT called
```

4. Using Reflection (Constructor.newInstance())

```
class Car {
    Car() {
                System.out.println("Reflection constructor called");
        }
}
Constructor<Car> cons = Car.class.getConstructor();
Car car = cons.newInstance(); // Constructor IS called
```

Indirect Questions

1. In which ways of object creation, the constructor doesn't get called?

Constructors don't get called during cloning (clone()) and deserialization (ObjectInputStream.readObject()), as they copy the object state directly.

2. how to create an object without using a new keyword?

We can create an object without new by using clone(), deserialization, or reflection

3. Which interface is must to clone the object?

Cloneable interface

4 Explain about 4 pillars of Object-oriented programming (OOPS) (Frequent Question).

1. Inheritance

Inheritance allows one class to acquire properties (fields) and behaviours (methods) of another. It promotes reusability and a natural hierarchy.

Interview Tip:

Always mention Java supports single inheritance with classes, but multiple inheritance via interfaces.

2. Encapsulation

Encapsulation is about protecting data by wrapping variables (fields) and code (methods) together in a class.

We restrict direct access to fields using access modifiers and expose behaviour through methods.

```
class Animal {
            void sound() {
                        System.out.println("Some generic animal sound");
            }
}

class Dog extends Animal {
      void bark() {
            System.out.println("Dog barks");
      }
}

class InheritanceDemo {
                  public static void
           main(String[] args) { Dog dog = new
           Dog();
                        dog.sound(); // inherited
           from Animal dog.bark();     // defined
           in Dog
               }
}
```

```
class Student {
            private int marks; // can't access directly from outside

            public void setMarks(int m) {
                        if (m >= 0) {
                                    marks = m;
                        }
            }

            public int getMarks() {
                        return marks;
            }
}

class EncapsulationDemo {
                  public static void
           main(String[] args) { Student s =
           new Student(); s.setMarks(85);
                        System.out.println("Marks: " + s.getMarks());
            }
}
```

Interview Tip:

Use private fields + public getters/setters to achieve Encapsulation.

3. Polymorphism

Polymorphism means "many forms" , the same object can behave differently based on context.

It is of two types: Compile-time (Method Overloading) and Runtime (Method Overriding).

```
class Shape {
            void draw() {
                        System.out.println("Drawing a shape");
            }
}

class Circle extends Shape {
            void draw() {
                        System.out.println("Drawing a circle");
            }
}

class PolymorphismDemo {
            public static void main(String[] args) {
                        Shape s1 = new Circle(); //
            Upcasting s1.draw(); // Calls Circle's
            overridden draw()
            }
```

Interview Tip:

Always say: Overriding = Runtime Polymorphism, Overloading = Compile-time Polymorphism. Runtime polymorphism (also called dynamic polymorphism) means the method that gets executed is determined at runtime, not compile time.

1. . Abstraction

Abstraction means hiding internal implementation and showing only essential details to the

```
}

class Bike extends Vehicle {
      void start() {
            System.out.println("Bike started");
      }
}

class AbstractionDemo {
                  public static void
            main(String[] args) { Vehicle v =
            new Bike(); v.start();
            }
}
```

user. It is implemented using abstract classes or interfaces

Interview Tip:

Always mention that interfaces provide 100% abstraction (prior to Java 8), and **abstract** classes can have both **abstract** and concrete methods.

5 How is abstraction different from encapsulation?

Abstraction focuses on hiding the internal process or complexity and showing only required things.

Example:

- We press a button on a washing machine and it washes clothes.

- We don’t see how it fills water, rotates the drum, or drains it.
- We only interact with the simple controls.

In code: Interfaces and abstract classes are common ways to apply abstraction.

Encapsulation focuses on hiding data and protecting it from direct access. It keeps the internal state safe and only exposes what is necessary.

Example:

- A car hides internal details like engine temperature or fuel control programs.
- The driver cannot access or modify them directly.

In code: We use private variables and public getters or setters to achieve encapsulation.

6 Questions on Encapsulation?

1. How to achieve encapsulation in Java?

Encapsulation is done by keeping class variables private and exposing public getter and setter methods. This restricts direct access to fields from outside the class.

2. Why is encapsulation needed?

Encapsulation helps protect data from unwanted changes. It keeps data and the methods that work on it in one place and controls who can access or change it. This makes the code safer and easier to manage.

3. Why do we make global variables private?

We make them private to prevent direct access or modification from outside the class. If fields are public, anyone can change them anytime, which can break the logic.

4. What is the role of setters and getters in encapsulation?

Setters and getters help us to control how someone uses the class's data.

A **getter** allows others to *see* the value, but not change it directly.

A **setter** allows to *change* it, but only the way we want.

5. If no encapsulation is implemented, what consequences may occur?

Without encapsulation, any part of the program can change the data. This can cause mistakes that are hard to find. One small change can break other parts of the program

6. Can you provide an example that illustrates the concept of encapsulation?

Imagine a BankAccount class with a private balance. We use deposit() and withdraw() methods to change it. Nobody can change the balance directly. they must go through these methods, which can include checks to prevent negative balances. That is a clean example of encapsulation.

7. How does encapsulation contribute to code organization and maintenance

Encapsulation helps to keep each part of the code separate. Every class handles its own data and work. So, we can change one part without worrying about breaking others. It also makes it easier for someone new to understand and work on just one part without looking at everything.

8. In what ways does encapsulation enhance code security?

Encapsulation hides important data so that other parts of the program can’t touch it directly. Only safe actions are allowed. This protects the data from being changed in the wrong way, either by mistake or on purpose. It is like putting a lock on something valuable so only the right

key can open it.

9. What are the potential drawbacks of overusing encapsulation in a program?

If we use encapsulation too much, the code can get bigger and more complicated. We might add getters and setters even when they aren't needed. It is better to use encapsulation only when it really helps.

7 Questions on Inheritance?

1. Why do we need inheritance in programming?

Inheritance helps reuse code and organize things better. Instead of writing the same features again and again, we just inherit them from a base class.

2. How do we make inheritance happen in code?

We use the extends keyword in Java. Let's say we have a class called Animal, and we want to create a class Dog that reuses Animal's code. We write class Dog extends Animal. Now Dog automatically gets all the public and protected stuff from Animal.

3. What does extends really mean?

It just means "I want to borrow everything public or protected from that class."

4. Can you give a real-life example of inheritance?

Think of a Vehicle class and a Car class. Vehicles can move and carry people. Cars do the same, but also have music systems and AC. So, Car extends Vehicle. This saves us from rewriting shared features.

5. Why can't we use multiple inheritance in Java?

It causes confusion. If two parents have the same method name, Java won't know which one to use, this is the "diamond problem". That is why, Java wont allow no multiple inheritance for classes. We need to use interfaces instead.

6. Can we do inheritance in Java without using extends?

Yes. We can use interfaces or composition. For example, instead of making a class extend another, we just include an object of that class inside.

7. Are constructors and private members inherited?

Constructors aren't inherited. The child class can call the parent constructor using super(), but it has to define its own.

Private members are also not inherited directly; We can't use them, But can access them by creating a public method.

8.How do you stop a class from being inherited?

Use the final keyword. If we write final class BankAccount, then no other class can extend it. We can also mark methods as final so they can't be overridden.

9. Why does Java allow multiple interfaces but not multiple classes?

Java allows a class to use many interfaces, but not many classes. That is because interfaces only have method names, not real code. So even if two interfaces have the same method, it is not a problem, the class that implements these interfaces writes its own version.

But classes have real code. If two classes have the same method with different logic, Java won't know which one to use. This creates confusion. So, Java allow a class extend only one class.

8 What is constructor in java?

A constructor in Java is a special method that gets called automatically when we create an object of a class. Its primary purpose is to initialize the newly created object, setting up its initial state or assigning default values to its attributes.

Points to remember:

- It has no return type (not even void).
- Its name is exactly the same as the class.
- We can have multiple constructors in the same class (this is called constructor overloading).

```
class Car {
        String model;

        // Constructor
    Car(String carModel) {
                model = carModel;
        }
}

public class Main {
            public static void main(String[]
        args) { Car myCar = new
        Car("Maruthi");
                System.out.println(myCar.model); // Prints "Maruthi"
        }
```

When new Car("Maruthi") is called, Java automatically runs the constructor, sets model = " Maruthi "

Indirect Questions:

1. What is difference of constructor and a method?

A constructor is used to initialize an object when it is created, while a method is used to perform actions after the object exists. Constructors don't have a return type and their name must match the class name. Methods can have return types and any name. Basically, constructors set up the object; methods operate on it.

2. Can a class have multiple constructors?

Yes, Java allows constructor overloading, so a class can have multiple constructors with different parameter lists. This helps in creating objects in different ways. For example, one constructor might set default values, and another might accept values from the user. The compiler figures out which one to call based on the arguments passed.

3. Explain the difference between a default constructor and a parameterized constructor

A default constructor has no parameters and sets fields to default values. It is either defined by us or provided automatically if no other constructors are defined. A parameterized constructor takes arguments and can initialize an object with specific vales.

3. Can a constructor have a return type?

No, constructors can't have any return type, not even void. If you try to give it a return type, Java treats it as a regular method, not a constructor. Constructor's return type is class type.

4. How is a constructor invoked in Java?

A constructor is automatically called when we create a new object using the new keyword. For example: new Student("John") invokes the constructor of the Student class

5. What happens if you don't provide a constructor in a class?

If we don't write any constructor, Java automatically provides a default no-arg constructor. It initializes object fields with default values, like 0 for numbers or null for objects.

6. Can you have both a default constructor and a parameterized constructor in the same class?

Yes, and it is very common. The compiler will invoke correct one based on the arguments.

7. Explain the concept of constructor Overloading?

Constructor overloading means writing multiple constructors in the same class with different parameter lists. It allows you to create objects in various ways

8. Can a constructor call another constructor of the same class? How?

Yes, using this(...). It is called constructor chaining. We can call another constructor in the same class from within a constructor using this() as the first line.

9. What is the purpose of a static block in Java, and how is it related to constructors?

A static block runs once when the class is first loaded, before any object is created or constructor is called. It is used to initialize static variables. It is not directly related to constructors. Static blocks are for class-level setup, while constructors deal with object-level initialization.

10. Explain the difference between an instance initialization block and a constructor.

An instance initialization block is a *code block* that runs before the constructor, every time an object is created. It is useful for common setup across multiple constructors. Constructors run after that. Both help initialize the object, but constructors are more flexible and commonly used.

11. Can we override the constructor?

No. Constructors are not inherited, so they cannot be overridden.

12. Can we overload the constructor?

Yes. We can have multiple constructors in the same class with different parameter lists.

13. Can a constructor be static or final?

Static: No, constructors are instance-level, not class-level.

Final: No, constructors cannot be final because they are not inherited and final applies to methods to prevent overriding.

14. Can a constructor have return type?

No, a constructor cannot have a return type, not even void.

If we declare a return type, it is treated as a regular method, not a constructor.

But every time we call **new**, the constructor implicitly returns the instance of the class itself.

9 Difference between == and .equals() in Java

What is ==?

- It checks **whether two references point to the same object in memory**.
- It does not compare values inside the object.

What is .equals()?

- It is a method used to compare the **actual values/content** of two objects.
- For most classes (like String), it is overridden to check logical equality.

```
String a = new String("CodingLyf");
String b = new String("CodingLyf ");

System.out.println(a == b); // false : different objects in memory
System.out.println(a.equals(b)); // true : same content
```

Interview Tip: Output based questions can be asked on this topic. Prepare for them. Will discuss in output-based questions section.

Related Questions:

1. What are different ways to create a String in Java? We can create strings using literals like

   String s = "hello"; -> this goes into the string pool.

   Using new keyword:

   String s = new String("hello"); -> this creates a new object in heap.

   We can also build strings dynamically using StringBuilder or StringBuffer.

   String s3 = new StringBuilder().append("he").append("llo").toString();

   From a character array using new String(char[]):

   char[] arr = {'h','i'}; String s4 = new String(arr);

2. What is the String Constant Pool?

   It is a special memory area in Java where string literals are stored.

   When we write String a = "hello"; and then String b = "hello";, both point to the same object in the pool so no duplicate objects created.

   It is to optimize memory since strings are used a lot and are immutable, so reusing the string is safe and efficient.

3. What does intern() do, and why is it useful?

   The intern() method Puts the string in the pool if it is not already there, or gives the pooled version. So even if we create a string using new, we can save the string in the pool.

Example:

```
String s1 = new String("hello");// Creates a new string object in the heap
String s2 = s1.intern();      // Adds "hello" to the pool and returns a reference
to it
String s3 = "hello";          // Directly references the "hello" in the string pool

System.out.println(s1 == s2); // false (s1 is in heap, s2 is in pool)
System.out.println(s2 == s3); // true (both refer to the same object in the pool)
```

Its primary benefit is memory optimization. By ensuring that only one instance of a unique string value exists in the string pool.

10 Is Java Purely Object-Oriented?

Java is not purely object-oriented. Here is why:

1. It supports primitive data types like `int`, `char`, `boolean`, etc., which are not objects.
2. It allows static methods and variables which belong to the class and not any instance.
3. Some operations like `System.out.println("Hello")` call static methods directly without creating an object.
4. Wrapper classes (like Integer, Double) were introduced to work with primitives in an object-oriented way.

So, Java is object-based and supports OOP concepts, but it is not 100% object-oriented.

11 Difference between final, finally, and finalize()?

They sound similar but do different things:

- final is a keyword. We can use it to make a variable constant, stop a method from being overridden, or prevent a class from being extended.

```
final int age = 25;
age = 30; // Error – We can't change it

final class Car {} // Can’t extend this class
```

- finally is the block of code in a try-catch that *always* runs, no matter what, even if there is an exception.

- finalize() is a method from the Object class that gets called just before the object is garbage collected. But honestly, no one really uses it anymore, it is considered outdated and unreliable.

Indirect Questions:

1. Can we combine abstract and final keywords in a class declaration?

No, we can’t use abstract and final together in a class. abstract means the class should be extended, while final means it can’t be extended.

2. Can we declare a variable as abstract?

No, you can’t declare a **variable** as abstract in Java.

3. Is it possible to declare a method as both final and abstract?

No, that is not allowed. final means the method can’t inherited, while abstract means it has no value or implementation and must be defined later.

4. Can you have static final variables in Java?

Yes, and it is very common. A static final variable is basically a constant. it belongs to the class and its value never changes. For example: static final double PI = 3.14;. It is shared across all instances.

5. Are there any built-in classes in Java that are declared as final?

Yes. Classes like String, Integer, and Math are declared as final. That means we can't extend or subclass them

12 Is Java pass-by-value or pass-by-reference?

Java is always pass-by-value, but there is a catch when it comes to objects.

- For **primitive types**, it is straightforward: a copy of the value is passed. So, changing the value inside the method doesn't affect the original.

  For primitives: We are sending a copy. Original value doesn't change.

```
    void
 changeValue(int x) {
 x = 20;
}

 int num = 10;
changeValue(num);
 System.out.println(num); // Still 10
```

- For **objects**, Java still passes by value, but that value is the reference to the
- object. So, the method gets a copy of the reference. We can use it to change the object's internal state (like modifying a field).

  For objects: We are sending a **copy of the reference**, so inside the method we can **change the object's internals**

```
class Dog {
                    String name;
}

void renameDog(Dog d) {
                    d.name = "Bruno";
}
Dog myDog = new Dog();
myDog.name = "Max";
renameDog(myDog);
System.out.println(myDog.name); // Bruno
```

Reference: Java: Pass By Value Or Pass By Reference | by Ananya Sen | Nov, 2020 | Medium | Medium

13 What is immutability, how to achieve it?

Immutability means: once an object is created, we cannot change its values.

Instead of modifying the same object, a **new object is created** when we try to change something.

In Java, this concept applies to strings, which means that once a string object is created, its content cannot be changed. If we try to change a string, Java does not modify the original one, it creates a new string object instead.

```
String name = "Ravi";
name = name + " Kumar";
System.out.println(name); // Output: Ravi Kumar
```

Here, "Ravi" did not become "Ravi Kumar". A new String "Ravi Kumar" was created and stored in name.

How to achieve immutability in Java?

To make a class immutable, follow these rules:

1. **Make the class final**
   So that no one can extend it.
2. **Make all fields private and final**
   This avoids direct access and ensures the value doesn't change after assignment.
3. **No setters, only getters**
   Don't provide methods to change the field values.
4. **If any field is an object, return a copy, not the original**
   So that internal data is not modified from outside. Refer (Output Question No :)

Indirect Questions:

1. How is immutability different from final keyword? final means we can't reassign the variable.

   Immutability means the object's internal data can't be changed.

2. Is final object immutable?

   No. A final object reference means we can't assign a new object to that variable. But the contents of the object can still be changed.

3. If a class has a mutable field (like a List), how do you stop callers from modifying it?

   if we return a mutable field like ArrayList directly from a getter, caller can update

```
public List<String> getList() {
        return list; // original list exposed
}
```

Someone can now call getList().add("Hack");

To make it immutable we need to return a copy of the List.

```
  public List<String> getSubjects() {
    return new ArrayList<>(subjects);
  }
```

4. How do you make a class thread-safe without using synchronized? 32

Make it immutable.

Immutable objects are naturally thread-safe, because no thread can change the state.

5. How would you ensure immutability in multi-threaded programming?

By declaring fields as final, making the class itself final, and not exposing setters or mutable references. This way, once the object is created, its state cannot be changed, ensuring thread-safety without extra synchronization.

6. Can we have mutable fields to immutable objects?

Yes, technically we can declare mutable fields inside an immutable object.

7. How would you design a custom immutable class that has ArrayList in it? Check

Question no. 3

8. Final vs immutable

Final and immutable may look similar, but they mean very different things in Java.

final is a Java keyword. It places restrictions on variables, methods, and classes. final doesn't freeze the object itself. If we make a List final, we can't assign a new List to that variable, but we can still add or remove items from the same List.

Immutability is about the design of the object itself. An immutable object does not allow its state to change after it is created.

The classic example is String. Once you create "Hello", we can't change its characters. Any modification creates a new object.

14 Why String is immutable in Java?

In Java, String is immutable, which means once we create a String, we cannot change its value. If we try to modify it, a new String object will be created.

Now let's understand why Java made String immutable:

1. **For security reasons**
   Strings are used in things like file paths, database connections, and user logins.
   If Strings were changeable, someone could easily tamper with these values, that would be dangerous.
   So, Java made String immutable to protect such sensitive data.

2. **To work properly in HashMaps and Sets**
   We often use Strings as keys in HashMap, HashSet, etc. These collections depend on the hashCode() of the key.
   If the String changes after adding it to the map, its hash code will also change and the map won't be able to find it again.

3. **Thread safety**
   Since Strings cannot be changed, multiple threads can use the same String object without any issue.
   No need for extra synchronization. This makes programs safer and faster.

4. **Memory efficiency using String Pool**
   Java stores common String values in a special memory area called the **String pool**.
   As Strings are immutable, Java can reuse them safely by saving memory and improving performance.

15 What is Object class in java and tell its methods?

In Java, Object class in Java is present in java.lang package. Every class in Java is directly or indirectly derived from the Object class. If a class does not extend any other class, then it is a direct child class of the Java Object class and if it extends another class then it is indirectly derived.

Here are the most important methods:

1. toString()
Returns a string representation of the object. Useful for printing/debugging.

2. equals(Object obj)
Used to compare two objects for logical equality.

3. hashCode()
Returns an integer used in hash-based collections like HashMap, HashSet. If we override equals(), we must also override hashCode().

4. getClass()
Returns the runtime class info of the object.

5. clone()
Used to create a copy of the object. By default, it does a shallow copy.
To use it, the class must implement Cloneable.

6. finalize() (Deprecated)
Was used to run cleanup code before garbage collection. Now deprecated. Avoid using it.

7. wait(), notify(), notifyAll()
Used for thread communication (in multithreading).
These are called on objects used as locks (inside synchronized blocks).

Reference: Object Class in Java - Scaler Topics

16 Deep Copy vs Shallow Copy?

A **shallow copy** makes a new object and copies everything from the old one. If it is a simple value (like an int), it copies the value itself.

If it is a complex object, it just copies the address/reference, so both still point to the same thing in memory.

Example:

Code link: JavaSamples/ShallowCopyExample.java at main · CodingLyf-Fullstack/JavaSamples

Explanation: In the below code, we have Person Object, it contains “name” as String and “Address” as object. When we copy a person object, “name” field will be copied as it is but since “Address” is object its reference will be copied not content.

So, when we change copied person name, it won’t affect the original person object name but when we change the address, it will affect the address of the original object.

```
public class Test {
        public static void main(String[] args) {
                // Original object
                Person originalPerson = new Person("John", new Address("Jubliee
        Hills"));

                // Shallow copy using copy constructor
                Person shallowCopyPerson = new Person(originalPerson);

                // Modifying the shallow copy
        shallowCopyPerson.setName("Jane");
        shallowCopyPerson.getAddress().setStreet("Madhapur");

                //Name is changed in Original Object but Address is changed
        System.out.println("Original Person: " + originalPerson);
        System.out.println("Shallow Copy Person: " + shallowCopyPerson);
        }
```

Deep Copy:

A deep copy makes a new object and also makes new copies of everything inside it. If it is a simple value (like an int), it copies the value itself.

If it is a complex object, it creates a new object in memory with the same data, so nothing is shared between the original and the copy.

We can achieve deep copy using “cloneable” marker interface.

Code link: JavaSamples/DeepCopyWithCloneable.java at main · CodingLyf- Fullstack/JavaSamples

Explanation: In the below code, we have a Person object that contains name as a String and Address as another object.

When we make a **deep copy** of the person object, both the name field and the Address object are copied independently.

So, when we change the copied person’s name, it won’t affect the original person’s name, and when we change the copied person’s address, it also won’t affect the original person’s address

```
public class DeepCopyWithCloneable {
        public static void main(String[] args) {
                // Original object
                Person originalPerson = new Person("John", new Address("123 Main
```

```
                // Deep copy using clone()
                Person deepCopyPerson = originalPerson.clone();

                // Modify the deep copy
        deepCopyPerson.setName("Jane");
        deepCopyPerson.getAddress().setStreet("456 Side St");

                // Print results
                System.out.println("Original Person: " + originalPerson);
        System.out.println("Deep Copy Person: " + deepCopyPerson);
        }
}
```

17 Explain the difference between String, StringBuilder, and StringBuffer.

All three are used to store and work with text in Java. But they behave differently based on mutability and thread safety.

String

- **Immutable**: We can't change a String once it is created.
  If we try to change it, Java creates a new object in memory.
- **Thread-safe**: Since we can't change it, it is safe to use in multi-threaded code.
- **Performance:** Slow for repeated changes. If we keep adding or changing, it keeps creating new strings. That wastes memory and slows down performance.

  Use when: The value won't change often, or we just want simple text.

StringBuilder

- **Mutable**: We can change the content without creating a new object.
- **Not thread-safe**: Not safe if multiple threads are trying to use the same object.
- **Performance:** Fast. Best for string changes inside loops or single-threaded programs.

  Use when: We want better performance and we are working in a single-threaded environment.

StringBuffer

- **Mutable**: Just like StringBuilder, it allows changes to the content.
- **Thread-safe**: All methods are synchronized, so it is safe when multiple threads are using it.
- **Performance:** Slower than StringBuilder. Because of thread-safety (synchronization), it is a bit slower even if we are not using multiple threads.

  Use when: We want to change strings in a multi-threaded program where thread safety is important.

Additional Question:

1. Is it good to use StringBuffer in Single threaded application?

No, because it adds overhead of Synchronization which slows down the process.

18 What are Access Modifiers in Java?

Access modifiers are keywords in Java that control who can access a class, method, or variable.

We can think of them like doors, they decide whether something is open to all, only to the family, or just to yourself.

Java has four access modifiers:

1. public

- Means **"open to all"**
- Any class, anywhere in the project, can access it.

Use it, when we want to make the method or variable available everywhere.

2. private

- Means "only for me"

- The variable or method can only be accessed within the same class.

If we try to use age or displayAge() from another class, we will get a compile-time error. Use it, when we want to hide data or logic from outside , for encapsulation.

3. protected

- Means "family access"
- Can be accessed:
    - Within the same class
    - In other classes in the same package
    - In subclasses (child classes) even if they are in different packages

  Use when: We want to share with related classes (like inheritance), but not with the whole world.

4. default (no keyword)

- If we don't write any modifier, it becomes default access
- Visible only to classes within the same package int salary

= 50000; // default access

This salary variable can only be used by classes in the same package, not outside. Use when: We are okay to share within the package, but not outside.

19 What are Wrapper Classes in Java and Why Do We Need Them?

In Java, primitive types like int, char, float are not objects. But Java is an object-oriented language, and sometimes, especially while working with collections or frameworks, we need everything to be an object.

That is where wrapper classes come in.

Wrapper classes are object representations of the primitive data types

Primitive	Wrapper Class
int	Integer
char	Character
float	Flot
double	Double
boolean	Booln
long	Long
short	Short
byte	Byte

```
int a = 10;                 // primitive

Integer b = Integer.valueOf(a); // wrapper object
```

Why do we need Wrapper Classes?

1. For Collections like ArrayList, HashSet, etc.

Collections in Java cannot store primitive types directly. They only accept objects. So, if we

want to store an int inside an ArrayList, we must use the Integer class.

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();

numbers.add(10);
```

2. For Utility Methods

Wrapper classes have helpful methods.

Example: We want to convert a String "123" into an int.

```
String s = "123";

int x = Integer.parseInt(s);
```

This method parseInt() is *a*vailable only in the Integer class, not with primitive types.

20 What is Autoboxing and Unboxing in Java?

In Java, sometimes we need to convert between primitive types (int, char, double, etc.) and their wrapper classes (Integer, Character, Double, etc.).
Java does this automatically in many cases, this is called Autoboxing and Unboxing.

What is Autoboxing?

Autoboxing means converting a **primitive** to its **wrapper** class automatically. Java does this when we are assigning a primitive to an object, like:

```
int a = 10;
Integer obj = a; // autoboxing: int -> Integer

List<Integer> list = new ArrayList<>();
list.add(100); // Java converts int 100 to Integer automatically
```

So no need to write Integer.valueOf(100) , Java does it for us.

What is Unboxing?

Unboxing means converting a **wrapper object** back to its **primitive type** automatically.

```
Integer obj = 50;

int a = obj; // unboxing: Integer -> int
```

Again, Java internally calls obj.intValue(), but we don't need to write that.

21 What is Type casting?

Type casting in Java is the process of converting a variable from one data type to another. It can be either widening casting (automatic) or narrowing casting (manual).

1. Widening Casting (Automatic)

Widening casting is when a smaller data type is automatically converted to a larger data type. This type of casting is done by Java without the need for any explicit instruction from the programmer.

Examples of Widening Casting:

byte -> short -> int -> long -> float -> double

```
int num = 10;
double doubleNum = num;  // Automatic widening from int to double
System.out.println(doubleNum);  // Output: 10.0
```

Narrowing Casting (Explicit)

Narrowing casting is when a larger data type is converted into a smaller one. This type of casting is not done automatically and requires explicit instructions from the programmer using

parentheses. There is potential for data loss, so Java requires confirmation for this casting.

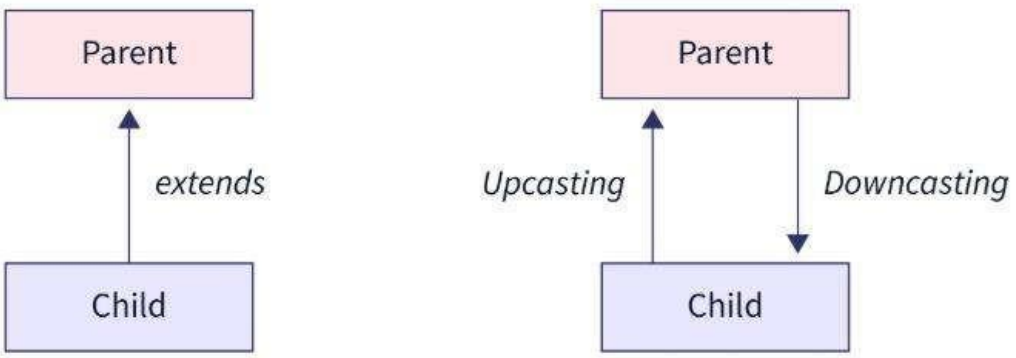
Examples of Narrowing Casting:

double -> float -> long -> int -> short -> byte

```
double doubleValue = 10.5;

int intValue = (int) doubleValue; // Manual narrowing from double to int
(truncates decimal part)

System.out.println(intValue); // Output: 10 (decimal part is lost)
```

Types of Type Casting in Java

1. **Primitive Type Casting**: Converting between primitive data types as shown above.
2. **Reference Type Casting**: Casting between objects of different types within an inheritance hierarchy. This requires upcasting (automatic) or downcasting (explicit).



Upcasting is nothing but, typecasting of a child object to a parent object.

Downcasting refers to typecasting a parent object to a child object. However, this cannot be an implicit typecasting.

22 What is a static keyword in Java (Commonly asked Question)?

In Java, if we want to access class members, we must first create an instance of the class. But there will be situations where we want to access class members without creating any variables.

In those situations, we can use the static keyword in Java. If we want to access class members without creating an instance of the class, we need to declare the class members static.The Math class in Java has almost all of its members static. So, we can access its members

```
class Counter {
        static int count = 0; // shared variable

        Counter() {
                count++;
        System.out.println(count);
        }
}
public class Main {
        public static void main(String[] args) {
                new  Counter();  //
        prints 1 new Counter(); //
        prints 2 new Counter(); //
        prints 3
        }
```

without creating instances of the Math class.

1. Static Variables

Static variables are shared among all instances of a class. They are stored in the class memory rather than the heap (where instance variables are stored).

When to Use:

- When we want to keep a single copy of a variable that should be shared by all instances of the class.

2. For constant Static Methods

Static methods belong to the class rather than any particular instance. They can access static variables directly but cannot access instance variables or methods.

In every Java program, we have declared the main method static. It is because to run the program the JVM should be able to invoke the main method during the initial phase where no objects exist in the memory.

- Is that belong to the class rather than any specific instance

3. Static Methods

Static methods belong to the class rather than any particular instance. They can access static variables directly but cannot access instance variables or methods.

In every Java program, we have declared the main method static. It is because to run the program the JVM should be able to invoke the main method during the initial phase where no objects exist in the memory.

```
class MathUtils {
        static int square(int n) {
                return n * n;
        }
}

public class Main {
            public static void main(String[] args) {
        System.out.println(MathUtils.square(5)); // prints 25
        }
}
```

4. Static Methods

Static methods belong to the class rather than any particular instance. They can access static variables directly but cannot access instance variables or methods.

In every Java program, we have declared the main method static. It is because to run the program the JVM should be able to invoke the main method during the initial phase where no objects exist in the memory.

When to Use:

- When we want a method that can be called without creating an instance of the class.
- For utility or helper methods that don't require any object state.

5. Static Blocks

Static blocks are used for static initialization of a class. They run when the class is loaded.

When to Use:

- To initialize static variables or perform operations that need to happen once when the class is loaded

The static block is executed only once when the class is loaded in memory. The class is loaded if either the object of the class is requested in code or the static members are requested in code.

```
class Test {
    // static variable
    static int age;

    // static block
    static {
        age = 23;
    }
}
```

Reference: Java Static Keyword (With Examples)

Indirect Questions:

1. Can a static constructor exist in Java?

When to Use:

- To initialize static variables or perform operations that need to happen once when the class is loaded

The static block is executed only once when the class is loaded in memory. The class is loaded if either the object of the class is requested in code or the static members are requested in code.

A class can have multiple static blocks and each static block is executed in the same sequence in which they have been written in a program.

No. Java doesn't support static constructors like C#. But we can use a static block to run code when the class is first loaded. It is usually used to initialize static variables. It runs only once, when the class is loaded into memory.

2. What is the process for accessing static variables in Java?

We can access static variables using the class name, like ClassName.variable. We can also use an object, but the class way is better and clearer. Since static variables belong to the class, we don't need an object to use them.

3. What is the difference between static and non-static methods?

Static methods belong to the class and can run without an object. Non-static methods belong to objects, so we need an instance to use them.

4. Can static methods access non-static variables in Java?

No, they can't directly access non-static variables because those belong to an object. Static methods don't have access to this, we can pass an object as a parameter, so they access instance variables or non-static methods.

5. Are static members shared among different instances of a class in Java?

Yes. Static members are shared across all instances of a class. If one object changes a static variable, that change is reflected in all other objects.

6. Can we throw checked exceptions from static block?

From a static block, We cannot throw checked exceptions directly. We must handle them inside the block or wrap them in an unchecked exception.

```
public class Example {
        static {
                try {
                        throw new IOException("Checked exception in static block");
                    } catch (IOException e) {
            e.printStackTrace(); // handle it here
                }
        }
}
```

Static blocks: Checked exceptions must be caught or wrapped; can't propagate.

23 What are Nested Classes?

In Java, sometimes we need to group classes that are logically related. One class can be declared inside another; this is called a Nested Class.

There are two primary types: Static Nested Class and Non-Static Inner Class.

Static Nested Class

A static nested class is declared with the 'static' keyword inside another class.

It belongs to the outer class itself, not to any instance. This means it can only access the static members of the outer class. It does NOT need an object of the outer class to be created.

```
class DatabaseConfig {
      static String driver = "com.mysql.jdbc.Driver";

       static class Credentials {
              void printDriver() {
                                      System.out.println("Driver: " + driver); //
                    accessing static

                           }
              }
}

//Usage Example for Static Nested Class
class StaticNestedDemo {
              public static void main(String[] args) {
                           DatabaseConfig.Credentials creds = new
             DatabaseConfig.Credentials(); creds.printDriver();
              }
}
```

When to Use Static Nested Class:

- When the nested class is a helper or utility related to the outer class.
- When it doesn't need access to instance (non-static) variables/methods.
- It helps in logically grouping classes and keeping code organized.

Non-Static Inner class:

A non-static inner class is an instance-level class defined inside another class.

It is tied to an instance of the outer class. It CAN access all members of the outer class, even private ones.

When to Use Non-Static Inner Class:

- When the inner class needs full access to the outer class instance's state.

- When it makes sense conceptually that the inner class is a part of the outer class.
- For example: Car -> Engine, Computer -> CPU

24 What is the difference between method overloading and method overriding? (Commonly asked question)

One of the most common interview questions in Java is the difference between method overloading and method overriding. These two concepts fall under polymorphism but they work very differently.

Method Overloading (Compile-time Polymorphism)

Method overloading means having multiple methods with the **same name but different parameter lists** (either in number, type, or order). It is resolved at compile time.

```
class Calculator {
        int add(int a, int b) {
                return a + b;
        }

        double add(double a, double b) {
                return a + b;
        }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

class OverloadingDemo {
            public static void main(String[]
        args) { Calculator calc = new
        Calculator();
                System.out.println(calc.add(10, 20)); // calls int, int
        System.out.println(calc.add(5.5, 4.5)); // calls double, double
        System.out.println(calc.add(1, 2, 3)); // calls int, int, int
        }
```

Key Point:
- All overloaded methods must differ in parameters.
- Return type alone is NOT enough to overload.

Method Overriding (Runtime Polymorphism)

Method overriding is when a subclass redefines a method of its parent class with the **same name, return type, and parameters**. It is resolved at runtime.

Output based questions can be asked on rules of Method overriding. Check the next question for rules.

```
class Animal {
        void speak() {
                System.out.println("Animal speaks");
        }
}
```

```
class Cat extends Animal {
            void speak() {
                        System.out.println("Cat meows");
            }
}

class OverridingDemo {
            public static void main(String[] args) {
                        Animal animal = new Cat(); //
            Upcasting animal.speak(); // Calls Cat's speak()
            method
            }
```

Key Rules:

- Method name, parameters, and **return** type must match exactly.

- Access modifier cannot be more restrictive. (Refer Next question for these rules).

- We can use @Override annotation to help the compiler.

Interview Tip:

Always mention overloading increases readability and flexibility.

Overriding enables dynamic behaviour and is key to runtime polymorphism.

Reference: Java Method Overloading and Overriding | Medium

Indirect Questions:

1. Can you overload a method by changing only the return type?

No, we cannot overload a method just by changing the return type in Java.
The compiler decides which method to call based on the method name and parameters (not return type).

25 What are the rules for overriding a method in Java?

Mostly, Output based questions can be asked on these rules. We will see those questions in Output based question section. Here are the rules that needs to be remembered.

Rule #1: Only inherited methods can be overridden.

Because overriding happens when a subclass re-implements a method inherited from a superclass, so only inherited methods can be overridden, that is straightforward. That means only methods declared with the following access modifiers: **public**, **protected** and default (in the same package) can be overridden. That also means **private** methods cannot be overridden.

Rule #2: Final and static methods cannot be overridden.

Rule #3: The overriding method must have same argument list.

Rule #4: The overriding method must have same return type (or subtype).

Rule #5: The overriding method must not have more restrictive access modifier.

This rule can be understood as follows:

- If a method is public, we cannot override it with protected, default, or private.
- If a method is protected, we cannot override it with default or private.
- If a method has default (package-private) access, we cannot override it with private **Rule #6: The overriding method must not throw new or broader checked exceptions Rule #7: Use the super keyword to invoke the overridden method from a subclass.**

Indirect Questions:

1. Why can't private methods, static methods, or constructors be overridden in a subclass?

Because private methods aren't visible outside the class, static methods belong to the class not the object, and constructors are meant to initialize that class not its child.

2. What happens if you try to override a final method in a subclass?

We will get a compile-time error, we cannot override the final class

3. Can a subclass provide its own implementation of a static method from the superclass? Yes, but that is method hiding, not overriding.

4. What happens if a subclass method has the same name but different parameters compared to the superclass method?

That is called overloading, not overriding. Java will treat it as a separate method.

5. Does changing the parameter types in a subclass method still considered as overriding?

No. To override, the method signature must match exactly, same name, same parameters.

6. How does Java differentiate between overriding and overloading in a subclass?

If the method signatures match, it is overriding. If only the names match but parameters are different, it is overloading.

7. Can a public method in a parent class be overridden as protected or private?

No, we can’t reduce visibility.

8. Can a protected method in a parent class be overridden as public? Yes. Increasing visibility is allowed.

9. Is it possible for an overriding method to have a completely different return type than the parent?

We cannot use completely different type. it must be a subtype of the original (covariant return), or else Java throws compile error.

10. How does covariant return type apply in method overriding?

Covariant return types in method overriding allow an overridden method in a subclass to return a more specific type (a subtype) of the return type declared in the superclass's method.

11. How can you call a parent method from a child?

Just use super.methodName(), this tells Java to call the parent method.

26 What is Shadowing in java and its types?

Shadowing happens when a variable or method in a subclass hides (shadows) a variable or method with the same name in the parent class.

1. Variable Shadowing

- If a subclass declares a variable with the same name as a variable in the superclass, the subclass variable shadows the parent one.

- The type of the variable can be different.
- Shadowing is different from overriding because it works on variables, not methods.

```
class Parent {
        String name = "Parent";
}

class Child extends Parent {
        int name = 10; // Shadows Parent's 'name' (different data type)

        void printNames() {
                System.out.println(name);           // 10 (Child's
        name) System.out.println(super.name); // "Parent" (Parent's
        name)
        }
}

public class Main {
            public static void main(String[]
        args) { Parent parent = new Child();
        System.out.println(parent.name); //Parent

                Child child = new Child();
        child.printNames();
        }
```

2. Method Shadowing (Static Method Hiding)

- If a subclass defines a static method with the same name as a static method in the parent class, the subclass method hides the parent one.
- This is not method overriding because static methods are bound at compile-time (not runtime).
- The method signature must match (same name & parameters).

```
class Parent {
            static void print() {
         System.out.println("Parent's static method");



        }
}

class Child extends Parent {
            static void print() { // Hides Parent's print()
        System.out.println("Child's static method");
        }
}

public class Main {
            public static void main(String[] args)
        { Parent.print(); // "Parent's static
        method" Child.print();   // "Child's static
        method"

                Parent obj = new Child();
                obj.print();    // "Parent's static method" (Not overridden!)
        }
```

27 What are super and this in Java? How are they used?

this keyword.

- Refers to the **current object** of the class.

- Used when we want to refer to the current instance variable or method.
- Also used to call another constructor in the same class.

Use Cases of **this**:

- Resolving name conflicts between instance variables and parameters.
- Passing current object as argument: someMethod(**this**)
- Chaining constructors: **this**(…) to call another constructor in same **class**.

```
class Student {
		String name;

		Student(String name) {
			this.name = name; // distinguishes instance variable from
		constructor
parameter
		}

		void display() {
			System.out.println("Name: " + this.name);
		}
}

class ThisKeywordDemo {
			public static void main(String[]
		args) { Student s = new
		Student("CodingLyf"); s.display();
		}
```

super keyword

- Refers to the parent **class**.
- Used to call parent **class**'s constructor, method, or variable.
- Helps when subclass overrides methods or hides variables.

Use Cases of **super**:

- Calling overridden method from parent class.
- Accessing hidden fields in superclass.
- Calling parent class constructor using **super**().

Interview Tip:

- **this** is about the current object, **super** is about inheritance.
- Always mention that **super**(...) must be the first statement in a constructor.
- **this**(...) also must be the first if used, but can't combine both in same constructor.

Practice a few scenarios involving constructor chaining and method overriding and you willl easily master these for interviews!

Reference: This and Super Keyword in Java - Scaler Topics

Indirect Questions:

1. In what situations would you need to use this in a constructor?

When the constructor parameter names are the same as instance variable names, this helps avoid confusion. For example: this.name = name; tells Java we are assigning the value to the object's field, not just the parameter.

2. Can this be used to call a method or access a variable?

Yes. We can use 'this' to access the instance variables or call other methods in the same class. It is helpful when local variable names clash or when we want to make it clear that we are working with the current object's fields or behavior.

3. Does this always refer to the current class instance?

Yes, it always refers to the current object of the class are are working with. It is how we point to the actual instance the code is running on. It doesn't refer to the class or other objects, only the current one.

4. In what situations would you use super() or this() in a constructor?

Use super() when we want to call a parent class constructor. Use this() to call another constructor in the same class.

5. Can it be possible to put super() or this() anywhere in a Java program?

No. We can only use super() or this() as the very first line inside a constructor. If we try to place it anywhere else, the compiler will throw an error.

28 Explain about Abstract classes in Java

An abstract class in Java is a class that cannot be instantiated directly. It is meant to be extended by other classes. Think of it as a **partial blueprint** for subclasses. It defines some behaviour, but leaves other parts for subclasses to implement.

Why to use abstract classes?

- To enforce common behaviour in subclasses
- To provide a base structure but allow custom logic in specific classes

```
abstract class Shape {
        abstract void draw(); // abstract method (no body)

        void move() {
                System.out.println("Moving the shape"); // concrete method
        }
}

class Circle extends
Shape { @Override


        void draw() {
                System.out.println("Drawing a circle");
        }
}

class AbstractClassDemo {
        public static void main(String[] args) {
                Shape s = new Circle(); // Allowed, reference is abstract,
        object is
concrete
                s.draw();
                s.move();
        }
```

Rules of Abstract Classes

- It Can have abstract methods (without body) and concrete methods (with body)
- It Cannot be instantiated directly (new Shape() is not possible)
- It Must be extended by subclasses
- Subclasses must implement all abstract methods, unless the subclass is also abstract

When to use abstract classes:

- When we have a common base but not every detail is known in base class
- When some methods should have default implementation, but others should be overridden

Interview Tip:

- Mention we use abstract class when we want to provide common behaviour with shared code.
- Prefer interface if we only need method declarations.

Real-Time Example: Payment System

Imagine we are building a payment system for an e-commerce app. We need a common structure for all payment methods (e.g., CreditCard, UPI, NetBanking).

Code: JavaSamples/AbstractDemo.java at main · CodingLyf-Fullstack/JavaSamples

Indirect Questions:

1. How can we achieve abstraction?

In Java, abstraction is done using abstract classes and interfaces. We hide implementation details and show only the required functionality.

2. Can you provide a real-life example of abstraction?

Think of an ATM machine. We interact using a screen and buttons, withdraw, deposit, check balance, but have no idea what's happening behind the scenes. That is called abstraction.

3. Are there any built-in abstract classes in Java?

Yes. Java has built-in abstract classes like AbstractList, InputStream, Reader.

4. Can we create an instance of an abstract class in Java? No, We can't.

But can create an object of a concrete subclass that implements the abstract class.

5. Can abstract class has constructor?

Abstract classes can have constructors. We can't create an object from them directly, but the constructor runs when a subclass is instantiated. It is useful for initializing the shared properties.

6. Can we declare an abstract method as static or private?

No, abstract methods can't be static or private. abstract means the method must be overridden in a subclass. But static means it belongs to the class and can't be overridden. private means it can't be accessed outside the class

7. Can an abstract class extend another abstract class? Yes

8. Can an abstract class implement the interface?

Yes, it can implement the interface and don't need to override the methods However, any concrete subclass extending that abstract class must override those methods.

In the below, get() is not overridden in Child class so it must be overridden any class the extends Child.

```
interface Parent
{
        void show();
        void get();

}
abstract class Child implements Parent
{
        public void show()
        {
                System.out.println("Inside Child");
        }

}
```

9. Is it possible to create reference variable to abstract class and assign the concrete class object?

We can't instantiate an abstract class, but we can create a reference variable of its type and assign it to a concrete subclass object.

```
abstract class Animal {
        abstract void sound();
}

class Dog extends Animal {
            void sound() {
        System.out.println("Bark");
        }
}
```

```
public class Test {
        public static void main(String[] args) {
                Animal a = new Dog();  // abstract class reference, concrete
        object a.sound(); // prints "Bark"
        }
}
```

29 Explain about Interfaces in Java

An interface in Java is like a contract. it *defines* a *set* of *methods* that a class must implement. Interfaces help achieve 100% abstraction (before Java 8) and allow multiple inheritance.

Think of it as a rulebook, any class that 'signs' the interface must follow the rules (i.e., implement the methods).

Key Features of Interface:

- All methods are public and abstract by default (until Java 7)
- From Java 8 onwards, interfaces can have default and static methods
- Interface cannot have constructors
- All variables in interface are public, static, and final
- A class can implement multiple interfaces (solves the multiple inheritance issue)

Interview Tip:

- Use interface when we only need to define the contract, not implementation
- We can combine multiple interfaces using `implements A, B, C` unlike classes

- Interfaces support default and static methods (since Java 8)

Real-World Example: Payment Gateway Integration

Let's say You are building an app that supports multiple payment providers like Razorpay, Paytm, and PhonePe.

Code: JavaSamples/InterfaceDemo.java at main · CodingLyf-Fullstack/JavaSamples

30 Interface: Indirect questions

1. Can a class implement multiple interfaces?

Yes, Java supports multiple interface implementation.

This *is* one way Java supports multiple inheritance of

type.

A class can implement many interfaces, separated by commas.

2. If two interfaces have methods with the same signature, what happens?

It will compile; the method needs to be implemented once in the class.

But if both interfaces have default methods with same name, we must override it. Java forces to resolve conflicts manually.

3. Can an interface have a *constructor*?

Interfaces can't have constructors because they can't be instantiated directly. Interfaces are meant for defining contracts, no for object creation.

Only classes have constructors in Java.

4. What's the difference between abstract class and interface if both can have abstract methods?

Abstract class can have state (variables, constructor, etc), interfaces can't (except constants). We can extend only one abstract class, but implement multiple interfaces.

Interfaces are ideal when we just need to define a structure.

5. Have you used any built-in Java interfaces in your code?

Yes, common ones are `Runnable`, `Comparable`, `Serializable`, `AutoCloseable`.

These are part of core Java libraries.

6. What happens if you don't implement all methods of an interface?

Then the class must be declared `abstract`.

Java will not compile it if we don't implement all the methods.

7. Can we define variables inside an interface?

Yes, but all variables are implicitly `public static final` (constants). We cannot define instance variables.

8. Can interface methods be private or protected?

No, all methods are `public` by default in classic interfaces.

Since Java 9, `private` methods are allowed for internal use only. `protected` is not allowed.

9. Can an interface extend another interface?

Yes. One interface can extend one or more interfaces. This helps in grouping related contracts.

10. Why are default methods introduced in Java 8 interfaces?

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Methods	Can have both abstract and concrete methods	Abstract by default (before Java 8), can have default and static methods (Java 8+)
Inheritance	Single inheritance (extends one class)	Multiple inheritance (implements many interfaces)
Variables	Can be any type (instance, static, final, non-final)	Only public static final (constants)
Constructors	Yes (can have constructors)	No (cannot be instantiated)
Access Modifiers	Can use any (private, protected, etc.)	All members are public by default
Default Implementation	Yes (can provide method implementations)	Yes (via default methods since Java 8)

To allow interfaces to have some method implementation. This helps add new methods without breaking existing code. Mainly used for backward compatibility and utility methods.

11. Can an interface have variables? What type are they by default?

Yes, interfaces can have variables, which are by default public, static, and final constants. This means they are accessible from anywhere, belong to the interface itself rather than any specific object, and their values cannot be changed after being initialized.

31 Interface vs Abstract Class

Think of Abstract Class as a semi-built house:

- Some walls, roof, and structure are already there
- Subclass (child class) needs to complete it

Think of Interface as a blank contract or blue print of the house:

- It just gives blue print of the paper and says what things must be done.
- We decide how to implement it.

32 What is Serialization in java?

Serialization is the process of converting a Java object into a byte stream. This byte stream can be stored in a file, sent over a network, or saved in a database.

Deserialization is the reverse , converting the byte stream back into a Java object.

- In a microservices architecture, different services communicate with each other over the network, often through REST APIs, gRPC, or message queues (like Kafka or RabbitMQ). Serialization allows the data objects to be converted into a byte stream so that they can be transmitted across network boundaries.
- **For example**, when a service needs to send a complex object as part of a request or a message, making the object Serializable ensures it can be converted into a format (e.g., JSON, XML) suitable for network transfer.
- In a RESTful service, serialization is commonly used to convert objects to a JSON or XML format for HTTP communication between a client and server. In Java, libraries like Jackson, Gson, or JAXB handle serialization (converting objects to JSON or XML) and deserialization (parsing JSON or XML back into objects)

Important Points:

- The class must implement **Serializable** interface to make it serializable
- **transient** keyword is used for fields which we don't want to save
- **SerialVersionUID** should be declared to maintain version compatibility

Indirect Questions:

1.How do you make a class serializable in Java?

To make a class serializable in Java, it must implement the `Serializable` interface. This is a marker interface with no methods.

2. What is serialVersionUID?

`serialVersionUID` is a unique identifier for a `Serializable` class. It is used during deserialization to ensure that the sender and receiver of a serialized object have loaded classes for that object that are compatible

3. What happens if you don’t define `serialVersionUID` in a Serializable class?

If `serialVersionUID` is not explicitly defined, the Java runtime will generate one automatically based on various aspects of the class. This can lead to unexpected `InvalidClassException` errors if the class is modified, as the generated `serialVersionUID` might change

4. When will be serialization useful?

Serialization is useful when we need to convert an object into a byte stream so it can be stored or transferred, then later rebuilt.

Typical cases:

- Saving an object’s state to a file or database.
- Sending objects over a network (like Rest APIs)

  In REST APIs, serialization is what happens when the Java object (like a DTO or entity) is

converted into JSON or XML before sending it in the HTTP response.

Likewise, when the API receives JSON/XML, it is deserialized back into a Java object. So, without serialization, the REST API couldn't exchange data with clients.

- Caching objects in memory or on disk.

33 What is transient in java?

In Java, the `**transient**` keyword is used to tell the JVM: "Do NOT save this variable during serialization." That means if an object is converted into a byte stream, all `transient` fields will be ignored.

Why is this useful?

Let's say we have a password field or some sensitive/confidential data , we don't want it stored in files or transferred.
Or maybe we have a field like a file stream or a socket, which is not serializable.

Possible Indirect Questions:

How do you prevent certain fields from being serialized?

We can prevent certain fields from being serialized by marking them with the `transient` keyword. Transient fields are ignored during the serialization process.

34 What is marker interface in java?

A Marker Interface in Java is an interface that has no **methods** or **fields**.
It is completely empty. We use it just to mark a class, like saying "this class has a special quality".

Think of it like sticking a 'special sticker' on a class.
JVM see that sticker and do something different for that class.

Examples for Marker Interfaces in Java:
- Serializable: tells JVM that objects of this class can be serialized.
- Cloneable: allows an object to be cloned using Object.clone().

Real-World Analogy:
Think of a movie ticket with 'VIP' stamped on it.
The stamp itself doesn't do anything, but the staff sees it and allow into a special lounge.
Similarly, JVM checks if a class implements certain marker interfaces and behaves accordingly.

Indirect Questions:

1. Have you written any custom marker interface?
2. Can you implement a custom marker interface for example?

Auditable -> Marks entities whose changes should be logged (e.g., created/updated

imestamps).

```
//Marker interface with no methods
public class AuditDemo {
        public static void main(String[] args) {
                LibraryItem book = new LibraryItem("Clean
        Code"); book.updateTitle("Clean Code - Updated");

                logChange(book);
        }

        static void logChange(Object obj) {
                    if (obj instanceof Auditable) {
            System.out.println("Audit log: " + obj);
                }
        }

        public void updateTitle(String newTitle) {
                this.title = newTitle;
                this.updatedAt = LocalDateTime.now();
        }

        @Override
        public String toString() {
                return "LibraryItem{" +
                                "title='" +
                title + '\'' + ",
                createdAt=" + createdAt + ",
                updatedAt=" + updatedAt +
                '}';
        }
}
```

Main class:

3. Can we achieve marker interface with annotations?

Yes, In Spring framework we can achieve this with annotation.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Marker {}

@Marker
public class MyClass {}
```

35 Explain about Association, Aggregation and Composition?

These three are used to define how objects are connected to each other.

Think of them as the strength of relationship between two classes.

Let's go from basic to strong: Association -> Aggregation -> Composition.

1. Association

- This is the most basic connection between two classes.
- Example: A Student attends a college. Student and College are related.

- But both can exist without each other. One doesn't depend on the other.

```
class Student
{ String name;
    College college; // Association
}

class College
{ String name;
}
```

2. Aggregation

- A more specific type of association where one object "has-a" relationship with another object.
- It means one class contains another, but the child can still exist on its own.
- Example: A Library has Books. Even if Library is deleted, Books can still exist elsewhere.
- Example: A Team object and a Player object. The team contains multiple players but a player can exist without a team.

```
class Library {
        List<Book> books; // Aggregation
}

class Book {
String title;
}
```

3. Composition

- The strongest form of association, also a "has-a" relationship.
- If the parent is deleted, child also goes.
- Example: Consider the case of Human having a heart. Here Human object contains the heart and heart cannot exist without Human.

```
class House {
        Room room = new Room(); // Composition
}

class Room {
    int size;
}
```

Indirect Questions:

1. Can you explain the difference between aggregation and composition? Aggregation is a weak relationship, child can exist on its own.
In composition, child's life depends on the parent.

2. How would you model a car and its engine in Java?
Car and Engine are tightly connected, so it is composition. If the car's gone, engine usually has no use also

3. What's the best way to model a Book and Library in OOP?
Books can exist without a Library. they can belong to another. So, it is aggregation.

4. Is a bank and account aggregation or composition?
That is aggregation, accounts can be moved to another bank. They don't die with the bank.

5. Can an object be part of multiple compositions? No, in composition, the parent fully owns the child. One child can't belong to multiple owners like that.

Code: JavaSamples/CompositionDemo.java at main · CodingLyf-Fullstack/JavaSamples

36 Why "Composition over inheritance"?

In object-oriented programming (OOP), we find two fundamental relationships that describe how objects relate and interact with each other: '**IS-A**' and '**HAS-A**'

In OOP, 'IS-A' is a relationship where one object is a subtype of another. A Car is a Vehicle; a Dog is an Animal; this relationship is manifested through inheritance. The 'HAS-A' relationship is

where one object contains or uses another object. A Car has an Engine; a Dog has a Tail; this relationship is realized through composition

Inheritance means one class can use the code from another class by extending it. That might be helpful, but it is not always a good idea.

Why? Because it creates a strong link between the two classes. If the parent class changes, it can break the child class. Also, Interfaces has N number of methods, we need to inherit all of them even though we don't need.

Reference: Composition over Inheritance [Object Oriented Programming]

Favoring Composition Over Inheritance | by Sheldon Cohen | Medium (Code might be C# but theory is good to understand)

37 What is multi-tasking?

Multitasking allows a system (or program) to run more than one task simultaneously. For example, we might be listening to music while downloading a file and editing a document, all happening at the same time.

In Java, multitasking is handled through multithreading or multiprocessing, depending on the context. Types of Multitasking in Java

There are two main types of multitasking:

1. **Process-based Multitasking**
2. **Thread-based Multitasking**

1. Process-Based Multitasking

Each task is a separate process, meaning a complete, independent program running in its own memory space.

Example

- Opening a browser, opening a code editor, and opening a video player at the same time. Points to remember:
    - Each process runs independently.
    - It requires more memory since each process has its own memory and resources.
    - Switching between processes involves context switching, which is expensive.

- In Java, this is done using the ProcessBuilder or Runtime.exec().

2. Thread-Based Multitasking

Each task is a separate thread, but all threads run within a single process. They share the same memory space.

Example:

- A video player: one thread handles video decoding, another thread handles audio, another thread handles sub titles.

Points to remember:

- Lightweight compared to processes.
- Threads share memory, which makes data sharing easy but also introduces the need for synchronization.
- Faster context switching between threads.
- Most commonly used in Java via Thread class or implementing Runnable.

38 Difference between Process and Thread? What is a Process?

A process is an independent running program.

- When we start a Java application (like java MyApp), the JVM (Java Virtual Machine) creates a new process.
- Each process has its own memory space, meaning it doesn't share data with other processes.
- Two processes cannot directly access each other's memory. They need special communication methods like sockets or inter-process communication (IPC).
- Processes are handled by the OS, but threads are what Java developers often use to get things done efficiently inside a single app.

Real-world example:

Opening two different apps on your computer, like a browser and a text editor, each runs as a separate process. They don't interfere with each other.

What is a Thread?

A thread is a smaller unit of execution within a process.

- A Java process (i.e., your running app) can have multiple threads running inside it.
- All threads in the same process share memory (especially the heap). This makes data sharing fast and easy.
- Each thread still has its own call stack (local variables), so they don't clash at every level.
- Threads are commonly used for multitasking, doing more than one thing at the same time. Real-world example:

In a video calling app (like Zoom), multiple threads may be running:

- One thread handles audio
- Another handles video
- A third listens for chat messages
- All of these threads live inside one single process: the Zoom app

Indirect Questions:

1. What is thread pool?

- A thread pool is a collection of reusable threads managed by the JVM.

- Instead of creating a new thread every time (which is costly), tasks are submitted to the pool and executed by existing threads.
- In Java, we typically use ExecutorService / Executors.newFixedThreadPool().

2. Explain object-level thread locks.

- When we mark a method or block as synchronized (non-static), the lock is taken on the current object (this).
- Only one thread can access the synchronized block/method of that object at a time.
- But different instances of the class have different locks.

39 What is Java Memory Model (JMM)?

The Java Memory Model explains how threads share data. Imagine every thread has its own notepad (working memory) where it keeps copies of variables from the main whiteboard (heap). A problem appears when one thread updates the whiteboard but another thread is still looking at its old notes.

Java Memory Model establishes rules and guarantees regarding the visibility, ordering, and atomicity of operations on shared variables, ensuring consistent and predictable behaviour in multithreaded environments.

Here are the key parts:

1. **Heap Memory**: Stores objects and shared data. All threads can access it.
2. **Stack Memory**: Each thread has its own stack, storing method calls and local variables.
3. **Working Memory (per thread)**: A thread often keeps a copy of variables in its own cache. This is faster, but can cause problems if other threads change the original variable in the heap.

3 rules JVM follows in multi-threading

1. Visibility Rule

- Without synchronization, Change made by one thread may stay in cache and not seen by others.
- Use volatile or synchronized so updates are visible across threads.

2. Ordering Rule

- JVM and CPU may reorder instructions for speed.
- It uses the **happens-before** rule: if action A happens before B, then B will always see A's changes.

3. Atomicity Rule

- The JMM guarantees that certain operations appear as a single, indivisible unit
- Reads/writes to int, boolean, etc. are atomic.

For long and double, use volatile or synchronized for atomicity.

Indirect Questions:

1. How JMM Affects Threads

When multiple threads run in Java, they don't directly read/write main memory all the time. Instead, each thread has its own working memory (CPU cache).

Thread-local caching: Each thread may keep variables locally to avoid constant memory access.

Main memory: Shared among all threads. Updates in one thread are not immediately visible to others unless synchronized properly.

If Thread A updates a variable, Thread B may not see the update immediately. Use volatile, synchronized blocks, or atomic variables.

2. What is ThreadLocal in java and where would you use it?

ThreadLocal is a special kind of variable in Java where each thread gets its own independent copy.

- Normally, if multiple threads use the same variable, they share it.
- With ThreadLocal, each thread has its own private copy, invisible to other threads.

Think of it like giving each worker in a team their own notepad. Even if they're all writing notes, no one can see or mess with anyone else's notes.

Use cases:

Database connections / transactions (per thread):

In frameworks, a thread might handle one request.

A ThreadLocal can store the current DB connection or transaction

User/session context:

In a web app, when a request comes in, we can store the current user or request ID in a ThreadLocal. Later, anywhere in the call chain, we can fetch it without explicitly passing it as a parameter.

40 What is multi-threading how is it different from multi-tasking?

Multithreading is a programming concept where a single process can perform multiple tasks concurrently using multiple threads.

Each thread is like a lightweight sub-process. They run in parallel (or appear to) and share the same memory and resources of the parent process.

Multitasking means the operating system is running *multiple programs* at once.

Example:

- We are running a Java app, a browser, and a PDF reader all at the same time.
- Each of these runs as a separate process.

- The OS switches between them so fast that it feels like they are all running at the same time.

Multitasking = Our computer runs many apps at once.

Multithreading = A single app handles many tasks at once internally.

41 What is Thread, how to create threads in Java?

A **Thread** is just a lightweight worker inside the Java program.

Every Java program starts with one which is called a main thread. But if we want the program to do more than one thing at the same time (like download a file and show a progress bar), we need more threads.

Real-Life Example:

Imagine we running a restaurant alone. We take orders, cook, serve, one at a time. That is a single-threaded program.

Now hire more staff, one handles orders, another cooks, another serves. Things move faster. That is **multi-threading**. **33**

Java lets us create such workers inside the program. There are **two ways to create a Thread**

1. By extending the Thread class:

```
class MyThread extends Thread {
        public void run() {
                System.out.println("Thread is running...");
        }
}

public class Main {
            public static void main(String[]
        args) { MyThread t = new MyThread();
        t.start(); // never call run() directly
        }
}
```

2. By implementing the Runnable interface

```
 class MyRunnable implements Runnable {
        public void run() {
                System.out.println("Runnable thread is running...");
        }
}

public class Main {
            public static void main(String[]
        args) { Thread t = new Thread(new
        MyRunnable()); t.start();
        }
}
```

Possible Indirect Questions:

1. What happens if we call run() instead of start()?

It runs like a normal method inside the main thread, no new thread is created. So, there's no parallel execution.

2. Can you run two threads on the same object?

Yes, we can create multiple Thread objects and pass the same Runnable.
Each thread will run its own run() but share the same object state.

Sample: JavaSamples/ThreadDemo.java at main · CodingLyf-Fullstack/JavaSamples

3. Why is Runnable preferred in real-world projects?

Because Java allows only one class to be extended, but multiple interfaces can be implemented. So Runnable gives more flexibility, especially in complex designs.

4. Can we create threads without extending the Thread class?

Yes , just implement Runnable or use lambda/anonymous classes with Thread. Extending Thread isn't required at all.

5. Is Java thread-safe by default?

No, most Java code is not thread-safe unless we explicitly handle it. We need to use synchronization, locks, or concurrent APIs.

6. Why threads are useful in Java?

Java uses threads heavily for better performance. For example:

- In **web servers**, one thread handles each user request.
- In **desktop apps**, one thread shows the UI, another handles background tasks.
- Java provides built-in tools like Thread, Runnable, ExecutorService to manage threads.

42 What are Runnable and Callable interfaces, and their differences?

In Java, when we want to run some tasks using threads. we usually write code that goes inside Runnable or Callable. Both run tasks in parallel, but there is one key difference:

Runnable doesn't return anything. Callable returns a result.

Another Difference is Runnable cannot throw Checked Exceptions, Whereas Callable can throw.

Runnable:

Think of Runnable like sending your friend to buy something, but we don't wait to hear what happened.
We just say: "Go do this" and move on.

Runnable is used when we don't need a result back. It is used to run some code.

```
class MyTask implements Runnable {
        public void run() {
        System.out.println("Task is running");
        }
}

public class Main {
        public static void main(String[]
        args) { Thread t = new Thread(new
        MyTask()); t.start();
        }
}
```

Callable

Now think of Callable like sending a delivery guy and expecting him to come back with your package. We will use it when we need result from the task.

Interview Tip:

```
class MyCallable implements Callable<String> {
        public String call() throws Exception {
                return "Task completed!";
        }
}

public class Main {
        public static void main(String[] args) throws Exception {
        ExecutorService service = Executors.newSingleThreadExecutor();
        Future<String> future = service.submit(new MyCallable());

                String result = future.get(); // waits for the
        result System.out.println(result);

                service.shutdown();
        }
}
```

Runnable is useful when we just want to execute code. Callable is better when we need the result or want to handle exceptions properly.

Callable is usually used with ExecutorService and Future to get results.

Indirect Questions:

1. Can threads return values in Java?
   Yes, by using Callable with ExecutorService, threads can return values. The result is accessed through a Future object.
2. Why does Runnable not support checked exceptions?
   Because Runnable.run() doesn't declare throws Exception, so checked exceptions aren't allowed. We have to handle them manually inside the method.
3. How would you handle result + exception in a thread? Use Callable and submit it to an ExecutorService.
   Then call future.get() , it gives the result or throws the exception.
4. How cancel the long running callable

We can wrap the Callable in a Future using an ExecutorService. Then call future.cancel(true) to try interrupting the running task.

```
ExecutorService executor = Executors.newSingleThreadExecutor();

Callable<String> longTask = () -> {
        //Code
};

Future<String> future = executor.submit(longTask);

System.out.println("Cancelling task...");
future.cancel(true);
```

43 What is Thread life cycle?

In Java, a thread goes through various states from its creation to termination. Understanding the thread life cycle is crucial for effective multithreaded programming. Here are the states in a thread's life cycle:

Thread States

1. **New (Born State)**
   When a thread is created (using new Thread() or implementing Runnable) but not yet started. The thread exists but isn't yet scheduled for execution.
2. **Runnable (Ready-to-Run)**
   After calling start() method, the thread enters the runnable state.
   The thread is ready to run and waiting for the thread scheduler to allocate CPU time. Note: This is sometimes called "Ready" state when the thread is waiting for CPU.
3. **Running**
   When the thread scheduler selects the thread, it moves to the running state. The run() method is executing.
4. **Blocked/Waiting**
   Blocked: When a thread is temporarily inactive (e.g., waiting for I/O, waiting to acquire a lock).
   Waiting: When a thread waits indefinitely for another thread to perform an

action (using wait(), join(), etc.).

Timed Waiting: When a thread waits for a specified time (using sleep(time), wait(timeout), etc.).

5. **Terminated (Dead)**

When the thread completes its run() method or is terminated abnormally.
The thread cannot be restarted (calling start() on a dead thread throws IllegalThreadStateException).

Indirect Questions:

1. What happens if you call start() twice on the same thread?

It throws IllegalThreadStateException. A thread can be started only once.2.

2. Can a thread go from RUNNABLE to TERMINATED without entering BLOCKED or WAITING? Yes, if it finishes execution without needing to wait or get blocked.

3. Can a thread be started twice? Why or why not?

No. Once it is dead, it is dead. We have to create a new thread object.

4. What happens if you call start() after run() was already called directly?

It still works. but run() just ran like a normal method, not a thread. Now start() creates a proper thread.

44 What is Executor Service and what are the uses??

Managing threads manually can become messy , like handling 50 workers without any manager. **ExecutorService** is that manager. It decides which thread does what and when. Saves memory, improves performance.

ExecutorService is a high-level API in Java that makes it easy to run and manage multiple threads. Instead of creating a new thread every time, we submit tasks to an executor, and it runs them using a pool of threads behind the scenes.

Technical Notes:

The ExecutorService interface is part of the java.util.concurrent package

It extends the Executor interface, which defines a single method execute(Runnable command) for executing tasks.

Executors is a utility class in Java that provides factory methods for creating and managing different types of ExecutorService instances.

Always remember to call **shutdown**() when we are done.

```
class MyTask implements Runnable {
        public void run() {
                System.out.println("Running in: " + Thread.currentThread().getName());
        }
}

public class Main {
        public static void main(String[] args) {
                ExecutorService service = Executors.newFixedThreadPool(3); // 3 threads
        in pool
```

```
            for (int i = 1; i <= 5;
        i++) { service.execute(new
        MyTask());
             }

             service.shutdown(); // important to release resources
    }
```

Indirect Questions:

1. How do you manage multiple threads in a clean way?

Use ExecutorService to handle tasks.It creates a thread pool, assigns work, and reuses threads. No need to manually start or stop threads each time.

2. What's the difference between execute() and submit()? execute() runs a task but gives no result back.
submit() runs a task and returns a Future .so we can get the result or catch exceptions.

3. Can ExecutorService handle exceptions from tasks?

Yes , but only if we use submit() and check the result with Future.get().

If we use execute(), exceptions may be lost unless we log them manually inside the task.

4. What happens if you don't shut down an executor?

The app will keep running because the executor's threads stay alive.

Always call shutdown() to release resources and stop the thread pool cleanly.

5. How would you design thread pool based on system load?

- By Using ThreadPoolExecutor, ThreadPoolExecutor is a Java class that manages a pool of worker threads to execute tasks asynchronously.
- It allows us to reuse threads, control the number of concurrent threads, queue tasks, and define policies for handling extra tasks when the pool is full.

Reference: Understanding ThreadPoolExecutor in Java | by Pranav Tiwari | Medium

45 What is Synchronization?

In multithreaded applications, multiple threads can attempt to access and modify the same shared resources (e.g., variables, objects, files) concurrently. Without synchronization, this concurrent access can lead to unpredictable and incorrect results.

Synchronization in Java is a mechanism that controls access to shared resources by multiple threads in a multithreaded environment. It ensures that only one thread can execute a critical section of code

Real-World Example (I did not get any best example than this 😊)

- Imagine a public toilet with one door and a key.
- If one person is inside and locks the door, no one else can enter.
- Only after they unlock and exit can the next person go in.

This is how synchronized works in Java. It gives the first thread a key (a monitor/lock) and blocks others until it is done.

Indirect Questions:

1. How do you avoid race conditions in multithreaded Java code?

Use synchronized, Locks, or thread-safe data structures to ensure only one thread accesses critical code at a time.

This prevents threads from messing with shared data at the same time.

2. What happens if two threads access the same variable at the same time?

If that variable isn't protected (no lock), both threads might read or write at the same time, causing unpredictable results.

This is how race conditions and weird bugs happen.

3. Can two threads call the same synchronized method at once?

No, if the method is synchronized on the same object, only one thread can enter it at a time. The other thread waits until the first one releases the lock.

4. Does synchronized make Java objects thread-safe?

Only the synchronized parts are safe, the rest of the object can still be accessed unsafely.

5. How would you ensure that a piece of code is executed by only one thread at a time? Use synchronized block or method.

6. Is Synchronized thread safe?

Yes, synchronized is a keyword and a mechanism used to achieve thread safety in languages like Java, ensuring that only one thread can access a shared resource or critical section of code at a time, thus preventing data inconsistencyIf static synchronized method and instance synchronized called simultaneously on same object, will it conflict?

No, they won't conflict.

- A static synchronized method locks on the Class object (ClassName.class).
- An instance synchronized method locks on the current instance (this).

46 Difference Between Synchronized Method and Synchronized Block?

As a part of Synchronization, we will lock some resource (method or object) using **synchronized** keyword.

But we don't always want to lock everything. Locking too much can slow down the app. So, we have two options: Lock entire method called **synchronized methods** or lock only that particular section of the code calling **synchronized blocks**

Real time example:

In our cupboard, there can be multiple drawers. We have two options:

1. Have a single door and lock entire cupboard (Synchronized Method).
2. Have multiple doors and lock only particular cupboard where valuable things are placed.

This is actually a better option (Synchronized Block).

Synchronized Method:

When we add synchronized to a method, the entire method gets locked.

```
public synchronized void updateBalance() {
  // Whole method is locked
  balance += 100;
}
```

Code: JavaSamples/SynchronizedMethodExample.java at main · CodingLyf-Fullstack/JavaSamples

Synchronized Block

Here, we only lock the particular section of the code.

```
public void updateBalance() {

    synchronized (this) {
    balance += 100; // Only this part is locked
    }
}
```

Code: JavaSamples/SynchronizedBlockExample.java at main · CodingLyf-Fullstack/JavaSamples

Indirect Questions:

1. What's the performance impact of using synchronized?
   It can slow things down if too many threads keep waiting to enter locked code. Use it only where needed, or the program will become slow.
2. How do you lock only part of a method?
   Just wrap the critical code inside a synchronized block instead of the whole method. That way, only that small section gets locked. it is faster, cleaner, and safer.
3. Can I use multiple locks in one class?
   Yes, we can create different lock objects and sync on each. It is useful when we want to protect different data separately.
   Think of it like having separate keys for different drawers instead of locking the whole cupboard.
4. What happens if I synchronize too much?
   Too much locking means threads keep waiting, and the app becomes slow or stuck. It is like blocking the whole road for a bike to pass. It is not efficient at all.
5. If two threads try to access the same synchronized method, what happens to the second one? It waits and gets into blocked state until the first one releases the lock.
6. Can a thread acquire multiple locks? Yes, a thread can acquire multiple locks.
7. What if order of acquiring multiple locks is inconsistent by threads?
   If multiple threads acquire multiple locks in different orders, it can lead to a deadlock.

47 Difference of wait() , notify() and notifyAll()?

In a multithreaded program, threads often need to wait for some condition to happen or signal other threads when they're done.

That is where wait(), notify(), and notifyAll() come into play. These are methods defined in the Object class , not in Thread class, because in Java, every object can act as a lock.

Real-World Example:

Imagine there's only one phone booth in your office.

- If someone is inside, the others wait.
- When the person inside comes out, they say "done" then only the next person can go. That is wait() (someone waiting), and notify() (someone getting informed that they

can go in).

wait(): Causes the current thread to pause its execution and release the lock it holds on the object. Thread goes to waiting state.

notify(): wakes up **one** thread that is waiting on the same object

notifyAll(): wakes up **all** waiting threads (only one can proceed at a time)

ProducerConsumer Program, one of the frequently asked Programs

Code: JavaSamples/ProducerConsumer.java at main · CodingLyf-Fullstack/JavaSamples

Possible Indirect Questions:

1. What's the difference between sleep() and wait()?

sleep() just pauses the thread but doesn't release the lock.

wait() pauses and also gives up the lock so other threads can work.

2. Can I use wait() without synchronization?

No, if we call wait() without holding the object's lock, Java throws an IllegalMonitorStateException**.**

Always use it inside a synchronized block.

3. Can multiple threads wait on the same object?

Yes, many threads can call wait() on the same object.

When notify() or notifyAll() is called, one or all of them will wake up.

4. What happens if a Thread calls notify where no threads are in waiting state?

If a thread call notify() but no other thread is currently waiting on the same monitor (object), the notify() method will simply do nothing.

5. What happens if you call notify() without holding the object's lock?

It throws IllegalMonitorStateException. We must own the lock to call notify().

6. What will happen if you call wait() outside synchronized?

It throws IllegalMonitorStateException. wait() needs the lock too.

48 Difference of sleep(), wait() and join()? sleep()

- sleep() is a static method in the Thread class.
- It causes the currently executing thread to pause for a specified time duration (in milliseconds).
- It does not release any locks held by the thread.
- It is used to introduce a delay in the execution of a thread.

wait()

- wait() is an instance method defined in the Object class, but it is typically called on an object within a synchronized block.
- It is used for thread synchronization and inter-thread communication.
- When a thread calls wait(), it releases the lock on the object and is suspended until another thread calls notify() or notifyAll() on the same object.
- It is crucial for scenarios where one thread needs to wait for a specific condition to be met by another thread.

join()

- join() is a method of the Thread class.

- It allows one thread to wait for the completion of another thread.
- When thread A calls threadB.join(), thread A waits until threadB finishes

Possible Indirect Questions:

1. If a thread is sleeping, can another thread enter its synchronized block?
   No, because sleep() doesn't release the lock. Other threads still have to wait.
2. Will a sleeping thread respond to notify()?
   Nope, notify only works on threads that called wait(), not sleep().
3. How can you ensure that Thread A completes its execution before Thread B starts? Call A.join() inside B. That way, B waits until A finishes.
4. Can join() cause a deadlock?
   Yes, if two threads are calling join() on each other, they'll wait forever.
5. Can you pause a thread without using sleep() or wait()? Yeah, use join()

49 Difference between Volatile and Atomic? Volatile:

Imagine you have two threads, one is writing to a variable; another is reading it.

By default, Java threads can cache variables locally. That means, Thread A might write a value, but Thread B still sees an old one.

When we mark a variable as volatile, we are saying:

"Don't cache this. Always read and write from main memory."

So now, when one thread updates a volatile variable, the change is immediately visible to other threads.

Here is the catch

volatile does not guarantee atomicity for compound operations.

For example, volatile int count = 0; count++; is not an atomic operation.

It involves a read, an increment, and a write, which can be interrupted by other threads, leading to race conditions.

To achieve atomicity, we use Atomic Classes.

Atomic:

Atomic classes are for atomicity and visibility. Atomic classes make sure that when one thread changes a value, it does it completely and no one else can mess with it halfway. That is called atomicity

At the same time, visibility means: once a thread updates the value, other threads can immediately see the new value. There's no delay.

```
AtomicInteger count = new AtomicInteger(0);

   public void increment() {
   count.incrementAndGet(); // Atomic and safe
}
```

Reference: Exploring Thread Safety with AtomicInteger in Java

Interview Tip for threads:

- Mention about ExecutorService over raw Thread for pooling and scalability.

Use Runnable/Callable instead of extending Thread**.**

- Tell about synchronized for simple locking, ReentrantLock for advanced control, and AtomicInteger or other atomic classes for counters.

- Mention about apply volatile for shared flags where visibility matters, but remember it doesn't ensure atomicity.

Possible Indirect Questions:

1.How does AtomicInteger work internally?

It uses something called Compare-And-Swap (CAS), a low-level CPU instruction.

2. Would you use volatile for a counter? No, it is not safe for counters.

Because even though threads can see the latest value, they can still overwrite each other's updates. So, always use AtomicInteger or synchronized when we are doing read-modify-write.

3. What's the difference between synchronized, volatile, and Atomic?

- Volatile is like saying "everyone, please read the latest value." That is it.
- Synchronized adds a lock: only one thread can enter that block at a time.
- Atomic gives fast, lock-free operations but only for specific things like counters or flags.

4.Is AtomicInteger better than synchronized?

Depends on what we are doing. For simple operations like incrementing a number, AtomicInteger is faster. But if we are doing multiple steps together (like check, then update), synchronized gives us a better control.Atomic operations use lock free so they are faster compared to synchronized

5. Can a volatile provide thread safety?

No. volatile does not provide full thread safety. It only ensures visibility of changes across threads, but doesn't make compound actions (like incrementing a counter) atomic.

6. How visibility will be ensured in multithreaded programming. Using volatile

50 Difference between Future and CompletableFuture?

Future and CompletableFuture , both help with asynchronous programming, but they're quite different in how they work.

Real time example:

Imagine you are ordering a pizza.

Scenario A: You went to the restaurant and ordered and wait there on the table. You are not doing anything else. Just sitting and waiting for your order. That is **Future**.
Scenario B: You place the order through an app from home. While it is being prepared, you watch Netflix, take a shower, and the app notifies us when it is done. That is **CompletableFuture**.

Reference: CompletableFuture vs. Future in Java - Java Code Geeks 1: If two APIs return values and you want to process them together?

We can call both APIs using CompletableFuture. Use CompletableFuture.allOf() to wait for both to finish. Once they're done, we can get both results and combine them however we want.

2: How do you run a task asynchronously in Java?

Just wrap the task using CompletableFuture.runAsync() or supplyAsync().

This tells Java to run it in the background thread, so the main thread doesn't get stuck waiting.

51 How Java handles exceptions?

An exception in Java is just an unexpected situation that breaks the normal flow of the program. Java handles exceptions using a structured mechanism that involves keywords: try, catch, finally,

try block: This block encloses the code segment that is susceptible to throwing an exception.

catch block: If an exception occurs within the try block, the corresponding catch block is executed. A try block can have multiple catch blocks to handle different exception types.

finally block: This block is optional and contains code that is guaranteed to execute, regardless of whether an exception occurred in the try block or was caught by a catch block. It is typically used for cleanup operations like closing resources.

```
try {
        BufferedReader reader = new BufferedReader(new
    FileReader("data.txt")); String line = reader.readLine();
        System.out.println(line);
} catch (IOException e) {
        System.out.println("Something went wrong while reading the file.");
    e.printStackTrace();
} finally {
        System.out.println("Cleanup code goes here.");
}
```

Possible Indirect Questions:

1. Difference between Exception vs Error

Exceptions are issues in the code that can be handled; Errors are serious problems like memory leaks the program can't handle.

2. Can we write only try block without catch and finally blocks?

No, a try block must be followed by either catch or finally. Java won't compile without it.

3. Does remaining statements in try block execute after exception occurs?

No, once an exception is thrown, the remaining code in that try block is skipped.

4. What Happens When an Exception Is Thrown by the Main Method? If not caught, JVM prints the stack trace and exits the program.

5. Does a try contain multiple catch blocks?

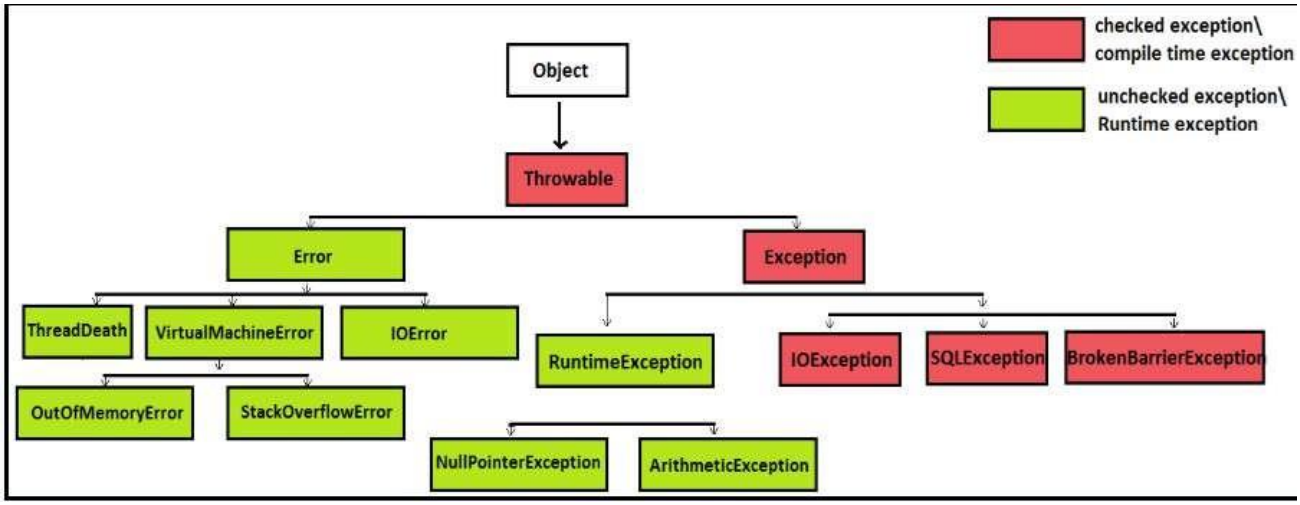
Yes, we can have multiple catch blocks to handle different exception types.

6. What if we return from try, will finally block still execute? Yes, finally will run even if try returns early.

7. When will finally won't execute?

With System.exit(0) Program exists, so finally will not be invoked.

8. Explain Exception hierarchy



At the top:

- **Throwable**: The root of all errors and exceptions in Java.

Two main branches:

- **Error**
  - Serious problems that applications shouldn't try to handle.
  - Examples: OutOfMemoryError, StackOverflowError.
  - Usually caused by the JVM or environment.
- **Exception**
  - Problems an application *can* catch and

handle. Interview Tip:

List out all the exceptions you have faced in your project and explain how to handled and how you fixed the errors.

52 What is the difference between checked and unchecked exceptions?

Checked exceptions: Compiler forces us to handle these.

Checked exceptions are checked by the compiler at compile time, and the programmer must handle them (either by catching them or declaring them in the method signature) or they will cause a compilation error

- IOException: File not found
- SQLException: Database down
- ParseException: Badly formatted input

Unchecked exceptions: we can handle them, but the compiler won't force us.

These are **runtime bugs**. Errors that happen because of mistakes inside the code.

- NullPointerException – Accessing Null
- ArrayIndexOutOfBoundsException – Accessing Invalid index from an Array
- IllegalArgumentException

```
int divide(int a, int b) {
        return a / b; // ArithmeticException if b is 0
}
```

Indirect questions:

1. Why does Java force you to catch IOException but not NullPointerException?

Because IOException comes from things *outside* the code , like reading a file, connecting to a server. We can't always predict if the file will be there or the server will respond. Java says, "This

can fail in real life, so handle it properly."

But NullPointerException? That is the code mistake. we tried to use something that was null. Java cant predict it. So we need to handle explicitly

2. What happens if you don't handle a checked exception? Java just won't compile the code.

53 What is the difference between throw and throws? Throw:

The throw keyword is used to **manually** throw an exception in the code

When throw is executed, the normal flow of execution is immediately terminated, and the control jumps to the nearest catch block that can handle the thrown exception. If no such catch block is found, the exception propagates up the call stack.

Real-World Example:

Imagine We are writing a banking app. If a user tries to withdraw more money than they have, we throw an InsufficientFundsException.

```
public class BankAccount {
        private double balance;

        public void withdraw(double amount) {
                if (amount > balance) {
                        throw new IllegalArgumentException("Insufficient funds!");
                }


                balance -= amount;
        }
}
```

Throws:

throws is used in a method's signature to declare the types of checked exceptions that the method might throw. This informs calling methods that they need to either handle these exceptions (using try- catch) or declare them as well (propagating them further).

Real World Example: We are writing a file reader method. Since files can be missing or corrupted, Java forces us to declare throws IOException to warn callers.

```
public class FileProcessor {
        // Warns that this method might throw an IOException
            public String readFirstLine(String filePath) throws IOException {
        BufferedReader reader = new BufferedReader(new FileReader(filePath));
        return reader.readLine();
        }
}
```

Feature	throw	throws
Purpose	Actually throws an exception (at runtime)	Declares that a method might throw exception(s)
Used in	Method **body** (inside the method logic)	Method **signature** (next to method

		name)	
Number of exceptions		Only **one** exception can be thrown at a time	Can declare **multiple** exceptions, comma-separated
Syntax	throw new ExceptionType("message");	public void method() throws Exception1, Exception2	
When it is used	When you want to manually throw an exception (like validation fails)		When your method deals with checked exceptions and you don't want to handle them inside.

Both checked and unchecked exceptions	Mostly used for **checked** exceptions (for

But for checked exceptions (like IOException), if we throw them, we must declare throws or handle them. Otherwise, Java will throw compile-time error.

1. What happens if I don't declare a throws for a checked exception?

If we don't catch it with try-catch or declare it with throws, our code just won't compile.

2. Can we use throw with multiple exceptions?

No. throw is for throwing a single exception instance at a time.

3. Can you override a method and change the exceptions it throws? Yes, but we can throw narrower checked exceptions.
We can't add new broader checked exceptions in the child class

54 What are custom exceptions and how create it?

Sometimes, Java's built-in exceptions like NullPointerException, IOException, or IllegalArgumentException don't exactly say what went wrong in the project**.**

Real-world Example:

Let's say we are building an e-commerce app, and a user tries to order more than the available stock. There's no OutOfStockException in Java. In such situations we create Custom Exceptions

How to create it?

We create custom exceptions either extending Exception class or RuntimeException.

When to extend Exception vs RuntimeException?

If the exceptional condition is recoverable or expected to be handled by the caller, consider extending **Exception.**

If the exceptional condition is unexpected or typically indicates a programming error, extend **RuntimeException**.

Examples:

extending Exception**:** OutOfStockException

```
public class OutOfStockException extends Exception {
    public OutOfStockException(String message) {
        super(message);
    }
}

public void placeOrder(int quantity) throws OutOfStockException {
        if (quantity > stock) {
                throw new OutOfStockException("Only " + stock + " items left.");
        }
}
```

extending RuntimeException: InvalidAgeException

Let's say we are implementing Registration API where age must be greater than 0. In such scenarios we want to terminate the program by throwing the exception.

```
public class InvalidAgeException extends RuntimeException {
        public InvalidAgeException(String message) {
                super(message);
        }
}

public void registerUser(int age) {
    if (age < 0 || age > 120) {
        throw new InvalidAgeException("Age must be between 0 and 120.");
    }
}
```

Reference: Custom Exceptions in Java. What is a Custom Exception? | by Priya Salvi | Medium

Interview Tip

- Use checked exceptions when the caller must handle the error (e.g., IOException).
- Use unchecked exceptions (RuntimeException) for programming errors that can't be reasonably recovered.
- Use try-catch-finally to handle exceptions and clean up resources.
- Prefer try-with-resources for automatic resource management (e.g., streams, DB connections).
- Create custom exceptions to provide meaningful, domain-specific error messages.
- Always log or rethrow exceptions; never leave catch blocks empty.

55 Explain equals() and hashCode() contract?

The equals method determines whether two objects are equal or not. By default, the implementation in the `java.lang.Object` class compares memory addresses. To compare objects logically, we need to override this method in our class.

The hashCode method returns an integer hash code that represents the object. This is used in hashing-based collections to efficiently locate the objects.

Contract:

- If two objects are equal (equals() returns true), they must have the same hashCode() value.
- If two objects have the same hashCode(), they might still not be equal. it is called a hash collision.

Reference: Java "equals & hashCode" Contract | Design Principles

Understanding the Equals and HashCode Contract in Java | by Kunal Bhangale | Medium

Indirect Questions:

1. Why do we need to override equals and hashcode method?

- Hash Based Collections uses hashCode() first to find the "bucket" and then equals() to check if the object already exists.
- If we override only equals() but not hashCode(), the collection might put the same "equal" object in multiple places because the hash codes differ.
- If we override only hashCode() but not equals(), we might get weird equality results where two objects in the same bucket are still considered different.

```
class Person {
      String name;
      int age;

      Person(String name, int
    age) {
            this.name = name;
            this.age = age;

//    @Override
//       public boolean equals(Object
//       o) { if (this == o) return true;
//          if (!(o instanceof Person)) return false;
//       Person p = (Person) o;
//          return age == p.age &&
//       name.equals(p.name);
//
//    @Override
//    public int hashCode() {
//          return Objects.hash(name,
//       age);
}

public class EqualsHashCode {
                  public static void main(String[] args) {
                                    Person person1 = new
                  Person("CodingLyf", 1); Person person2 = new
                  Person("CodingLyf", 1);

                                    HashSet<Person> set = new
                  HashSet<>(); set.add(person1);
                                    set.add(person2);

                                    System.err.println(set.size())
                  ;
```

Explanation: Here we have two person object that are logically same, same name and same age.

If we don't override equals and hashCode(), Set will store them two times as their hashcodes are different. (we will get set.size() has 2)

If we override equals and hashCode(), Set size will be 1. Means it store the object only once by checking equals and hashcode methods.

2. What if hash function returns same values how it saves values in HashMap?

If two keys have the same hash code, the HashMap puts them in the same bucket (a linked list or a balanced tree from Java 8).
Then it uses the key's equals() method to differentiate them and store/retrieve the right value.

56 What do you know about Collection in Java?

Interview Tip: Collections are the most important part of day-to-day work in real time projects. So, we can expect good number of questions and programs using collections.

Things we need to remember while preparing or working with collections are:

1. Internal working of each of them
2. Performance implications
3. Memory consumption
4. Order of the items
5. Synchronized or not

6. Handling of null values
7. How they maintain sort order or not
8. Differences between them
9. Use cases means, in which scenario we use which collection.

All these points were discussed for each collection in the upcoming questions.

Definition:

In Java, a collection is a framework that provides an architecture for storing and manipulating a collection of objects. In JDK 1.2, a new framework called "Collection Framework" was created, which contains all of the collection classes and interfaces.

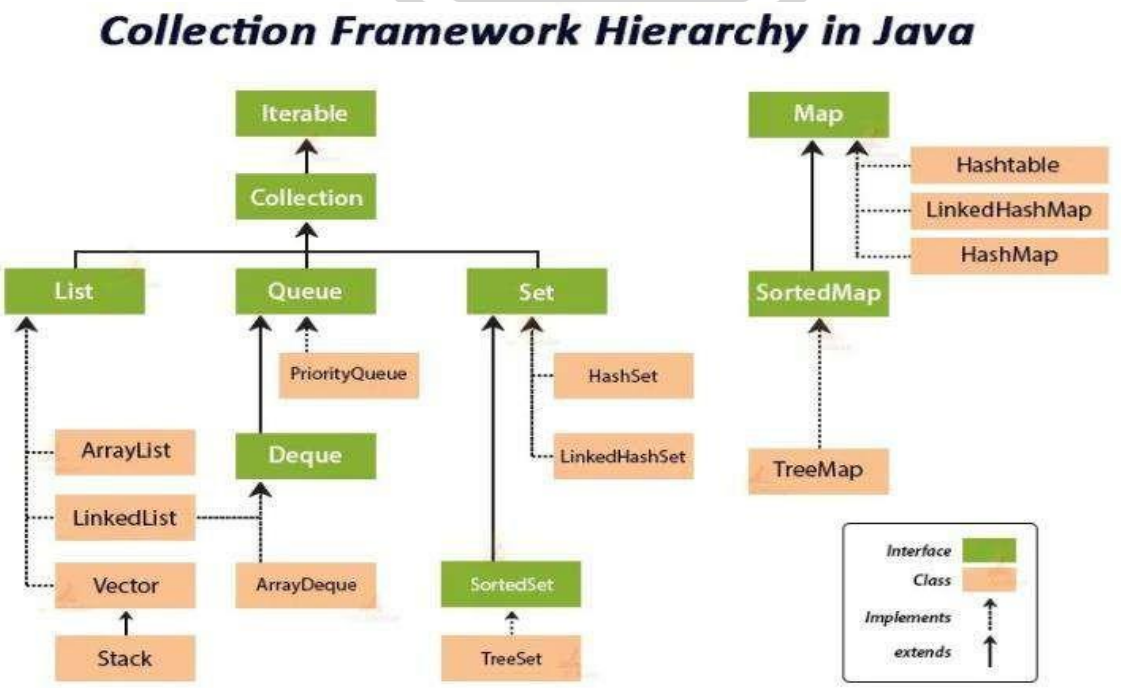
Collection: This is the main interface of the framework. It defines fundamental operations that can be performed on any collection of objects, including adding, removing, and checking for the presence of elements.

In Java, there are **two main building blocks** for storing groups of objects:

1. **Collection interface** (java.util.Collection). It is used for things like Lists, Sets, and Queues.
2. **Map interface** (java.util.Map). It is used for key-value pairs, like a dictionary. Under the **Collection interface**, we have several types:

- **List**, for an ordered list (e.g., ArrayList, LinkedList, Vector)
- **Set**, to stores unique items (e.g., HashSet, LinkedHashSet, TreeSet)
- **Queue / Deque**, for managing elements in a certain order (e.g., PriorityQueue, ArrayDeque) Under the **Map interface**, we have:
- HashMap, TreeMap, LinkedHashMap, etc., which store **key-value pairs**.

So, the **Java Collection Framework** is a group of **interfaces and classes** that helps to work with different types of data structures easily.



Reference: Java Collections

A List is like a list of items. It keeps the order in which we add things. We can store duplicates. We can access items by their position (index).

So, if we add apple, banana, and apple again , it will store all of them in the order we added them.

Set:

A Set is like a bag of unique items. It doesn't allow duplicates. Also, it doesn't care about order (unless we use something like LinkedHashSet or TreeSet).

If we try to add apple, banana, and apple again , it'll silently ignore the second apple.

Map:

A Map is for key-value pairs. Think of it like a dictionary where each word (key) maps to a meaning (value). The keys must be unique, but values can repeat.

List	**Set**	**Map**
Allows duplicate elements	Does not allow duplicate elements	Does not allow duplicate keys (values can be duplicated)
List	**Set**	**Map**
Maintains insertion order	Does not maintain insertion order (except LinkedHashSet)	Does not maintain insertion order (except LinkedHashMap)
Can add any number of null values	Allows at most **one** null value	Allows one null key and multiple null values
Implementation classes: ArrayList, LinkedList	Implementation classes: HashSet, LinkedHashSet, TreeSet	Implementation classes: HashMap, Hashtable, TreeMap, ConcurrentHashMap, LinkedHashMap
Has get(index) method	No get(index) method	No get(index) method, but can get by key
Best when we need index-based access	Best when we need a collection of **unique** elements	Best when we need to store **key-value pairs**
Use ListIterator to traverse	Use Iterator to traverse	Use keySet(), values(), or entrySet() to traverse

Interview Tip:

Always start by explaining the use-case: " use a List when I need order and allow duplicates, a Set when uniqueness is required, and a Map when I need key-value pairs."

Mention common implementations and why: "For List, ArrayList for fast access, LinkedList for frequent insert/remove. For Set, HashSet for speed, TreeSet for sorting. For Map, HashMap for general use, TreeMap for sorted keys."

Tell about performance: "ArrayList get/put is O(1) on average; LinkedList insert/remove is O(1) if at node, but access is O(n). HashMap operations are O(1)."

Always mention real-world scenarios: "Pick LinkedHashMap if I need insertion order, or ConcurrentHashMap for multi-threaded caching."

Indirect Questions

1. Can a List contain null values?

Yes, a List can hold any number of null values.

2. What about Set and Map?

A Set allows only one null value, and a Map allows one null key and many null values.

3. Can a Map contain null keys?

Yes, a Map (like HashMap) can have one null key.

4. What happens when you add the same element twice in a Set? The duplicate is ignored because Set keeps only one copy.

5. How would you ensure only unique usernames are allowed? Store usernames in a Set or use them as keys in a Map.

6. What happens if you try to add a duplicate key in a HashMap? The old value is overwritten with the new one.

7. Why does not collection interface extend the Map interface?

The Collection interface does not extend Map because they represent different concepts. A Collection is a group of individual elements, while a Map is a group of key-value pairs.

58 ArrayList vs LinkedList? Internal Structure:

ArrayList: An ArrayList is backed by a dynamic array. This means it can resize itself as needed, but elements are stored contiguously in memory.

LinkedList: A LinkedList uses a doubly linked list. Each element (node) stores a reference to the next and previous nodes.

Access:

ArrayList : Accessing an element by its index (e.g., get(index)) is very fast (O(1) time complexity) because the array provides direct access to elements.**LinkedList**: Accessing an element by index is slower (O(n) time complexity) because we need to traverse the list from the beginning or end until we reach the desired index.

Insertions/Deletions:

ArrayList: Inserting or deleting elements in the middle of an ArrayList can be slow (O(n) time complexity) because it may require shifting elements to make space or close gaps. Appending or removing from the end is faster.

LinkedList: Inserting or deleting elements anywhere in the list is fast (O(1) on average for insertions/deletions at the beginning or end) because it only involves updating node references.

Memory Usage:

ArrayList: Generally more memory-efficient as it only stores the element data.

LinkedList: Consumes more memory than ArrayList due to the overhead of storing references to the next and previous nodes.

Use Cases:

ArrayList: Best for scenarios where we need frequent random access to elements and modifications (insertions/deletions) are not a primary concern.

LinkedList: Ideal for scenarios where we need frequent insertions or deletions, especially at the beginning or end of the list, and random access is not a primary requirement.

Reference: Difference Between ArrayList and LinkedList in Java | by Ramesh Fadatare | Medium

Indirect Questions:

1. How does Java manage dynamic resizing in ArrayList?

When the internal array is full, ArrayList creates a new array with 50% more capacity.

It then copies all elements into the new array.

2. Why is LinkedList not preferred for random access?

LinkedList stores elements as nodes with references to next and previous.
Accessing an element at index i requires traversal from head or tail , O(n)
time. So it is slow compared to ArrayList's O(1) where we can access via
indexing.

3. If you are inserting an element in the middle of a LinkedList with 10 million items, how will it perform compared to an ArrayList?

LinkedList: Inserting in the middle requires traversing to that position (O(n)) and then updating pointers (O(1)). So overall O(n).

ArrayList: Inserting in the middle requires shifting all elements after that index

(O(n)). For 10 million items:

- Both are O(n), but in practice, LinkedList can be slightly faster for insertion since no shifting of elements is needed, only pointer updates.

Rule:

- Use LinkedList if frequent insertions/deletions in the middle are needed.
- Use ArrayList if random access and iteration speed matter more.

4. If you need to implement a queue that frequently adds and removes elements from both ends, which list would be a better choice and why?

Pick LinkedList over ArrayList.

LinkedList implements Deque, so it is designed for efficient insertions and deletions at both ends in O(1) time.

ArrayList is backed by an array, so adding/removing at the front or middle requires shifting elements, which is O(n).

5. Defining the constructor capacity in ArrayList increases

performance? Yes, it can.

An ArrayList in Java grows dynamically. If we don't set the capacity, it starts with a default size (10) and when more elements are added, it creates a new bigger array (usually 1.5x of old size), copies all
elements into it, and then continues.

This resizing and copying cost time and memory.

So, if we know in advance that we will store, say, 10,000 items, defining the constructor like:

`List<Integer> list = new ArrayList<>(10000);` it saves multiple resizes and improves performance, especially in large datasets.

59 ArrayList vs Vector? Internal Structure

ArrayList and **Vector** both use a **dynamic array** internally.

The difference is,

- **ArrayList** increases size by **50%** when it is full.
- **Vector** increases size by **100%** (it doubles).

So, Vector tends to allocate more memory when resizing. That can lead to slightly fewer resizes but more unused memory.

Access

Access speed is the same for both , **O(1)** for reading any element using .get(index) because both are array-based. So if the main job is reading by index, both perform well.

Insertions/Deletions

- If we insert/delete at the end, both are fast O(1).
- If we insert/delete in the middle or beginning, performance is bad O(n) , because elements have to be shifted.

  So again, they behave similarly here.

Memory Usage

- **ArrayList** is more memory-efficient. It resizes conservatively.
- **Vector** tends to waste memory by doubling its size on every resize. That means if we are tight on memory, go with ArrayList.

Thread Safety: This is the biggest difference.

- **Vector is synchronized** , meaning all its methods are thread-safe.
- **ArrayList is not synchronized** , it is not safe to use with multiple threads unless we handle locking yourself or wrap it with Collections.synchronizedList().

  Synchronized does not mean fast. Vector is thread-safe but slower due to locking. if we need thread safety, we need to use **CopyOnWriteArrayList** or **ConcurrentLinkedQueue** depending on the use case.

60 HashMap vs TreeMap vs LinkedHashMap?

1. Internal Structure

- HashMap: Uses a hash table (like a big index system) to store data.
- LinkedHashMap: Same as HashMap, but also keeps a linked list to remember the order in which items were added.
- TreeMap: Stores data in a sorted tree structure (like a family tree), it keeps keys in order.

2. Order of Elements

- HashMap: No order , items can come in any sequence when we retrieve them.
- LinkedHashMap: Remembers insertion order. Items come out in the same order we put them in.
- TreeMap: Sorts keys automatically (alphabetically or numerically).

3. Performance (Speed)

- HashMap: Fastest for adding, removing, and finding items.
- LinkedHashMap: Slightly slower than HashMap because it also tracks insertion order.
- TreeMap: Slower than HashMap because it keeps keys sorted.

4. Handling Null Values

- HashMap: Allows one null key and many null values.
- LinkedHashMap: Same as HashMap , one null key allowed.
- TreeMap: Does NOT allow null keys , but allows null values.

When to Use Which?

Use HashMap: When we need fast access and don't care about order.
Use LinkedHashMap: When we need insertion order (like a history log). Use TreeMap: When we need sorted keys (like a dictionary).

Indirect Questions:

1. If you want to maintain the insertion order of key-value pairs, which Map implementation would you choose and why?

Use LinkedHashMap. It maintains the order in which we added the keys, so when we iterate, we get the elements in the same order.

2. How would you store user records where the keys must always stay sorted alphabetically?

Go for TreeMap. It automatically keeps keys sorted based on natural order (like alphabetical for strings) or a custom comparator.

3. What would happen if you insert multiple elements with the same hashcode into a Map? How is ordering handled in such a case?

In a HashMap, they go into the same bucket. Order is not guaranteed.

In a LinkedHashMap, they're linked in insertion order even in the same bucket. In a TreeMap, hashcode doesn't matter, only key ordering does.

4. In what scenarios could maintaining a sorted order of keys lead to performance trade-offs?

When using a TreeMap, every put, get, or remove operation takes O(log n) time due to the underlying Red-Black Tree. This is slower than a HashMap's average O(1), but useful when sorted order is required.

5. If you are building a cache that needs predictable iteration order and fast access, which Map would you use and why?

LinkedHashMap is ideal. It offers fast access like a HashMap and maintains order, which is great for things like LRU (Least Recently Used) caching.

```
Map<Integer, String> map = new
HashMap<>();
```

6. How can we make HashMap Synchronized?
7. What are the consequences of non-final fields in a class that is used as a key in a HashMap?

Using non-final fields in a class that is used as a key in a HashMap can cause serious issues because HashMap depends on two things:

1. hashCode(): determines which bucket the key goes into.
2. equals(): determines whether two keys are the same inside that bucket.

Problem:

If the fields used in hashCode() or equals() are **mutable** (non-final), and if we change them after inserting the object into the map, then:

- The key's hashCode() may change, so the object is now in the wrong bucket.
- A lookup using the same key (but with changed values) will fail , we will get null even though the entry exists.

```
class Person {
        String name; // mutable, not final

    Person(String name) {
        this.name = name;
        }

        @Override
        public int hashCode() {
                return Objects.hash(name);
        }

        @Override
        public boolean equals(Object o) {
                if (this == o) return true;
                if (!(o instanceof Person)) return
        false; Person p = (Person) o;
                return Objects.equals(name, p.name);
        }
}

public class HashMapProblem {
            public static void main(String[] args)
        { Map<Person, String> map = new HashMap<>();

                Person p1 = new
        Person("Raju"); map.put(p1,
        "Engineer");

                // Lookup works
        System.out.println(map.get(p1)); // Engineer

                // change
        the key p1.name =
        "Raghu";

                // Lookup fails
```

- When we put p1 in the map, its hash is based on "Raju".
- After changing p1.name to "Raghu", its hash changes but the entry is still stored under the old bucket.
- So, lookup fails, the object exists in the map, but HashMap can't find it anymore.

8. What happens if we have added mutable object to HashMap and change it. Check above answer

61 HashMap vs HashTable?

Synchronization:

HashMap: Doesn't provide thread safety.

HashTable: Provides thread safety by synchronizing all its methods, ensuring that only one thread can access the table at a time.

Performance:

HashMap: Due to the lack of synchronization overhead, HashMap generally offers better performance than HashTable

HashTable: The synchronization mechanism adds overhead, potentially impacting performance.

Null keys and values:

HashMap: One null key and multiple null values are permitted.

HashTable: Does not allow, Attempting to insert null keys or values will result in a NullPointerException.

Iterators:

HashMap is Fail-fast , if it is modified after an iterator is created a ConcurrentModificationException is thrown

HashTable is fail-safe, meaning modifications to the table during iteration will not cause an exception

When to Use Which

Choose HashMap , when performance is critical and thread safety is handled externally or is not a concern.

Choose HashTable , when thread safety is a requirement and performance is less of a priority.

In modern Java, ConcurrentHashMap is often preferred over HashTable for concurrent access scenarios as it provides better performance while maintaining thread safety.

Indirect Questions:

1. Is HashMap thread-safe by default?

No, HashMap is not thread-safe. If multiple threads access it concurrently and at least one modifies it, we can get unpredictable results.

2. Why does Hashtable not allow null keys or values?

To avoid ambiguity during lookups and to keep the behaviour consistent across threads, Hashtable disallows null keys and values.

3. Which is faster: HashMap or Hashtable?

HashMap is faster because it is unsynchronized. Hashtable uses locks on every operation, which slows things down in single-threaded scenarios.

4. What is the difference between Iterator and Enumerator?

Iterator is newer, supports remove(), and is fail-fast. Enumerator is older, does not support removal, and is not fail-fast.

5. When should we use ConcurrentHashMap over Hashtable?

Use ConcurrentHashMap when we need high-performance thread-safe operations. It allows concurrent reads and partial concurrent writes.

6. Why is Hashtable considered a legacy class?

Because it was part of early Java and later replaced by better alternatives like HashMap and ConcurrentHashMap in the Collections Framework.

Reference: Difference Between HashMap and HashTable - Scaler Topics

62 ConcurrentHashMap vs SynchronizedHashmap ?

1) Locking Mechanism:

SynchronizedHashMap maintain object level lock. i.e the whole map is locked..

But in ConcurrentHashMap, the whole map is not locked. The map is divided into number of segments and each segment maintains its own lock.

2) Synchronized operations:

In synchronizedHashMap all operations are synchronized. That means, whatever the operation we want to perform on the map, whether it is read or update, we have to acquire object lock.

But in ConcurrentHashMap, only update operations are synchronized. Read operations are not synchronized.

3) Number of threads

Only one thread can enter into a SynchronizedHashMap.

On the other hand, minimum 16 threads can perform update operations on ConcurrentHashMap at a time

4) Null Keys And Null Values

SynchronizedHashMap allows one null key and any number of null values.
ConcurrentHashMap doesn't allow even a single null key and null values.

5) Nature Of Iterators

Iterators returned by SynchronizedHashMap are fail-fast in nature. i.e they throw ConcurrentModificationException if the map is modified after the creation of iterator.

Iterators returned by ConcurrentHashMap are fail-safe in nature. i.e they DONT throw ConcurrentModificationException if the map is modified after the creation of iterator.

Possible Indirect Questions:

1. What is ConcurrentModificationException when it occurs?

A ConcurrentModificationException happens when we try to change a list (like adding or removing items) while looping through it. This can happen in single-threaded code during iteration or in multi- threaded code

2. How does thread safety work in different map implementations?

HashMap is not thread-safe. Hashtable and Collections.synchronizedMap() lock the whole map, while ConcurrentHashMap uses fine-grained locking.

3. Which map implementation is best for concurrent updates in Java?

ConcurrentHashMap is the best choice. It allows safe concurrent access with better performance than synchronized alternatives.

4. Why is the SynchronizedMap slower during multi-threaded access?

Because it locks the entire map for every operation. This creates contention and reduces performance when multiple threads access it.

5. Does ConcurrentHashMap throw ConcurrentModificationException when modified during iteration?

No, it doesn't. It provides a weakly consistent iterator that doesn't fail if the map is modified during iteration.

6. How to make HashMap thread-safe?

we can wrap it using Collections.synchronizedMap(new HashMap<>()), but it will be slower under high concurrency.

7. What are the alternatives to Hashtable for high concurrency?

Use ConcurrentHashMap. It is faster, non-blocking for reads, and designed specifically for multi-threaded environments.

8. Can we use HashMap in multi-threaded scenario?

The problem arises when there is concurrent update on Hash Map. Hence, during such scenarios, we can use the HashTable or ConcurrentHashMap

9. Can we insert and delete at time same from SynchornizedMap?

Yes, we can insert and delete at the same time from a SynchronizedMap.

Collections.synchronizedMap(...) makes individual operations like put(), get(), and remove() thread-safe by wrapping them in synchronized blocks.

63 Why ConcurrentHashMap won't allow null keys and null values??

In a normal HashMap, we can store one null key and many null values. That is fine because everything happens in a single-threaded context.

But in a ConcurrentHashMap, multiple threads read and write at the same time. If null were allowed, two big problems show up:

The Problem with Null Keys **Null Key in HashMap**

In a normal HashMap, if we put null as a key, Java just keeps it in bucket 0. Since only one thread works at a time, that is fine.

Problem in ConcurrentHashMap

ConcurrentHashMap is different:

- It splits data into many buckets.
- Keys go into buckets based on their hashCode().
- This allows many threads to work on different buckets at the same

time. But with null:

- There's no hashCode() to calculate.
- If they force all null keys into bucket 0, then all threads must fight over that one bucket.
- This slows things down and makes concurrency messy.

The Problem with Null Values

Imagine a scenario where null values are allowed. If we call map.get(key) and it returns null, what does that mean? In a single-threaded HashMap, null could mean either:

1. The key doesn't exist in the map.
2. The key exists, but its value is explicitly set to null.

In a concurrent setting, this ambiguity becomes a nightmare:

- Thread A checks map.get(key) and gets null, assuming the key doesn't exist.
- But Thread B might have just set the value to null using map.put(key, null).

Now Thread A can't tell whether the key is absent or its value is null, leading to race conditions or incorrect logic. By banning null values, ConcurrentHashMap ensures that get() returning null always means the key doesn't exist

Reference: Why Nulls Are Banned: The Surprising Reason ConcurrentHashMap Doesn't Allow Null Keys or Values | by The Code Alchemist | Medium

Additional Question:

1. What is the work around to handle the nulls in ConcurrentHashMap?

Use a Placeholder Object: This is the most common and efficient approach. Create a special, unique object that represents null.

```
public static final Object NULL_VALUE = new Object();
ConcurrentHashMap<String, Object> map = new ConcurrentHashMap<>();

// To "put null"
map.put("key", value == null ? NULL_VALUE : value);
```

```
// To get and check
Object value = map.get("key");
if (value == NULL_VALUE) {
        // Handle the case where you *would* have stored null
} else if (value != null) {
        // Handle actual value
} else {
        // Key is absent
}
```

64 HashSet vs LinkedHashSet vs TreeSet?

1) Ordering:

HashSet: Unordered. Elements are stored based on their hash codes, which means there's no predictable order of iteration.

LinkedHashSet: Maintains insertion order. Elements are stored in the order they were added to the set.

TreeSet: Sorted. Elements are stored in a sorted order, either according to their natural ordering or based on a provided Comparator.

2) Implementation:

HashSet uses a HashMap internally to store elements.

LinkedHashSet uses a LinkedHashMap internally, which maintains a doubly-linked list to track insertion order.

TreeSet uses a TreeMap internally, which is a self-balancing binary search tree.

3) Performance:

Hashset: Generally offers the best performance for add, remove, and contains operations (O(1) on average) due to the efficient hashing mechanism.

LinkedHashSet: Performance is slightly slower than HashSet because of the overhead of maintaining the linked list, but still generally fast (O(1) on average for most operations).

Treeset: Performance is generally slower than both HashSet and LinkedHashSet (O(log n) for add, remove, and contains operations) because of the sorting overhead.

When to use which?

Use HashSet , when we need to store unique elements and don't care about the order they are stored in, and when we need fast performance

Use LinkedHashSet, when we need to store unique elements and need to iterate through them in the same order they were added.

Use TreeSet, when we need to store unique elements in a sorted order.

Indirect Questions

1. How would you maintain insertion order in a Set? Use LinkedHashSet
   Don’t say HashSet, it doesn’t maintain order
2. What if I need to always keep the elements sorted?
   Use TreeSet. It uses Red-Black Tree internally for ordering
3. How would you implement a deduplicated list of elements while keeping insertion order? LinkedHashSet, Because it is a Set (removes duplicates) + maintains order

4. Which Set would perform best for search operations? In most cases, HashSet gives O(1) for contains()
TreeSet takes O(log n)

5. Can I store null in a Set?

HashSet and LinkedHashSet allow one null
TreeSet throws NullPointerException (if using natural ordering)

6. What happens if I insert elements in random order, then iterate the Set?

- In HashSet: iteration order will be unpredictable
- In LinkedHashSet: elements come out in the order they went in
- In TreeSet: elements come out sorted

7. How does TreeSet know how to sort the elements?

It uses compareTo() (natural ordering) Or a custom Comparator if passed during construction

8. Why Treeset does not allow null?

A **TreeSet** compares each element either by comparable or comparator. If TreeSet allows null, then it will throw NullPointerException.

9. What happens if you add elements to a HashSet with duplicate hashCode() values but different equals() implementations?

If two objects have the same hashCode() but equals() returns false, the HashSet will store both objects. Here is why:

- HashSet first checks the bucket using hashCode().
- Within the bucket, it uses equals() to see if the object already exists.
- Since equals() is false, HashSet treats them as different, even though they collide in the hash bucket

Reference: HashSet Vs TreeSet Vs LinkedHashSet In Java

65 Explain about Queue?

A **Queue** in Java is a linear data structure that follows the **First-In-First-Out (FIFO)** principle, meaning the first element added to the queue will be the first one to be removed. Queues are widely used in scenarios like task scheduling, buffering and producer consumer problems.

Since the Queue is an interface, we cannot provide the direct implementation of it.

Queues in Java can be classified into **bounded** and **unbounded** implementations based on their capacity.

1. **Unbounded Queue**: Can grow dynamically (no fixed size).
2. **Bounded Queue**: Has a fixed maximum capacity.

66 Explain different types of Queue? Types of Queue

Implementations in Java

Queues in Java can be categorized into two main types based on their capacity:

1. **Unbounded Queue**: Can grow dynamically (no fixed size).
2. **Bounded Queue**: Has a fixed maximum capacity.

1. Bounded Queues

A bounded queue has a fixed size. Once it is full, we can't just keep adding elements, either the operation fails, or it waits until space is available.

- **ArrayBlockingQueue**: It Uses an array with a fixed capacity. It also supports fair ordering if we want threads to access in a strict first-come, first-served manner.
- **LinkedBlockingQueue**: Works like a linked list and we can set the capacity. If we don't, it defaults to a very large size (Integer.MAX_VALUE).
- **PriorityBlockingQueue**: Instead of strict FIFO, it orders elements based on priority (either natural ordering or a custom comparator).

2. Unbounded Queues

Unbounded queues don't force a hard capacity limit. They'll grow as needed, limited only by the available memory.

- **PriorityQueue**: Keeps elements ordered by natural order or a comparator, but it is not thread-safe.
- **ConcurrentLinkedQueue**: A thread-safe option that works well in highly concurrent scenarios.
- **LinkedList (as Queue)**: LinkedList also implement Deque. So, it can be also used as Queue.

67 When will you use each queue?

Interviewer either give a scenario or you need to think of a scenario as an example

ArrayBlockingQueue : For fixed-size producer-consumer problems. For example, a logging system where producers (threads writing logs) and consumers (threads processing logs) should (Refer Question No: 181: Java Scenario 8: Producer Consumer Problem).

LinkedBlockingQueue:

Useful when we want flexibility in size (bounded but large by default). For example, handling requests in a web server where we may want to queue thousands of tasks.

```
BlockingQueue<Runnable> taskQueue = new LinkedBlockingQueue<>();

// Add tasks
taskQueue.offer(() -> System.out.println("Task 1 executed"));
taskQueue.offer(() -> System.out.println("Task 2 executed"));

// Execute
Runnable task = taskQueue.poll();
if (task != null) task.run();
```

PriorityBlockingQueue

When tasks have priorities, like a hospital system where emergency patients must be treated before routine checkups(Refer Question No:).

PriorityQueue: When we need ordered processing but don't care about thread safety

ConcurrentLinkedQueue: When multiple threads are adding and removing without blocking.

LinkedList (as Queue): For simple, single-threaded use cases where we just need FIFO behavior.

68 How ArrayList works internally?

ArrayList in Java works internally by using a dynamic array to store its elements. This means it is backed by an Object[] array, which holds the actual data.

```
List<Integer> list = new ArrayList<>(10000);
```

1. Initial Capacity

When we create an ArrayList, it starts with a default capacity , which is usually 10, unless we give a specific value: `List<String> list = new ArrayList<>(); // default capacity = 10`

Capacity here means: how many elements can be stored before the array has to grow.

2. Capacity vs Size

- Capacity -> Total slots available in the internal array
- Size -> How many elements we have added so far

  Let's say we created a list with default capacity 10, and added 3 elements then

  Capacity = 10 and Size = 3

3. How it Grows

When we keep adding items and the array runs out of space, it automatically grows like this:

- A new array is created with more space (typically 1.5x the old size)
- All elements are copied from the old array to the new one
- The old array is discarded

  This resizing is done inside a method called

  grow(). Example:

  Old capacity = 10 and new capacity = 15

  This resizing takes time, so adding many items at once can temporarily slow things down.

4. Adding Elements

When we call add(element):

- It first checks if there's room in the array
- If yes -> Just put it at the next available index
- If no -> Resize the array, then add

  So, add () is usually O(1) time, but occasionally it can be slower when resizing happens.

5. Accessing Elements

We can access any element by its index using .get(index)
This is super-fast; O(1) time

6. Inserting or Deleting in the Middle

Here is the catch:

- If we insert or remove from the middle, it has to shift elements to keep the order.
- That takes time; O(n) time

```
list.add(2, "hello"); // shifts everything from index 2 onward
list.remove(2); // also shifts everything back
```

All the possible Questions on Array List:

ArrayList Interview Questions in Java - Scientech Easy

69 How HashMap works internally?

I would suggest you go through the below videos before going to the theory.

Reference: How HashMap Internally Works in Java With Animation | Popular Java Interview QA | Java Techie

Map and HashMap in Java with Internal Working- Interview Question

HashMap works by using a hash function to map keys to indices in an internal array (buckets).

When a key-value pair is added, the hash code of the key is used to determine which bucket to store it in. If multiple keys hash to the same index, a linked list (or a balanced tree in Java 8+ for larger lists) is used to store these collisions.

1. Hashing:

When we insert a key-value pair (e.g., put(key, value)), the hashCode() method of the key object is invoked. The result, along with the current size of the HashMap, is used to calculate an index within the internal array.

2. Collision Handling:

If multiple keys generate the same hash code (a hash collision), they are stored in the same bucket. In older versions of Java (before 8), this was handled using a linked list. In Java 8 and later, if the number of elements in a bucket exceeds a threshold (usually 8), the linked list is converted to a balanced tree (red- black tree) for better performance.

3. Retrieval:

When we retrieve a value using get(key), the same hashing process is used to determine the correct bucket. If the key is not directly found in the bucket, the linked list or tree (depending on the implementation) is traversed to find the key-value pair.

4. Resizing:

If the HashMap becomes too full (exceeds a load factor threshold, typically 0.75), it will resize its internal array to a larger size, typically doubling it. This process, called rehashing, involves recalculating the hash codes and re-distributing the elements into the new array, which can be a costly operation.

Key Concepts:

hashCode(): A method that returns an integer representation of an object. It is crucial for determining the bucket index.

equals(): Used to compare keys for equality when collisions occur. If hashCode() returns the same value for two keys but equals() returns false, it signifies a collision.

Load Factor: A threshold that determines when the HashMap should resize.

Indirect Questions:

1. What is the load factor in HashMap?

HashMap's performance depends on two things first initial capacity and second load factor. whenever we create HashMap initial capacity number of the bucket is created initially and load factor is the criteria to decide when we have to increase the size of HashMap when it is about to get full. A load factor is a number that controls the resizing of HashMap. When a number of elements in the HashMap

cross the load factor as if the load factor is 0.75 and when becoming more than 75% full then resizing trigger which involves array copy.

2.How does resize happens in HashMap?

The resizing happens when the map becomes full or when the size of the map crosses the load factor. For example, if the load factor is 0.75 and then becomes more than 75% full, then resizing trigger, which involves an array copy. First, the size of the bucket is doubled, and then old entries are copied into a new bucket.

3. What is the difference between the capacity and size of HashMap in Java?

The capacity denotes how many entries HashMap can store, and size denotes how many mappings or key/value pair is currently present.

4. Apart from String, what else predefined class we can use as Keys in A Map?

Wrapper classes: Integer, Long, Double, Float, Boolean, Character

5. If a poorly designed hash function causes all elements to collide in a HashMap, what would be the performance impact?

Normally, HashMap lookup is O(1) on average. But If all elements collide, every key ends up in the same bucket. But with collisions, it degrades to O(n) because it must scan through a linked list. That is why Java 8 improved this by switching to a balanced tree after many collisions

70 How HashSet works internally?

I would suggest you go through the below video before going to the theory.

Reference: Core JAVA: How does HashSet work internally? Implementation | Why are its elements unique?

HashSet uses a **HashMap** behind the scenes to store its elements. Here is how it actually works:

1. HashMap is the backbone

When we add something to a HashSet:
- That value is stored as a **key** in an internal HashMap.
- The **value** for that key is just a constant dummy object (called PRESENT).
- HashMap doesn't allow duplicate keys, this is what keeps all HashSet elements unique.

2. How it decides where to store

When we do set.add("apple"):

- Java calculates the **hashCode** of "apple".
- That hash is used to pick a **bucket** inside the HashMap.
- If the bucket already has something, Java checks using equals() to see if "apple" is already there.
    - If yes, it won't add it again.
    - If no, it stores it alongside the others (usually using a list or tree).

3. What happens with duplicates

If we try to add an element that is already in the set:

- The underlying HashMap.put() will return the old dummy value.
- That tells HashSet the item was already there.
- So, add() returns false.

4. Why it matters

- HashSet depends completely on hashCode() and equals() of the objects we add.
- These two methods determine where the object is placed and whether it is considered a duplicate.

71 How LinkedHashMap and LinkedHashSet works internally? I would suggest you go through the below videos.

LinkedHashMap and LinkedHashSet: LinkedHashMap and LinkedHashSet in Java | Internal Working

LinkedHashMap : How does LinkedHashMap work internally? | VS Hashmap internals? | LRU cache | Is it synchronized?

72 How TreeMap works internally?

I would suggest you go through the below video.

🌳 How TreeMap Works Internally in java | #programmingkt #java #javatutorial #javatutorialforbeginners

The java.util.TreeMap class in Java's Collections Framework is an implementation of a Red-Black Tree. It provides a sorted map where elements are ordered by their natural ordering of keys, or by a custom Comparator.

When we don't provide a Comparator, TreeMap uses the natural ordering of keys. That means the key class must implement Comparable interface.

Visual of a Red-Black Tree

Imagine this TreeMap with numbers as

keys: [10 ♥]

/ \

[5 🟢] [15 🟢]

/ \

[12 ♥] [20 ♥]

Legend:

- 🟢 = Red node
- ♥ = Black node
- Each node = key-value pair
- Always follows Red-Black tree rules (Below are the rules good to know)

  - **Node color**: Every node is either red or black.
  - **Root rule**: The root is always black.
  - **Leaf rule**: All leaves (null children, considered "NIL" nodes) are black.
  - **Red rule**: If a node is red, both its children must be black (no two reds in a row).
  - **Black-height rule**: Every path from a node to its descendant NIL leaves must have the same number of black nodes.

How TreeMap Maintains Sorted Order? Natural Ordering

If we just write:

TreeMap<Integer, String> map = new

TreeMap<>(); The keys will be sorted in **increasing order**: 1, 2, 3, 4...

Custom Ordering

We can also pass our own logic:

TreeMap<String, String> map = new

TreeMap<>(Comparator.reverseOrder()); Now keys will be sorted in **reverse**.

Operations:

Insertion: put(key, value

Let's say this is current tree:

[10 ♥]
/ \
[5 ●] [15 ●]

Now if we insert 1 It goes left of 5

[10 ♥]

/ \
[5 ●] [15 ●]
/
[1 ●]

But now 5 and 1 are both red. That **breaks the red-red rule**, so the tree **recolors and rotates**:

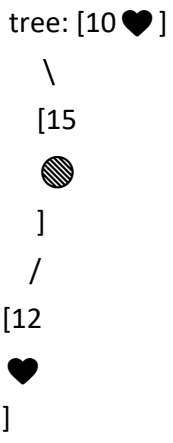
[10 ♥]
/ \
[5 ♥] [15 ♥]
/
[1 ●]

It stays balanced

Search: get(key)

To find a key, TreeMap walks down the tree:

To find 12 in this

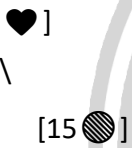


- 12 > 10 , so go right
- 12 < 15 , so go left
- Found it

Only takes a few steps thanks to the balanced structure.

Deletion: remove(key)

If we remove a node, like 10:



TreeMap finds the in-order successor (like 12 or 15), replaces 10 with it, then **re-balances** the tree by rotating/recolouring.

Indirect Questions:

1. Will TreeMap allow key or value?

- TreeMap don't allow null as key but allow null as value

2. Why TreeMap wont allow null key and what will happened if we add null as a key?

- TreeMap maintains its keys in sorted order, To maintain this sorted order, TreeMap needs to compare keys with each other. A null value cannot be compared with a non-null value using these comparison methods. Attempting to do so results in a NullPointerException

- Same with TreeSet as well

73 How TreeSet works internally?

A **TreeSet** in Java stores elements in **sorted order**.

Behind the scenes, it actually uses a **TreeMap** to do all the work. When we create a TreeSet, Java internally creates a TreeMap, and every element we add to the set is stored as a **key** in that

TreeMap. TreeSet implements two important interfaces:

- SortedSet: keeps elements sorted
- NavigableSet: lets we navigate the set (like getting higher, lower, ceiling, floor

values) So even though it behaves like a HashSet (no duplicates), the difference is:

HashSet uses a HashMap (no order).

TreeSet uses a TreeMap (sorted

order).

Reference: TreeSet internal working &treeset internal implementation in java-JavaGoal

74 How ConcurrentHashMap works internally?

Video 1 : Internal working of Concurrent HashMap & Interview Questions - JAVA | Concurrent Collections

Video 2: ConcurrentHashMap internal working in java | ConcurrentHashMap internal implementation in java

Reference: How ConcurrentHashMap Works Internally After Java 8?

It is a special kind of Map in Java that allows multiple threads to read and write at the same time without messing up the data.

Before Java 8: Segment-Based

- The map was divided into smaller pieces called segments.
- Each Segment can contain number of buckets.
- Each segment had its own lock.
- If two threads worked on different segments, they could proceed without waiting.
- But still, locking was needed per segment, which made things a bit complex.

. From Java 8 Onwards: Bucket-Based Locking

- Segments were removed.
- Now the map is divided into buckets (like in HashMap).
- Each bucket holds entries (either a linked list or tree).
- Instead of locking the whole map, it only locks the specific bucket being updated.
- This means more threads can work in parallel.

CAS (Compare-And-Swap)

- For small updates like adding or changing a value, it may skip locking altogether.
- It uses CAS, a technique to check and change a value in one atomic step.
- This reduces thread fighting and improves speed.

Internal Operations

Adding or Updating an Entry

- Calculate the hash of the key to find which bucket it belongs to.
- If that bucket is empty, just insert the new entry , so no lock needed.
- If the bucket is not empty:
    - Lock only that bucket.
    - Check if the key exists:
        - If yes, update the value.
        - If not, add the new entry.
    - Then release the lock.
    - In simple cases, CAS may be used instead of locking.

Reading an Entry

- No locking is needed for reads.
- Thanks to special memory rules (volatile), the reading thread will see the most up-to-date value.

Removing an Entry

1. Find the right bucket using the key's hash.

2. Lock that specific bucket.
3. Remove the entry.
4. Release the lock.

Indirect Question:

1. Why ConcurrentHashMap better than HashMap + synchronized?

Because it doesn't lock the entire map. so threads don't block each other . That means better speed and performance.

2. How to make a List as Read Only List?

An ArrayList can be made read-only easily with the help of **Collections.unmodifiableList()** method. This method takes the modifiable ArrayList as a parameter and returns the read-only unmodifiable view of this ArrayList

75 Iterator vs List Iterator?

Both Iterator and ListIterator are used to traverse collections in Java.

Supported Collections:

- Iterator can be used with any collection like List, Set, Queue, or Map.
- ListIterator works only with List collections like ArrayList, LinkedList

Direction:

- Iterator allows only forward traversal of a collection.
- ListIterator supports both forward and backward traversal.

Real-World Example:

Imagine scrolling through a photo gallery.

- With Iterator, we can only hit "Next".
- With ListIterator, we get both "Next" and "Previous".

Modification:

- Iterator can remove elements during iteration.

```
  //Iterator example - one direction only
  List<String> names = new ArrayList<>();
  names.add("Dhoni");names.add("Virat");
  Iterator<String> iterator = names.iterator();

    while(iterator.hasNext()) {
    System.out.println(iterator.next());
  }
//ListIterator example - both directions
ListIterator<String> listIterator = names.listIterator();

while(listIterator.hasNext()) {
              System.out.println("Forward: " +
       listIterator.next());
}

while(listIterator.hasPrevious()) {
              System.out.println("Backward: " +
```

- ListIterator can add, remove, or modify elements during iteration.

Starting Point:

- Iterator always starts at the beginning of the collection.
- ListIterator can start at any specific index of a list.Additional

Features:

- Iterator: Simpler, with basic methods like hasNext(), next(), and remove().
- ListIterator: Provides extra methods like hasPrevious(), previous(), add(), and set().

When to Use:

Iterator: Use for simple traversal of any collection.

ListIterator: Use for advanced list traversal when we need both directions or to modify the list.

Indirect Questions:

1. What is the best way to traverse a Set, and why doesn't ListIterator work for it?

Use Iterator because Set doesn't maintain index order, and ListIterator needs positional access which only Lists provide.

2. How do you safely remove elements while looping through a collection?

Use the Iterator's remove() method right after next() to avoid ConcurrentModificationException.

3. What would you use to insert an element while iterating?

Use ListIterator.add() which inserts right after the current position in a List.

4. How do you iterate backward through a list?

Use ListIterator and loop with hasPrevious() and previous() methods.

5. What happens if you try to modify a collection during iteration without using the right tool?

We'll get a ConcurrentModificationException because the collection detects structural changes mid- iteration.

6. What collection types restrict you from using a ListIterator, and why?

Set, Queue, and Map don't support ListIterator because they don't guarantee indexed order.

7. When should you use Iterator over ListIterator in real-world Java applications?

Use Iterator when working with Sets or non-indexed collections; it is simple and universal**.**

76 Fail-Fast vs Fail-Safe? Definition:

Fail-Fast throws a ConcurrentModificationException if a collection is modified during iteration (other than via the iterator).

Fail-Safe does not throw exceptions during iteration, as it works on a copy of the collection or uses special mechanisms to handle changes.

Examples:

Fail-Fast: ArrayList, HashMap, HashSet, LinkedList (use their standard iterators).

Fail-Safe: CopyOnWriteArrayList, ConcurrentHashMap.

How They Work:

Fail-Fast detects structural modifications like adding or removing elements using a **modCount** variable, which is checked during iteration.

Fail-Safe operates on a copy of the original collection, so modifications do not affect the iterator.

Performance:

Fail-Fast is Faster, as it directly iterates over the original collection.

Fail-Safe is Slower, as it creates a separate copy or snapshot of the collection.

Use Cases:

Use Fail-Fast when thread safety is not required, and we want errors to be caught immediately (example, single-threaded applications).

Use **Fail-Safe** in multi-threaded environments where collections may be modified concurrently.

77 How FAIL-FAST iterator works? why it throws concurrent modification exception?

Fail-fast iterators directly **work the current state of the collection**.

Fail-fast iterators check if the collection has been modified while iterating. If it detects a change, it immediately throws a ConcurrentModificationException.

How it works:

1. The collection keeps a modCount (modification count) that increases with every change (add, remove, etc.).
2. When an iterator is created, it saves the current modCount as expectedModCount.
3. Before each operation , the iterator checks if modCount == expectedModCount.
4. If they don't match, it means the collection was modified, and the iterator throws an

exception. Example:

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");

for (String s : list) {
	list.add("C"); // Throws
	ConcurrentModificationException System.err.println(s);
}
```

- when the iterator is created , modCount = 2.
- Inside the loop, list.add("C") changes modCount to 3.
- The iterator checks and sees modCount != expectedModCount
- ConcurrentModificationException

Indirect Questions:

1. Which types of iterators consume more memory; fail-fast or fail-safe?

Fail-safe iterators consume more memory. They work on a copy of the collection, so any changes to the original don't affect iteration. Fail-fast iterators, on the other hand, directly iterate the collection and throw ConcurrentModificationException on modification, so they use less memory.

78 How FAIL-SAFE iterator works? why it does NOT throw concurrent modification exception?

Fail-safe iterators don't throw ConcurrentModificationException because they don't operate on the actual collection. Instead, they work on a copy of the collection's data, taken at the moment the

iterator was created.

How it works:

- When we call .iterator() on a fail-safe collection (like CopyOnWriteArrayList or ConcurrentHashMap), it creates a snapshot (copy) of the data.
- The iterator reads from this snapshot, not the live collection.
- That is why modifications to the original collection during iteration don't throw ConcurrentModificationException.

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("A");
list.add("B");

for (String s : list) {
            list.add("C"); // No
      ConcurrentModificationException System.out.println(s); //
      Prints only A, B
```

Here, list.add("C") modifies the original list, but the iterator only print "A" and "B", since it works on copy.

79 Comparable and comparator, explain their differences?

In Java, when we want to **sort a list of objects**, Java needs to know **how to compare** those objects.

Primitive types like int, double, Java knows how to compare.

But if we have own own class, like **Student**, Java doesn't know if we want to sort by **marks**, **name**, or **age**.

To solve this, Java gave us two interfaces:

- Comparable
- Comparator

Comparable is an interface in Java that enables **comparing an object with other objects of the same type**. Comparable interface is provided by the java.lang package. Several built-in classes in Java
like Integer, Double, String, etc., implement the Comparable interface.

Comparable is used for sorting objects by **natural or default ordering**

```
public class Student implements Comparable<Student> {
        int marks;

        public int compareTo(Student s) {
                return this.marks - s.marks; // ascending order
        }
}
```

The **Comparator** interface is used to define multiple ways of sorting objects. It is found in the java.util package and has two primary methods: compare() and equals(), although equals() is rarely overridden.

```
class NameComparator implements Comparator<Student> {
        public int compare(Student a, Student b) {
                return a.name.compareTo(b.name);
        }
}
```

Interview Tip:

Use Comparable for **default sorting**. Use Comparator when we need **custom sorting**. Also mention, Java uses them in sorting and ordering, like in Collections.sort(), TreeMap, or TreeSet. Without them, Java won't know how to compare objects. TreeMap in Java relies on either the Comparable interface or
a Comparator

Reference: Difference between Comparable and Comparator in Java - Scaler Topics

Comparable: JavaSamples/ComparableExample.java at main · CodingLyf-Fullstack/JavaSamples Comparator: JavaSamples/ComparatorExample.java at main · CodingLyf-Fullstack/JavaSamples **Indirect Question:**

1. If a class defines its own natural ordering using Comparable, but I also provide a custom Comparator while sorting, which ordering will be applied?

If a class implements Comparable, that defines its natural ordering (say by ID or name). But if we pass a Comparator explicitly to a sort method (Collections.sort(list, comparator) or stream.sorted(comparator)), the Comparator takes precedence.
So, natural ordering is used only when no Comparator is given.

80 List.of() vs Collections.unmodifiableList? List.of()

(Java 9+)

- Creates a **truly immutable** list (cannot be modified in any way).
- If we try to modify it (add, remove, set), it throws an UnsupportedOperationException.
- **Nulls are not allowed** (throws NullPointerException if we try to add null).
- More memory-efficient (optimized for small lists).

```
List<String> immutableList = List.of("A", "B", "C");
immutableList.add("D"); // Throws UnsupportedOperationException
```

Collections.unmodifiableList()

- Wraps an existing list and makes it **unmodifiable** (but the original list can still change).
- If the original list changes, the unmodifiable view also reflects those changes.
- **Allows** null (if the original list had null).

```
List<String> originalList = new ArrayList<>(List.of("A", "B", "C"));
List<String> unmodifiableList = Collections.unmodifiableList(originalList);

unmodifiableList.add("D"); // Throws UnsupportedOperationException
originalList.add("D");    // This works! Now unmodifiableList = ["A", "B", "C", "D"]
```

81 What do you know about generics in Java?

When Java first introduced collections, they were powerful but also dangerous. we could put anything inside a List or Map, and the compiler wouldn't complain.

```
List students = new ArrayList();
students.add("Dhoni");
students.add("Kholi");
students.add(123); //Inserting number here
```

But when we tried to take things out and cast them to the right type, the program would crash at runtime with a ClassCastException.

```
String name = (String) students.get(2);
// Runtime error: ClassCastException
```

Generics were introduced in Java 5 to solve this exact problem. Generics in Java provide a way to write classes, interfaces, and methods that operate on objects of various types while maintaining type safety.

How Generics Work Type Parameters

Generics introduce placeholders for types, usually written as single capital letters like T for Type, E for Element, K for Key, V for Value. These placeholders sit inside angle brackets (<>).

Generic Classes

We can define a class that works with any type. For example, a simple Box class:

```
class Box<T> {
        private T value;


        void set(T value) {
                this.value = value;
        }


        T get() {
                return value;
        }
}
```

Now, when we create a Box, we specify what type it holds:

```
Box<String> stringBox = new Box<>();
stringBox.set("Hello Generics");
System.out.println(stringBox.get()); // Hello Generics


Box<Integer> intBox = new Box<>();
intBox.set(42);
System.out.println(intBox.get()); // 42
```

Generic Methods

We can also make methods generic. The type parameter goes before the return type:

```
public <T> T printAndReturn(T data) {
System.out.println(data);
    return data;
}
```

Now the method works with any type:

```
printAndReturn("Java");   // prints Java
printAndReturn(123);      // prints 123
```

Bounded Type Parameters:

We can restrict the types that can be used for a type parameter using the extends keyword.

```
public <T extends Number> void process(T
num) {
System.out.println(num.doubleValue());
```

Now process() will only accept Number or its subclasses (Integer, Double, etc.).

Wildcards

What if we don't know the exact type but still want flexibility? That is where wildcards (?) come in

```
List<? extends Number> numbers;
```

This means the list can hold Integer, Double, or any subclass of Number.

Important Thing to Know

- **Primitives don't work**: We can't use int or char directly with generics. Instead, we use wrapper classes like Integer and Character.

82 What is Garbage Collection?

Garbage collection in Java means Java automatically removes objects from memory when they're no longer needed.

If the program doesn't use an object anymore and nothing refers to it, Java's Garbage Collector will clean it up to free memory. We don't have to delete it manually. Java takes care of it in the background.

How It Works:

- **Identifies Garbage:** The GC scans memory and marks objects that are unreachable (no longer referenced by any part of the program).
- **Removes Garbage:** It deletes these unused objects, reclaiming memory.
- **Compacts Memory:** Some GC algorithms also rearrange remaining objects to reduce fragmentation.

83 What are the different types of Garbage Collectors?

1. Serial Garbage Collector

This is the most basic type of GC.

It uses a single thread for garbage collection

2. Parallel Garbage Collector (also called the Throughput Collector)

This GC uses multiple threads to perform garbage collection.

It focuses on high overall performance (high throughput), even if that means longer pause times.

3. Concurrent Mark-Sweep (CMS) Collector

This GC tries to reduce pause times by doing most of its work concurrently with the application. It is designed for applications that need quick responses, like web servers or GUI applications.

Important Note:

CMS is deprecated starting from Java 9 and removed in Java 14. It was popular for low-latency applications before G1 GC became the default.

4. G1 Garbage Collector (Garbage First)

G1 is the default collector in Java from version 9 onwards.
It divides the heap into small regions and prioritizes collecting regions with the most garbage first.

5. ZGC (Z Garbage Collector)

These are modern, advanced GCs designed for very large heap sizes (often 10 GB or more).

They are capable of ultra-low pause times (often under 10 milliseconds), even as memory usage grows.

84 How Garbage Collections works internally?

Java Garbage Collectors implement a *generational garbage collection strategy* that categorizes objects by age

Step by Step Process from Object creation to object deletion from memory

1. **Heap**: When we create an object in Java using new keyword, it goes to the heap. The heap memory in the JVM is divided into generations.

Young Generation and Old Generation.

2. **Young Generation** again divided into 3 section : Eden Space, FromSpace (S0) and ToSpace (S1) Eden space - all new objects start here, and initial memory is allocated to them
Survivor spaces (FromSpace and ToSpace) - objects are moved here from Eden after surviving one garbage collection cycle. When objects are garbage collected from the Young Generation, it is a minor garbage collection(Minor GC) event.

3. Old Generation: Objects that are long-lived are eventually moved from the Young Generation to the Old Generation. This is also known as Tenured Generation, and contains objects that have remained in the survivor spaces for a long time. Here Major GC event occurs to clear the objects.
Major GC uses
Marks and Sweep Algorithm.

Mark: The garbage collector "marks" all reachable objects, starting from root nodes.

Sweep: After marking, its "sweeps" through memory to delete objects that were not marked as

reachable, freeing up space.

Reference: Garbage Collection in Java – What is GC and How it Works in the JVM

https://medium.com/@rakeshrdy8/garbage-collection-in-java-explained-bbebdc5f75ac

9. Java Memory Management and Garbage Collection in Depth (Video)

Indirect Questions:

1. Where are the objects created, in heap or stack?

Objects are always created in the heap. Stack holds method calls and references, not the actual objects.

2. Which part of the memory is involved in Garbage Collection, Stack or Heap?

Garbage Collection only works on the heap. The stack is managed separately by the JVM.

3. Who manages the Garbage Collector?

The JVM handles Garbage Collection internally. Developers don't control it directly.

4. How to request Garbage Collection explicitly?

We can call System.gc() or Runtime.getRuntime().gc(), but JVM may ignore the request.

5. When does an object become eligible for Garbage Collection?

When there are no more references pointing to it. Basically, it becomes unreachable.

6. Different ways to make an object eligible for Garbage Collection? Set its reference to null, reassign it, or let it go out of scope.

1. Setting reference to null

```
String s = new String("Hello");
s = null; // Now "Hello" is eligible for GC
```

2. Reassigning the reference

```
String s = new String("Java");
s = new String("Python"); // "Java" becomes unreachable
```

7. What is generation strategy?

It is a JVM optimization that divides the heap into young, old, and sometimes permanent spaces. Most objects die young, so it collects young generation more often.

8. What is HotSpot?

HotSpot is the JVM implementation from Oracle. It uses Just-In-Time (JIT) compilation and smart Garbage Collection strategies for better performance.

9. How does Metaspace is different from PermGen?

PremGen is part of Heap memory, It contains in meta data of the class. It is fixed in size. From Java 8 Metaspace is introduced which is not fixed in size.

85 Difference between WeakReference and SoftReference?

WeakReference: The garbage collector clears the object as soon as there are no strong references to it, even if there's plenty of memory available. It is like telling the GC, "If nobody else is holding it, just clear it from memory."

SoftReference: The garbage collector keeps the object around as long as there's enough memory. It is only cleared when memory starts running low. Think of it as saying, "Keep it if you can, but drop it when space gets tight."

Code link: JavaSamples/ReferenceDemo.java at main · CodingLyf-Fullstack/JavaSamples

Example: **WeakHashMap** uses weak references for its keys, so if a key is no longer strongly referenced elsewhere, the entry will be removed automatically during GC.

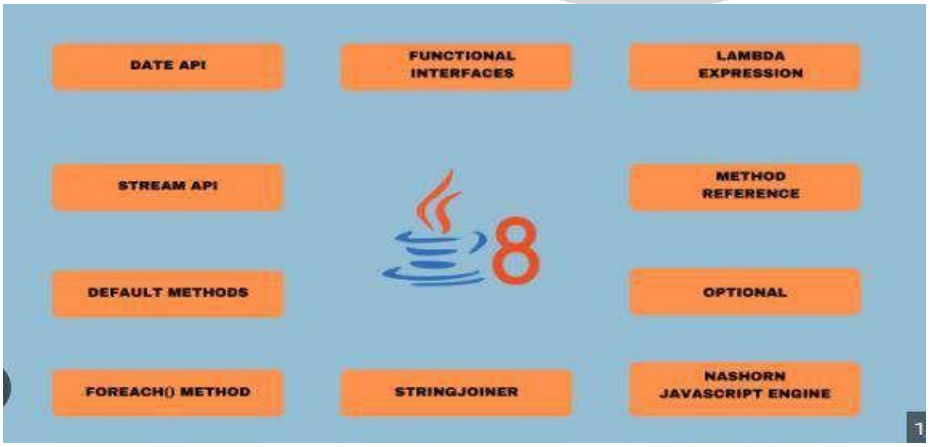
```
public class WeakHashMapExample {
            public static void main(String[] args) {
                        Map<Object, String> map = new WeakHashMap<>();

                        Object key1 =
            new Object(); Object key2 =
            new Object();

                        map.put(key1, "Value
            for key1"); map.put(key2, "Value
            for key2");

                        System.out.println("Before GC: " + map);

                        // Remove strong
            reference to key1 key1 = null;

                        System.gc(); //
            Suggest GC
            System.runFinalization();

                        System.out.println("After GC: " + map);
            }
}
Output:
```

What happened here

- **key1** had NO strong reference after key1 = null, so during GC the weak reference in WeakHashMap was cleared, and its entry disappeared automatically.
- **key2** was still strongly referenced, so its entry stayed.

86 Can you list out the Java 8 features?

Below image contains Java 8 features. These features are very important to prepare for the interviews. We will go through one by one.



Interview Tip: No need to mention all the features Start with 3–4 important ones, explain how they help in real projects. The major ones are lambdas, streams, functional interfaces, and default methods. We used lambdas and streams heavily when working on data transformations and in business logics.

87 What are default methods and why are they are introduced?

A default method allows to add a new method to an interface without breaking the classes that already implement it.

Before Java 8, interfaces can only have abstract methods. So, if we add a new method to an interface, every class that implemented it has to override that method, or we will get a compile-time error. That was a huge headache, especially for large codebases or libraries where interfaces were implemented in hundreds of places.

To fix this, Java introduced default methods. Now, when we add a new method to an interface, we can also provide a default implementation. In this way, existing classes won't break, they can either use the default or choose to override it.

The main goal here is backward compatibility.
Example:

The stream() method was added as a **default method** in the java.lang.Iterable interface in Java 8:

```
default Stream<E> stream() {
        return StreamSupport.stream(spliterator(), false);
}
```

Now any class that implements Iterable, like List, Set, or Queue, automatically got the ability to use streams:

```
List<String> names = Arrays.asList("Coding", "Lyf");
names.stream()
          .filter(name -> name.length() > 4)
     .forEach(System.out::println);
```

We don't have to go back and modify ArrayList, LinkedList, or any collection class.

Indirect Questions:

1. Is it necessary to override default methods of interface in Java 8?

No, they contain built in implementation. We override only if we want custom behaviour.

2. Is the default keyword one of the access modifiers?

No, it is not. It is a special keyword for method implementation in interfaces, not for setting access levels like public or private.

3. How to override default methods?

We can override like a like any normal method. override it in the class using @Override, and provide own body.

4. If two interfaces have the same default method, and you implement both, what will happen?

If a class implements two interfaces that each have the same default method, a compilation error will occur because the compiler cannot determine which default method to inherit.

To resolve this, the class must explicitly override the default method and provide its own implementation, or explicitly call the desired interface's default method using InterfaceName.super.methodName().

5. Can you call a default method from a class that implements Interface?

Yes, we can call a default method from a class that implements the interface, but only within that class or its subclasses, and only using InterfaceName.super.methodName().

Interview Tip: Try to tell with real world example like, Default methods were introduced so interfaces can be good choice without breaking existing implementations. For example, if an interface is used in 100 places, adding a new method would normally break them. With default, we can add it with a basic body and avoid rewriting everywhere.

88 Why Java 8 introduced static methods to the Interfaces?

Before Java 8, utility methods related to interfaces were in separate classes. Think about how we always used Collections.sort() or Math.max() , those were just static helpers sitting outside the actual interface logic.

So, in Java 8, They allowed interfaces to group their own utility methods?

That is how static **methods in interfaces** came to picture. They belong to the interface, not to the implementing class. So, we can call them like InterfaceName.methodName(), not through an object.

JDK Example:

In Java 8, the Comparator interface added static methods like:

```
public static <T extends Comparable<? super T>> Comparator<T> naturalOrder() {
        return (a, b) -> a.compareTo(b);
}
```

```
Now we can use like:
List<Integer> values = Arrays.asList(212, 324, 435, 566, 133, 100, 121);

// naturalOrder is a static method
values.sort(Comparator.naturalOrder());
```

Indirect Question:

1. Can an interface in Java have utility methods?

Yes, starting from Java 8. We can now add static methods to interfaces to hold related utilities.

2. What's the difference between a static method in a class and a static method in an interface?

A static method in a class can be called from its child class if the child doesn't define the same method. But a static method in an interface can only be called using the interface name. it doesn't get passed to the class that implements it.

3. Why do we still have utility classes like Collections or Arrays if interfaces can now have static methods?

Because they were created before Java 8, and changing them now would break backward compatibility across millions of projects.

4. If a class implements an interface, can it access the static method inside the interface via object reference?

Nope. Static methods in interfaces must be called using the interface name not via the object.

5. Can we override the static method of an interface?

No. Static methods in interfaces can't be overridden, they belong only to the interface, not to the implementing class.

89 What is optional and it uses?

Let's say we are calling a method and we are not sure if it'll return a value or null. Before Java 8, We had to check whether the value is null or not if not, there is high chance of getting **NullPointerException.**

Optional is a container that may or may not hold a non-null value. It is used to clearly show that a method might not return a result. Instead of returning null, the method returns an Optional to help avoid NullPointerException and make the code safer and easier to understand.

Example**:** In the below example, if user is null, user.getName() will throw NullPointerException.

```
String getName(User user) {
   return user.getName();
}
```

Now from Java 8, instead of returning String we will return Optional<String> which tells that , this may contain null and should be handled.

```
Optional<String> getName(User user) {

      return Optional.ofNullable(user.getName());
}

String name = getName(user).orElse("Anonymous"); //If user is null it returns 'No User'
```

Differences of each method in

optional: Optional.of() vs

Optional.ofNullable():

Optional.of(value): Use it when the value is guaranteed to be non-null. If value is null, it throws a NullPointerException.

Optional.ofNullable(value): Use it when the value might be null. If value is non-null, it returns an Optional containing the value; if value is null, it returns an empty Optional (Optional.empty()).

Optional.empty() vs null:

Optional.empty(): Represents the explicit absence of a value within an Optional container. It is an object that clearly signals "no value present."

null: Represents the absence of a reference to an object. It can lead to NullPointerExceptions if not handled carefully and doesn't explicitly convey the intent of "no value." Optional aims to reduce the need for null checks.

Optional.get() vs **Optional.orElse() vs orElseGet():**

Optional.get():

Returns the contained value if present; otherwise, it throws a NoSuchElementException.

This method should only be used when isPresent() has confirmed the presence of a value.

Optional.orElse(defaultValue)

Returns the value if present. If not, returns the defaultValue we give. But defaultValue is always be executed, even if it is not needed.
Example:

```
String name = optionalName.orElse(createDefaultName());
```

Here, createDefaultName() will always be executed even optionalName is not null.

Optional.orElseGet(supplier)

Returns the value if present. If not, it *calls* the supplier to get the default.

The supplier runs only if needed, so it is better for heavy or costly operations.
Example:

```
Strin name  = optionalName.orElseGet(() -> createDefaultName());
```

Optional.isPresent() + get() vs ifPresent():

Optional.isPresent() + get()

We first check if the value is present using isPresent(), and then use get() to retrieve it.

```
if (optionalName.isPresent()) {
System.out.println(optionalName.get());
}
```

This works, but it is a bit long and not the cleanest way to write code.

Optional.ifPresent()

This is a shorter and cleaner way. If a value is present, it runs the code. No need to check manually.

```
optionalName.ifPresent(name -> System.out.println(name));
```

This avoids NullPointerException and looks neater.

Interview Tip: Mention a real-world scenario like, we can use Optional for API responses, if a user was not found, we returned Optional.empty() instead of null. That avoided null pointer bugs and we can use orElse to return default value if no user found.

90 What is functional interface and why they are introduced in Java?

First let's see what is functional programming

Java was always an object-oriented language. But as apps grew bigger and data processing became more complex, writing clean and reusable code became harder. Developers wanted a way to **focus more on "what to do" than "how to do it",** especially when working with **collections, filtering, mapping, reducing, etc.**

That is where Functional Programming came into picture. Java adopted it in Java 8 with features like:

- Lambda expressions - short way to write functions
- Functional interfaces - interfaces with just one abstract method
- Streams API - to process collections in a chain-

style These made code shorter, cleaner.

Functional Programming:

Functional programming is a way of writing code where we build programs using functions, not objects or changing variables. It focuses on "what to do, not "how to do" it.

Example:

Let's say we have a list of names and want only names that start with "S".

Before Java 8:

```
List<String> names = Arrays.asList("Sam", "Ravi", "Sneha", "John");
List<String> result = new ArrayList<>();
```

```
for (String name : names) {
            if (name.startsWith("S")) {
        result.add(name);
        }
}

System.out.println(result); // Output: [Sam, Sneha]
```

Here we wrote:

- A loop
- An if condition
- Manually added items to a new list

From Java 8 (Functional Way):

```
List<String> names = Arrays.asList("Sam", "Ravi", "Sneha", "John");

List<String> result = names.stream()
        .filter(name -> name.startsWith("S"))
        .collect(Collectors.toList());

System.out.println(result); // Output: [Sam, Sneha]
```

Here: We don't need write loops, no manual adding.

Reference: Functional Programming in Java with Examples - GeeksforGeeks

Functional interface: A functional interface is an interface that contains exactly one abstract method, although it can also contain multiple default or static methods. The presence of a single abstract method allows instances of functional interfaces to be created with lambda expressions.

```
@FunctionalInterface
public interface MyFunctionalInterface {
        void execute();
}
```

This interface has only one abstract method, execute, making it a functional interface.

Functional interfaces are used extensively with lambda expressions. A lambda expression provides a clear and concise way to implement the single abstract method of a functional interface. Here is how we can implement the MyFunctionalInterface using a lambda expression

```
MyFunctionalInterface myFunc = () -> System.out.println("Executing...");
myFunc.execute();
```

Reference: Functional Interfaces in Java - Scaler Topics

Tricky Questions:

1.Can we include default and static methods in functional Interface?

Yes, A functional interface can have exactly **one abstract method** and any number of default and static methods.

2. Can a functional interface extend another functional interface? Yes, but with a condition

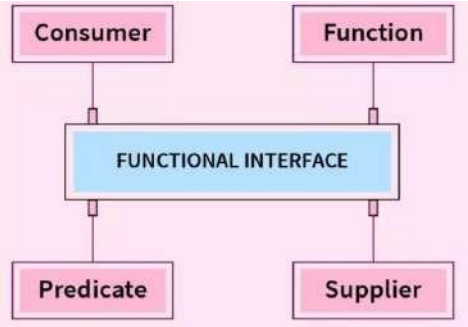
- If the parent functional interface has one abstract method, and the child does not add another abstract method, then the child is still functional.
- If the child adds another abstract method (other than what it inherits), it is no longer a functional interface.

3. Can a functional interface implement another interface?

An interface can extend another interface, but it cannot implement one. Implementation is only for classes.

91 Explain different types of functional interfaces?

Java SE 8 included 4 main kinds of functional interface.



Consumer: This interface represents an operation that accepts a single input argument and returns no result. It is used when an action needs to be performed on an object without producing a return value. Example: forEach()

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

Supplier: This interface represents a supplier of results. It takes no arguments and returns a value of a specified type. It is used when a value needs to be generated or retrieved. Example: Optional.orElseGet

```
@FunctionalInterface
public interface Supplier<T> {


    T get();
}
```

Predicate: A Predicate is a functional interface that represents an operation that takes an input of type `T` and returns a boolean result. Example: filter()

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

Function: This interface represents a function that accepts one argument and produces a result. It is used for transforming an input value into an output value of a potentially different type. Example: map()

```
@FunctionalInterface
  public interface Function<T,
  R> { R apply(T t);
}
```

Indirect Questions:

1. What interface would you use to check if an item matches a criteria?

Use Predicate<T>. It returns a boolean and is perfect for filtering data in streams.

```
Predicate<Integer> isEven = num -> num % 2 == 0;
```

2. What interface allows converting one data type into another in streams?

Function<T, R> : it maps one type to another, like converting String to Integer.

```
Function<String, Integer> toInteger = str -> Integer.parseInt(str);
```

3. How do you perform logging or save to DB while processing a stream?

Use Consumer<T>. It handles side effects like printing, logging, or saving records.

```
Consumer String> logger = msg -> System.out.println("Logging: " + msg);
```

4. How can you generate random numbers or timestamps in a stream?

Use Supplier<T>. It takes no input and returns something like new Date() or UUID.

```
Supplier<Double> randomGenerator = () -> Math.random();
```

5. How do you lazily provide a default value in case of null using Optional?

Use Optional.orElseGet(Supplier). It'll only call the Supplier if the value is missing.

```
Optional<String> name = Optional.ofNullable(null);
String result = name.orElseGet(() -> "Default Name");
```

6. How do you think the Supplier interface can be useful when writing lazy initialization code?

Supplier can delay object creation until it is actually needed, it is great for saving memory or resources.

7. Which functional interfaces existed in Java before Java 8?

Runnable and Callable were considered functional interfaces before Java 8 as they had a single abstract method.

92 Explain about Lambda expression?

Before Java 8, if we want to do something simple, filter a list, or run some code in a new thread we had to write a lot of code using anonymous inner classes.

Java 8 changed this by introducing the idea that **functions can be first-class citizens**. That means functions can be treated just like variables, we can pass them around, store them, and return them from methods.

Before Java 8 (Anonymous Inner Class): if we want to create a thread that prints "Hello".

```
    Thread thread = new Thread(new Runnable() {
    @Override
            public void run() {
        System.out.println("Hello");
        }
});
thread.start();
```

After Java 8 (Lambda Expression): Same behaviour, written in a much cleaner way:

```
Thread thread = new Thread(() -> System.out.println("Hello"));
thread.start();
```

Now we are passing behaviour (a function) like data, that is what we mean by functions becoming **first- class citizens**.

Code Link: JavaSamples/LambdaExample.java at main · CodingLyf-Fullstack/JavaSamples

93 Difference between lambda expression and anonymous class?

Anonymous Classes An anonymous class is like a temporary, nameless class that we create and instantiate on the spot. We can use it to implement an interface or extend an abstract class.

Before Java 8, if we wanted to create a new Runnable to run in a thread, we had to write it as an anonymous class:

```
Runnable r = new Runnable() {

        @Override
            public void run() {
        System.out.println("Running a task!");
        }
};
```

Lambda Expressions Introduced in Java 8, a lambda expression provides a much more concise way to represent an implementation of an interface with a single abstract method (known as a functional interface).

Using a lambda expression, the same Runnable task can be written in a single line:

```
Runnable r = () -> System.out.println("Running a task!");
```

Anonymous Inner Class	Lambda Expression
It is a class without name.	It is a method without name (anonymous function). It can't extend abstract and concrete class.
It can implement an interface that contains any number of abstract methods.	It can implement an interface which contains a single abstract method.
Inside this we can declare instance variables.	It does not allow declaration of instance variables; variables declared act as local

variables.

Anonymous inner class can be instantiated.	Lambda Expression can't be instantiated.
Inside Anonymous inner class, "this" always refers to current anonymous inner class object but not to outer object.	Inside Lambda Expression, "this" always refers to current outer class object (enclosing class).
It is the best choice if we want to handle multiple methods.	It is the best choice if we want to handle interface.
At the time of compilation, a separate .class file will be generated.	At the time of compilation, no separate .class file will be generated; it is converted into a private method of the outer class.
Memory allocation is on demand, whenever we are creating an object.	It resides in a permanent memory of JVM

94 Explain about Method references and uses?

Method references are like shortcuts. Instead of writing a full lambda expression, we can just point to an existing method.

Lambda style: list.forEach(s -> System.out.println(s));

It works fine. What we are doing is, taking “s” and passing it to System.out.println.

Now with method reference:

list.forEach(System.out::println); Types of Method

References:

Static Method Reference: Refers to a static method of a class.

```
// Without method reference
Arrays.sort(numbers, (a, b) -> Integer.compare(a, b));

// With method reference - much cleaner!
Arrays.sort(numbers, Integer::compare);
```

Instance Method Reference of a Particular Object: Refers to an instance method of a specific object.

```
class Printer {
            public void print(String message) {
        System.out.println(message);
         }
}

Printer myPrinter = new Printer();

// Without method reference
messages.forEach(msg -> myPrinter.print(msg));

// With method reference
messages.forEach(myPrinter::print);
```

Constructor Reference: Refers to a constructor of a class

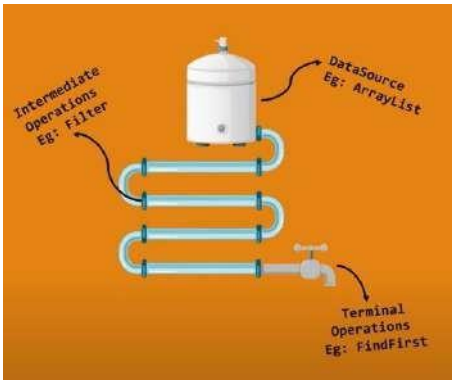
```
// Without method reference
Supplier<List<String>> supplier = () -> new ArrayList<>();

// With method reference
Supplier<List<String>> supplier = ArrayList::new;
```

95 Explain about Streams?

Before Java 8, we used to write long for-loops to filter, map, or collect data.

Streams are introduced in Java 8. Streams make the code shorter, readable, and more declarative; they provide a functional and efficient way to process collections by creating a pipeline of operations. They enable declarative programming, lazy evaluation, and parallel processing without modifying the original data source.

Think of Java Streams like water flowing through pipes.



Lazy Evaluation -> Water doesn't flow until the tap is opened. Similarly, Stream operations don't execute until a terminal operation like collect or forEach is called.

Parallel Processing -> Just like multiple pipes can distribute water to different locations simultaneously, parallel streams split data into chunks and process them across multiple threads for better performance.

Immutable Flow -> Water in the pipeline doesn't affect the water tank. Similarly, a Java Stream doesn't modify the original collection; instead, it produces a transformed result.

It provides various operators to process the data. They can be segregated like

Source Operators

This is where the stream starts. We can turn a list, array, or even lines from a file into a stream. Example: List.of(1,2,3).stream() or Files.lines(path)

Intermediate Operators

These are the steps in the middle. They transform the stream but don't run until a terminal operation is called (this is called lazy evolution).
Example: .filter(n -> n % 2 == 0).map(n -> n * 2)

Short-Circuit Operators

These stop the stream early when a condition is met. Useful when we don't need to process the entire stream.
Example: .limit(5), .anyMatch(x -> x > 100)

Terminal Operators

These are the end of the stream. Once we call a terminal operation, everything runs and gives us a result like a list, count, sum, etc.
Example: .collect(toList()), .count(), .forEach(System.out::println)

Indirect Questions:

1. What is lazy evaluation in streams?

Streams don't do anything until a terminal operation is called. That means nothing runs during filter() or map() until we finally say something like collect() or forEach().

2. Common pitfalls to avoid with Streams?

Reusing the same stream twice doesn't work it throws an error.

3. How stream operations are optimized internally?

Java combines operations like filter() and map() into a single loop behind the scenes , so we don't pay for each one separately. It is called loop fusion.

4. What is loop fusion?

Loop fusion in Java Streams is an internal optimization technique where multiple intermediate stream operations are combined and executed in a single pass over the data source. Instead of processing each element through one operation, then passing the result to the next, and so on, loop fusion merges these operations into a single, optimized loop.

```
list.stream()
.filter(x -> x % 2 == 0)
.map(x -> x * x)
.forEach(System.out::println);
```

This looks like multiple steps, but under the hood it is one

loop. How Fusion Happens

- Each intermediate op (filter, map) doesn't create a new list.
- Instead, it wraps its logic into a chain of operations.
- When the terminal op (forEach) pulls elements, the pipeline executes all stages in sequence for each element.

So for element x = 4:

- Check filter (4 % 2 == 0 > true)
- Apply map (4 * 4 > 16)
- Send to forEach (print 16)

Only then does it move to the next element.

This is why it is called loop fusion: multiple logical loops (filter > map > forEach) are fused into a single physical loop.

5. Are Streams always faster than loops? Why or why not?

No, Streams are NOT always faster than loops.

Overhead: Streams introduce additional abstraction and object creation, which can make them slower than plain for-loops, especially for small or simple tasks.

Performance: For basic iteration (e.g., summing an array), a classic loop is often faster due to lower overhead.

Reference: Is Java Stream Really Faster Than For-Loop? 🚀 | Java Interview Question

96 Can you list of streams operators you have used or you know?

1. Source Operators: Create a stream from a collection, array, or I/O source.

• stream() – Converts a collection into a sequential stream.

• parallelStream() – Creates a parallel stream for multi-threading.

• Stream.of() – Generates a stream from given values.

• Arrays.stream() – Converts an array into a stream.

• IntStream.range() – Creates a stream of numbers within a range.

• BufferedReader.lines() – Converts lines of a file into a stream.

2. Intermediate Operators: Transform or filter data without executing immediately (lazy evaluation).

• filter() – Selects elements that match a given condition.

• map() – Transforms each element using a function.

• flatMap() – Flattens nested structures into a single stream.

• distinct() – Removes duplicate elements from the stream.

• sorted() – Sorts elements in natural or custom order.

• limit(n) – Restricts the stream to the first n elements.

• skip(n) – Skips the first n elements in the stream.

• peek() – Used for debugging without modifying elements.

3. Terminal Operators: Trigger execution and produce a final result, consuming the stream.

• collect() – Converts a stream into a List, Set, or Map.
• forEach() – Performs an action for each element.

• count() – Returns the number of elements in the stream.

• findFirst() – Retrieves the first element of the stream.

• findAny() – Retrieves any element from the stream.

• reduce() – Aggregates elements into a single result.

• toArray() – Converts stream elements into an array.

4. Short-Circuiting Operators: Stop execution early when a condition is met for better performance.

• anyMatch() – Returns true if any element matches a condition.

• allMatch() – Returns true if all elements match a condition.

• noneMatch() – Returns true if no elements match a condition.

• limit(n) – Stops processing after selecting n elements.

• findFirst() – Retrieves the first element and stops further operations.

Many indirect or Scenario based questions can be asked on these operators. Example

1. How do you debug the stream operational flow?

peek() is an intermediate operation that allows us to inspect elements as they flow through the stream pipeline . it is typically used for debugging, to observe the elements as they flow through the pipeline.

2. Can we access outside variable in streams?

In Java streams (or lambdas), we can access variables from outside the stream/lambda body if they are effectively final. That means the variable isn't reassigned after its initialization.

```
int factor = 2; // effectively final
List<Integer> numbers = List.of(1, 2, 3);

numbers.stream()
               .map(n -> n * factor)    // accessing outside variable
               .forEach(System.out::println);
```

3. Can we change the outside variable in streams?

No, if we try to modify an outside variable inside the lambda/stream, we will get a compilation error.

```
int sum = 0;
List<Integer> numbers = List.of(1, 2, 3);

numbers.forEach(n -> sum += n); //Compilation Error
```

Error: Local variable sum defined in an enclosing scope must be final or effectively final

4. Is there any way to update outside variables in streams?

If we really need to update state from inside a stream, we can use Mutable containers like AtomicInteger or a list.

```
AtomicInteger sum = new AtomicInteger(0);
numbers.forEach(n -> sum.addAndGet(n));
System.out.println(sum.get());
```

5. If you don't use the terminal operation in Stream pipeline, will the intermediate operations be executed, why or why not?

- Streams in Java are lazy. Intermediate operations (map, filter, sorted, etc.) are only recorded, not run immediately.
- They get executed only when a terminal operation (forEach, collect, reduce, etc.) is called.
- Without a terminal operation, the pipeline never triggers, so nothing runs.

97 Difference between Streams and Collections?

Collections, like List, Set, and Map are used hold the data. It is like a container. We can add the items, retrieve the items and update them. They will be saved in memory.

Streams are different from collections. Streams don't store anything. They just describe how data should be processed. We don't add or remove things from a Stream. We just say, 'take this list, filter out the
even numbers, map each number to its square, and collect the result.'

The important difference is, Streams are lazy. They don't do anything until we tell them to finish, we usually do with a terminal operation like .collect() or .forEach().

Let's say we have a list of 1000 numbers. With a Collection, we already have all those numbers in memory. With a Stream, nothing really happens until we start processing them. The data flows through like water in a pipe, step-by-step.

So, the core difference is, Collections are about storing data. Streams are about processing data.

98 Difference between map and flatMap?

The **map()** method transforms each element of a stream into another object. It takes a Function as an argument, which defines the transformation logic. This function is applied to each stream element, and the result is collected into a new stream.

The **flatMap()** method takes each element in the stream, applies a function to it, and that function returns **another stream** (could have 0, 1, or many elements).Then, flatMap() takes all those small
streams and joins them into one big stream.

Reference: Exploring the Differences: Java map vs. flatMap | by Reetesh Kumar | Medium

Indirect Question:

1.List of lists and need to be converted it into a single list of all elements, how you do it? Use flatMap

99 How reduce works?

reduce() is a terminal operation in Java Streams that takes a sequence of elements and combines them into **a single result** using a function , like adding, multiplying, or finding the max.

Example:

Lets say, We have a box of numbers:
[5, 10, 15]

We want to **add** them together. Normally, we say:
5 + 10 = 15, then 15 + 15 = 30

That is exactly what reduce() does. It takes two elements at a time, applies an operation (like sum), and continue further.

```
List<Integer> numbers = Arrays.asList(5, 10, 15);

int sum = numbers.stream()
        .reduce(0, (a, b) -> a + b); // 0 is the starting value

System.out.println(sum); // Output: 30
```

100 forEach vs collect()? collect()

- It collects the processed stream into a data structure (like a List, Set, or Map)
- It returns that data structure
- We use it when we want to store or reuse the results

forEach()

- It goes through each item and performs an action (like printing or logging)
- It returns nothing (void)
- We use it for side effects, not for building data

Indirect Question:

1. What happens if you call terminal operation without any intermediate operations? It works. The stream directly applies the terminal operation over the source data. For example: count()

```
List<String> list = Arrays.asList("a", "b", "c");

long count = list.stream().count();
System.out.println(count); // Output: 3
```

101 How can you optimize a Stream pipeline for better performance?

- Apply filtering and short-circuiting operations (like filter, limit, findFirst) early in the pipeline to reduce the number of elements processed downstream.
- Use primitive specialized streams (IntStream, LongStream, DoubleStream) to avoid unnecessary boxing/unboxing.
- Use parallel streams only when working with large datasets and the workload is CPU-bound.

102 Difference between sequential stream and parallel stream?

Sequential Streams

By default, all streams in Java are sequential. This means the data is processed one item at a time, in order, using a single thread, usually the main thread or the one we are currently in.

Example:

```
List<String> names = List.of("Rohit", "Gill", "Hardik");

names.stream()
          .map(String::toUpperCase)
          .forEach(System.out::println);
```

This will process "Rohit", then "Gill", then "Hardik", one after the other.

Parallel Streams

A parallel stream splits the data into chunks and processes them in parallel using multiple threads behind the scenes (from the common fork-join pool). The goal is better performance, especially with large datasets.

Just change .stream() to .parallelStream():

```
List<String> names = List.of("Rohit", "Gill", "Hardik");

names.parallelStream ()
          .map(String::toUpperCase)
          .forEach(System.out::println);
```

Now "Rohit", "Gill", and "Hardik" might be processed *out of order* and *at the same time* by different threads.

When .parallel() is called, Spliterator **splits the work** into chunks that can be processed in parallel by ForkJoinPool.

Indirect Questions:

1. When to use Sequential Streams?

We will use sequential streams when the order of operations matters or when we have a small amount of data.

Ex: Read data in sequential manner and processing it.

2. When to use Parallel Streams?

Use parallel streams for large, CPU-intensive tasks where the order of elements doesn't matter. Example. If we have a million numbers and we need to calculate the square of each one; a parallel stream can split the work and do it much faster

3. When not to use Parallel Streams?

Avoid parallel streams when order is must, the operations are fast and the dataset is small, as the overhead of managing threads will hit the performance.

4. What is a Spliterator?

Spliterator stands for Split + Iterator. It is a special iterator introduced in Java 8 that supports:

1. Sequential traversal (like Iterator).
2. Efficient splitting of data into chunks for parallel

processing. It is the backbone of how the Stream API works

under the hood.

103 How do you handle exceptions in streams?

Let's divide this into checked and unchecked exceptions.

Unchecked Exceptions:

We can catch these unchecked exceptions while processing the data. We can handle unchecked exceptions with two approaches

1. Handle It Inside
2. Filter Out the Bad Ones

Example: Suppose we have a list and we would like to convert strings to number. But it will throw NumberFormatException because of "four"

List<String> numbersAsString = Arrays.asList("1", "2", "3", "four", "5");

In "**Handle it Inside**" approach, we catch the error and return some fallback value. In the below we have handled the exception and return -1 as a fallback value.

```
List<String> numbersAsString = Arrays.asList("1", "2", "3", "four", "5");

numbersAsString.stream()
        .map(s -> {
                try {
                        return Integer.parseInt(s); // throw NumberFormatException


                } catch (NumberFormatException e) {
                        return -1; // fallback value
                }
        })
        .forEach(System.out::println);
```

In **"Filter out the bad ones**" approach we will filter them using filter method. Here we are returning null when exception occurs and filtered using Objects:nonNull.

```
List<String> numbersAsString = Arrays.asList("1", "2", "3", "four", "5");

numbersAsString.stream()
    .map(s -> {
        try {
             return Integer.parseInt(s);
        } catch (NumberFormatException e) {
                        // We caught the exception! Let's return a default value, like -
            1. System.err.println("Could not parse: " + s);
                        return null;
                }
        })
        .filter(Objects::nonNull)
        .forEach(System.out::println);
```

We can also use Wrapper to handle exceptions to write more cleaner code. We will see that in next section.

Code Link: JavaSamples/LambdaUnCheckedException.java at main · CodingLyf-Fullstack/JavaSamples

Checked Exceptions:

Java streams don't work well with checked exceptions because the functional interfaces used in stream operations (like Function, Predicate, etc.) don't declare any checked exceptions. Let's see three approaches

1. Wrap in RuntimeException

The simplest approach is to catch the checked exception and wrap it in a runtime exception. In the below we are catching Checked Exception and throwing Runtime Exception.

```
list.stream()
.map(item -> {
        try {
                return someMethodThatThrowsCheckedException(item);
        } catch (IOException e) {
                throw new RuntimeException(e);
        }
})
.forEach(System.out::println);
```

2. Create a Wrapper Function

I would suggest to watch the video and code link to understand better.

Video link: Java 8 Streams | Exception Handling Mechanism | lambda | JavaTechie

Code: JavaSamples/LambdaCheckedException.java at main · CodingLyf-Fullstack/JavaSamples

3. Propagate Exception

If we need to propagate the checked exception, we need to collect results and handle exceptions after processing:

```
List<String> results = new ArrayList<>();
List<Exception> exceptions = new ArrayList<>();

list.forEach(item -> {
            try {
        results.add(someMethodThatThrowsCheckedException(item))
        ;
            } catch (CheckedException e) {
        exceptions.add(e);
        }
});

if (!exceptions.isEmpty()) {
    // Handle exceptions
```

104 What are disadvantages or when not to Use Java Streams?

Java Streams are great for writing clean and short code when working with collections. But they're not always the best tool. Sometimes, using a regular for or while loop is easier, faster, or just makes more sense. Here is when we should avoid using streams:

1. When we Need to Change the Original List

Streams don't change the original data. They work on a copy and return a new result. If we need to add, remove, or update items directly in the original list, use a loop. It is clearer and usually faster.

2. When we Want to Exit Early with Complex Logic

Streams have methods like findFirst() or anyMatch() for early exit. But if our logic is more complicated, like we want to stop after checking several conditions, a loop with break is easier to follow and debug.

3. When we are Dealing with Checked Exceptions

Streams don't handle checked exceptions well inside functions like map() or filter(). We need to wrap the code in try-catch blocks, which makes it messy and hard to read. With loops, you can just handle exceptions normally.

4. When the Code Has Side Effects

Streams are meant for pure functions, can not change the outside variables. If we are updating variables outside the stream (like counters or flags), it can lead to bugs, especially if the stream is parallel. Loops are safer and more predictable in such cases.

5. When We Need to Use Indexes

Streams don't give the easy access to element positions. If the logic depends on knowing the index (like comparing the current and next item), a regular loop is much easier to write and understand.

105 What's the difference between "this" in a lambda vs. an anonymous?

In Java, this just means "**the current object**"

But when we write code inside lambdas or anonymous classes, this behaves differently

In a Lambda, this refers to the outer class , the class where the lambda is written. Example:

```
public class MyClass {
        private String name = "Coding Lyf";

            public void
        greet() { Runnable r
        = () -> {
                        System.out.println("Hello, " + this.name); // accessing instance
            variable
using `this`
                };
                r.run();
        }

        public static void main(String[] args) {
                new MyClass().greet();
```

In above example, this.name prints "" , in Lambdas this refers to the class where it is written. But In an anonymous class, this refers to the anonymous instance NOT to outer class

Example : Accessing instance variable in **anonymous class**

```
public class MyClass {
        private String name = "CodingLyf";

        public void greet() {
                Runnable r = new Runnable() {
                        String name = "Inside anonymous";
```

```
                        public void run() {
                                // `this.name` doesn't work , 'this' refers to anonymous
                class, not
MyClass
                                System.out.println("Hello, " + this.name);
             System.out.println("Hello, " + MyClass.this.name); // have to use
       MyClass.this
            }
                };
                r.run();
        }

        public static void main(String[] args) {
                new MyClass().greet();
        }
```

Here this.name prints ‘`Inside anonymous`. MyClass.this.name prints ‘`CodingLyf`.

Indirect Questions:

1. Can you use this inside a Stream lambda?

Yes. Lambdas have access to the enclosing class's this.

2. Does this behave the same inside a lambda and an anonymous class?

No. In a lambda, this refers to the enclosing class. In an anonymous class, this refers to the anonymous instance

3. Can you access instance methods or fields inside Stream lambdas? Yes, because this is accessible.

106 Features of different versions?

Interview Tip: Generally, we will be asked the latest version we are using and features of it. Let’s see all major versions and important features. As we have seen Java 8 features already, let’s see from Java 11 – Java 23

Java 9:

Java 9 introduced convenience static factory methods on the List, Set, and Map interfaces to easily create immutable collections. These methods provide a concise way to create collections whose state cannot be changed after construction. Attempting to modify such collections will result in
an UnsupportedOperationException.

List.of(), Set.of(), Map.of(), Map.ofEntries()

```
// Empty immutable list
List<String> emptyList = List.of();
System.out.println("Empty List: " + emptyList);
```

```
// Immutable list with elements
List<String> fruits = List.of("Apple", "Banana", "Orange");
System.out.println("Fruits List: " + fruits);

// Attempting to modify will throw UnsupportedOperationException
// fruits.add("Grape");
```

Java 10:

- Local variable reference
- Optional API: orElseThrow

Local variable reference : Local Variable Type Inference , allows to declare the local variables without explicitly stating their type. Java looks at the value we assign and automatically guesses the correct type for the variable. This feature utilizes the reserved type name **var**

```
public class VarExample {
        public static void main(String[] args) {
                var message = "Hello, Java 10";     // inferred as String
                var number = 42;// inferred as int
                var list = List.of("a", "b", "c"); // inferred as List<String>

                System.out.println(message);
        System.out.println(number);
        System.out.println(list);
        }
}
```

Java 11 (Long Term Support)

1. var keyword in lambda expressions

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.forEach((var number) -> {
        System.out.println(number);
});
```

2. HTTP Client: Java 11 includes a new HTTP client API that provides a more modern and efficient way to send HTTP requests and receive responses. The new HTTP client API supports both HTTP/1.1 and HTTP/2 protocols and includes features like HTTP/2 push, server push, and WebSocket. The new API is also asynchronous and non-blocking, making it more scalable and performant than the previous HttpURLConnection API.

3. **String methods**: isBlank(), strip(), stripLeading(), stripTrailing(), lines(), repeat(int)

Java 12:

Collectors.teeing(): The Collectors.teeing() method in Java was introduced in Java 12 as part of the Stream API. It allows us to collect the same stream using two different collectors at once, and then combine their results using a merging function (BiFunction).

Reference: Mastering Collectors.teeing() in Java Streams (With Examples) | by A cup of JAVA coffee with NeeSri | Medium

File.mismatch() Method: A new method mismatch(Path, Path) was added to the Files class, allowing

for easy comparison of two files and returning the index of the first differing byte or -1 if they are identical.

String API Enhancements: New methods like indent() and transform() were added to the String class for easier string manipulation.

String.indent(int n): To add or remove indentation (leading spaces) from each line of a multiline string.

String.transform(): To apply a custom transformation to a string

```
String result = "hello"
                    .transform(str -> str.toUpperCase())
          .transform(str -> "Prefix_" + str);

System.out.println(result);
```

Java 14:

Switch Expressions: In Java 14, switch expressions were finalized Standard feature. This allowed switch to return values and eliminated the need for explicit break statements

```
int day = 2;
String dayName = switch (day) {
        case 1, 7 -> "Weekend";
        case 2, 3, 4, 5, 6 -> "Weekday";
        default -> "Invalid day";
};
System.out.println(dayName);
```

- The switch statement can now be used as an expression that returns a value.
- Arrow (->) syntax introduced for cleaner case expressions.
- The break statement is no longer needed in this context.

Yield: the yield keyword is used within switch expressions to return a value from a case block.

```
String dayOfWeek = "Monday";
String activity = switch (dayOfWeek) {
            case "Monday" -> {
        System.out.println("Starting the week!");
        yield "Go to work";
        }


        case "Saturday", "Sunday" -> "Relax";
        default -> "Study";
};
System.out.println("Today's activity: " + activity);
```

Java 15

Text Blocks: A Text Block is a multi-line string literal enclosed in triple double quotes ("""). It simplifies the creation of strings that span multiple lines, such as HTML, JSON, SQL queries, or other structured text.

```
String htmlContent = """
                    <html>
                        <body>
                            <h1>Welcome!</h1>
                        </body>
                    <
        /html>
```

Java 16:

Record: In Java, a record is a special type of class declaration aimed at reducing the boilerplate code. Java records were introduced with the intention to be used as a fast way to create data carrier classes,
i.e. the classes whose objective is to simply contain data and carry it between modules, also known as POJOs (Plain Old Java Objects) and DTOs (Data Transfer Objects)

```
public record Person(String name, int age) {}
```

This one line gives:

- A constructor
- getName() and getAge() methods
- A readable toString()
- Proper equals() and hashCode()

  Note: All fields are immutable.

Pattern matching for instanceof in Java simplifies type checking and casting by combining them into a single operation.

Before pattern matching, checking the type of an object and then casting it to access its specific members required two separate steps:

```
Object obj = "Hello World";
if (obj instanceof String) {
        String s = (String) obj; // Explicit
    cast System.out.println(s.toUpperCase());
}
```

With pattern matching, as a standard feature in Java 16, the explicit casting is no longer needed

```
Object obj = "Hello World";
    if (obj instanceof String s) { // 's' is the pattern variable
    System.out.println(s.toUpperCase());
}
```

Java 17:

Sealed classes provide a mechanism to restrict the inheritance hierarchy of a class or interface. This means we can explicitly control which classes are allowed to extend a sealed class or implement a sealed interface.

```
public sealed class Shape permits Circle, Square, Triangle {
        // ...
}
```

A class or interface is declared as sealed using the sealed modifier.

The permits clause explicitly lists the classes or interfaces that are allowed to extend or implement the sealed type.

When we use permits in a sealed class, the classes or interfaces we list must be marked with one of these:

- **final**: This means the class can't be extended by any other class. It is the end of the chain.
- **sealed**: This class is also sealed, so it controls who can extend it, just like the parent.
- **non-sealed**: This class removes the restriction. Any class can extend it now, breaking the sealed rule from this point on.

Java 21:

Pattern matching for switch

In older versions of Java, switch could only work with fixed values like numbers, strings, or enums. But from Java 21, with **pattern matching for switch**, we can now use switch to check both the **type** of an object and do something based on that type

This means we don't need to write extra if or instanceof checks. Java handles that for you right inside the switch.

```
static String handle(Object obj) {
        return switch (obj) {
                case String s -> "It is a string:
        " + s; case Integer i -> "It is a number:
        " + i; case null -> "It is null";
                default -> "Something else";
        };
}
```

So, if we pass a String, it will go into the String s case. If it is an Integer, it goes into the Integer i case.

Record Patterns:

Records are a simple way to create classes that just hold data (introduced in Java 14, made stable in Java 16).
Now, starting with Java 21, With **Record Patterns** we can easily extract the values from a record using pattern matching, without manually calling getters.

Before record patterns, if we had a record "**record** Person(String name, **int** age) {}", we read like

```
Person p = new Person("Macha", 25);
String name = p.name();
int age = p.age();
```

```
    if (p instanceof Person(String name, int age)) {
    System.out.println("Name: " + name + ", Age: " + age);
}
```

With **record patterns**, Java do this in a more readable way, especially inside if or switch:

Here, Java is checking if p is a Person, and at the same time, it pulls out name and age, no need to call .name() or .age() manually.

Switch:

```
switch (p) {
    case Person(String name, int age) ->
    System.out.println(name + " is " + age + " years old");
}
```

- Record patterns make it easier to **extract data from records**.
- We can use them with instanceof or switch.

Sequenced Interfaces:

Java introduced 3 new interfaces in the collection hierarchy.

- SequencedCollections
- SequencedMap
- SequencedSet

SequencedCollection works with ArrayList and LinkedList, it gives new methods like

- getFirst() – gives the first item
- getLast() – gives the last item
- addFirst() – adds an item at the beginning
- addLast() – adds at the end
- reversed() – gives a reversed view of the collection

```
SequencedCollection<String> sc = new ArrayList<>();
sc.addLast("Apple");
sc.addLast("Banana");
sc.addFirst("Mango");

System.out.println(sc); // [Mango, Apple, Banana]
System.out.println(sc.getFirst()); // Mango
System.out.println(sc.getLast()); // Banana
System.out.println(sc.reversed()); // [Banana, Apple, Mango]
```

SequencedSet and SequencedMap works in LinkedHashSet and LinkedHashMap respectively.

Output Based Questions

107 Output-Based 1: How many String objects will be created in memory from below code?

```
String s1 = new String("hello");
String s2 = "hello";
```

Answer and Explanation: 2 String objects

1. String s1 = new String("hello");

- This creates a new object in heap memory and String “hello” in String pool.
- So now there are two strings:
  - One in the string pool ("hello")
  - One in the heap (s1)

2. String s2 = "hello";

- This points directly to the string pool version of "hello".
- So now:
  - s1 points to heap object
  - s2 points to pool object

108 Output-Based 2: How many String objects will be created in memory from below code?

```
String s1 = new String("hello");
String s2 = "hello";
String s3 = s1.intern();
```

We have added 3rd line s1.intern();

intern(): When invoked on a String object, it checks if an identical string already exists in the pool. If found, it returns a reference to the existing string. If not, it adds the string to the pool and returns a reference to the newly added string.

Since “hello” is already existed in the string pool, It returns same object. So s1.intern() wont create new string object.

Answer is 2 String objects.

109 Output-Based 3: String ==?

```
String a = "CodingLyf";
String b = "CodingLyf";

System.out.println(a == b);
System.out.println(a.equals(b));
```

Explanation:

a == b is true

- Both a and b are string literals with the same content.
- Java stores string literals in the String pool, so both point to the same memory location.
- Same reference -> == is

true. a.equals(b) is true

- .equals() checks if the contents are the same.
- "CodingLyf" equals "CodingLyf" , so it returns true.

110 Output-Based 4: String == and equals?

```
String a = "hello";
String b = "hello";
String c = new String("hello");

System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));
```

Explanation:

- a == b: true, because both refer to the same string literal in the String pool.
- a == c: false, because new String() creates a new object in heap memory.
- a.equals(c): true, because equals() compares values, not references.

111 Output-Based 5: String concatenation?

```
String x = "codinglyf";
String y = "coding";
String z = y + "lyf";

System.out.println(x == z);
System.out.println(x.equals(z));
```

Explanation:

- z = y + "lyf" is built at runtime using a variable, so it is a new object in memory.
- Even though x and z look the same, they are not the same object, so == gives false.
- .equals() checks the actual text inside both strings, which is the same, so it returns true.

112 Output-Based 6: valueOf()?

```
String a = "hello";
String b = String.valueof(a);
System.out.println(a == b);
```

Explanation:

- a is "hello"
- The method String.valueOf(Object obj) does this internally: return (obj == null) ? "null" : obj.toString();
- a.toString() for a String just returns itself (not a new object)..
- So b ends up pointing to the **same object** as a. Output is true

113 Output-Based 7: Substring()?

```
String a = "hello";
String b = a.substring(0);
System.out.println(a == b);
```

Explanation:

- "hello" is stored in the String Pool, “a” points to it.
- a.substring(0) returns the same object when the whole string is taken.
- So a == b is true because both references point to the same string.

114 Output-Based 8: String == , equals and intern()?

```
String s1 = new String("hello");
String s2 = "hello";
String s3 = s1.intern();
System.out.println(s1 == s3)
System.out.println(s3 == s2);
```

Explanation:

s1 is created using new, so it is a separate object in heap, and creates string literal “hello” in the pool. s2 points to the string pool version of "hello".
s3 = Since “hello” is already there in string pool, s1.intern() won’t create new String, it just returns the pool version of "hello", so s3 == s2 is true, but s1 == s3 is false

115 Output-Based 9: On StringBuilder.

```
StringBuilder sb1 = new StringBuilder("abc");
StringBuilder sb2 = new StringBuilder("abc");;

System.out.println(sb1 == sb2);
System.out.println(sb1.equals(sb2));
```

Explanation: Both print
false sb1 == sb2

- This checks reference equality.
- sb1 and sb2 point to two different objects in memory, even though the contents are the same.
- So this returns false.

sb1.equals(sb2)

- We might expect this to compare contents, like String.equals() does.
- But StringBuilder does not override. equals() from Object.
- So, it behaves like ==, it still checks reference equality.
- Again, false.

To compare the content in StringBuilder, we need to use

```
System.out.println(sb1.toString().equals(sb2.toString()));
```

116 Output-Based 10: On StringBuilder, Checking the reference and content.

```
StringBuilder sb1 = new StringBuilder("abc");

StringBuilder sb2 = sb1;

sb1.append("d");
System.out.println(sb1 == sb2);
System.out.println(sb1.equals(sb2));
```

Explanation: Both print

true sb1 == sb2

- We are assigning sb2 = sb1, so both references point to the same object in memory.
- So == returns true.

sb1.append("d")

- We have modified the content of the object both sb1 and sb2 point to.
- Since S1 and S2 refer to the same object, the change is visible to both.

sb1.equals(sb2)

- Again, StringBuilder doesn't override .equals(), so it uses Object equals, so it checks ==.
- And since both refer to the same object, it returns true.

117 Output-Based 11: Array reference comparison vs. content comparison.

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a == b);
System.out.println(Arrays.equals(a, b));
```

Explanation: false and true

a == b

- This compares the references (memory addresses).
- a and b are two separate array objects, even though their contents are the same.
- So, this prints false.

Arrays.equals(a, b)

- This compares the contents of the arrays, element by element.
- Both arrays contain the same values in the same order: {1, 2, 3}.
- So, this prints true.

118 Output-Based 12: Pass by Value.

```
class Main {
            public static void
        change(int x) { x = 100;



    }

        public static  main(String[] args) {
        int a = 50;
        change(a);
        System.out.println(a);
    }
}
```

Explanation:

- In Java, **all arguments are passed by value**, even primitives like int.
- So, when change(a) is called, a **copy of a's value (50)** is passed to x.
- Inside change() method, x = 100 modifies the **copy**, only local variable will be change, not the original variable.
- The original 'a' in main() remains unchanged, so the output is 50.

119 Output-Based 13: Passing the reference.

```
public class Test {
            public static void change(StringBuilder sb) {
        sb.append(" Lyf");
        }

            public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Coding");
        change(sb);
                System.out.println(sb);
        }
}
```

Explanation:

- In Java, everything is passed by value, including object references.
- sb in main() holds a reference to the StringBuilder object.
- When passed to change(), a copy of that reference is passed, now both sb in main() and sb in change() point to the same object in memory.
- sb.append(" Lyf") changes the actual object in memory, not the reference variable. so the change is visible even after the method ends.
- So, System.out.println(sb) prints . Point

to remember:

Java passes object references by value.
We are not passing the object, we are passing a copy of its reference, but both refer to the same underlying object. So, changes to the object are reflected even outside the method.

120 Output-Based 14: Passing the reference and reassigning.

```
public class Test {
            public static void
        reassign(StringBuilder sb) { sb = new
        StringBuilder("New");
        }

            public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Original");
        reassign(sb);
                System.out.println(sb);
        }
}
```

Explanation:

- In main(), sb points to a StringBuilder object with the content "Original".
- When you call reassign(sb), a copy of the reference is passed into the method.
- Inside reassign(), sb = new StringBuilder("New") just reassigns the local copy to a new object. Now it is completely new object.
- So, the original sb in main() still points to "Original", so output is “Original”.

121 Output-Based 15: Passing reference of the array.

```
public class Test {
            public static void modify(int[]
        arr) { arr[0] = 99;
        }

            public static void
        main(String[] args) { int[] nums =
        {1, 2, 3}; modify(nums);
                System.out.println(Arrays.toString(nums));
        }
}
```

Explanation:

- In main(), nums is an array with values {1, 2, 3}.
- When passed to modify(), a copy of the reference to the array is passed.
- Java passes object references by value, so both arr and nums refer to the same array object.
- arr[0] = 99 changes the content of the original array in memory, since both arr and nums point to the same object.
- So, when we print nums, the change is visible.

122 Output-Based 16: Passing reference of the array's copy.

```
public class Test {
        public static void modify(int[]
    arr) { arr[0] = 99;
    }

        public static void main(String[]
    args) { int[] nums = {1, 2, 3};
    modify(Arrays.copyOf(nums, nums.length));
    System.out.println(Arrays.toString(nums))
    ;
    }
```

Explanation:

- Arrays.copyOf(nums, nums.length) creates a new array object with the same contents as nums. So changes made inside the method do not affect the original.
- This new array is passed to modify(), where arr[0] = 99 modifies only the copy, not the original.
- The original nums array in main() stays unchanged.
- So System.out.println(nums) prints [1, 2, 3].

123 Output-Based 17: Final variable.

```
final int x = 10;
x = 20;
System.out.println(x);
```

Explanation:

- final primitive = value can't be changed.
- Compilation error: cannot assign a value to final variable 'x',

124 Output-Based 18: Final array.

```
final int[] arr = {1, 2, 3};
arr[0] = 99;
System.out.println(Arrays.toString(arr));
```

Explanation:

- final means, we can't reassign the value to 'arr' but we can change the content.
- So arr[0] = 99 is allowed and updates the original array.
- Output is [99, 2, 3]

125 Output-Based 19: Final Object.

```
class Person {
        String name = "Alex";
}

public class Test {
        public static void
    main(String[] args) { final Person p
    = new Person(); p.name = "Sam";
    System.out.println(p.name);
    }
}
```

Explanation:

- final means, we can't reassign the value to Person object 'p' but we can change the state.

```
final Person p = new Person();
p = new Person(); //It throws compilation error as we are reassigning
```

- So p.name updates the name.
- Output is "Sam"

126 Output-Based 20: Multiple Exceptions.

```
try {
        // code
} catch (Exception e) {
        // handle
} catch (ArithmeticException ae) {
        // handle
}
```

Explanation:

- More specific exceptions must come before general exceptions.
- It throws Compile Time Error

127 Output-Based 21: Try with return and finally.

```
try {

    return;
} finally {
        System.out.println("In finally");
}
```

Explanation:

- Even try return the value, finally still executes before method returns.
- So, Output is "In finally".

128 Output-Based 22: Try and Finally with returns.

```
public static int test1() {
        try {
                return 1;
        } finally {
                return 2;
        }
}
```

Explanation:

- Output is 2, because finally overrides the return of try.

129 Output-Based 23: Execution flow with static

```
public class Test {

	static int x = Init();

	static {
		System.out.println("Static Block");
	}

	private static int Init() {
	     System.out.println("Static variable initialization");
			return 1;
	     }

	     public static void main(String[] args) {

	}
}
```

Explanation:

When the class is loaded (before main() runs), Java does these things in order for static:

- All static variables are initialized first in the order they are defined.
- Static blocks are executed in the order they appear in the code.

- Here in this class static variable is defined at first so the output is

  1. Static variable initialization
  2. Static block
  3. Main method (Does thing in above

code) Points to remember:

Static blocks & static variables run once when the class is loaded.

Instance variables and instance blocks run before the constructor for each new object.

130 Output-Based 24: Execution flow with instance.

```
public class Test {

	int x = instanceInit();

	{
		System.out.println("Instance Block");
	}

	Test() {
		System.out.println("Constructor");
	}


	private int instanceInit() {
		System.out.println("Instance Variable Initialization");
		return 10;
	}

	public static void main(String[] args) {
		new Test();
	}
}
```

Explanation:

- When new Test() is called, object creation starts.
- First, instance variable x is initialized, which calls instanceInit() and prints: Instance Variable Initialization.
- Then the instance block runs and prints: Instance Block.
- After that, the constructor runs and prints: Constructor.
- Concept: For every object, Java runs instance variable initializations (depends on order) > instance blocks (depends on order) > constructor

Answer is:
Instance Variable Initialization
Instance Block
Constructor

131 Output-Based 25: Overriding: Method overriding basics.

```
class A {
        void show() {
                System.out.println("A");
        }
}
class B extends A {
        void show() {
                System.out.println("B");
        }
}
public class Test {
            public static void main(String[]
        args) { A obj = new B();
                obj.show();
        }
}
```

Explanation:

- A obj = new B(); creates a reference of type A but an object of type B , this is upcasting.
- At runtime, method overriding comes into picture , we call it as dynamic method dispatch.
- Since show() is overridden in class B, the method from B gets called, not A.
- Output is B. It demonstrates the runtime

polymorphism. What is Dynamic method dispatch?

Dynamic method dispatch is a concept in Java where a call to an overridden method is resolved at runtime rather than compile time.

It allows a parent class reference to refer to a child class object, and when a method is called, the version in the actual object's class runs, not the reference type. This is key for polymorphism. For
example, if Animal a = new Dog(); and both have a sound() method, calling a.sound() will run Dog's sound(), not Animal's. It makes code flexible, letting one reference handle different object types safely and efficiently.

132 Output-Based 26: Overriding: Variable hiding.

```
class A {
    A() {
                System.out.println("A constructor");
        }
}
    class B extends A
    { B() {
                System.out.println("B constructor");
        }
}
public class Test {
        public static void main(String[] args) {
                new B();
        }
}
```

```
            public static void main(String[]
         args) { A obj = new B();
         System.out.println(obj.x);
         }
 }
```

Explanation:

- A obj = new B(); it is upcasting , means reference is of type A but object is of type B.
- int x = 10 in A and int x = 20 in B . This is variable hiding, not overriding.
- Point to remember: Variable resolution happens at compile-time based on reference type, not object type.
- So, obj.x access the A's variable, Output: 10.

133 Output-Based 27: Overriding: Constructor Execution Order.

Explanation:

- When new B() is called, B's constructor runs.
- But since B extends A, A's constructor is called first which is implicit super() call.
- This shows constructor chaining in inheritance.
- So, output is:

  A constructor

  B constructor

134 Output-Based 28: Overriding: Static method hiding.

```
class A {
            static void show() {
         System.out.println("A static");
         }
}
class B extends A {
```

```
    static void show() {
        System.out.println("B static");
    }
}
public class Test {
    public static void main(String[]
    args) { A obj = new B();
        obj.show();
    }
}
```

Explanation:

- show() is a static method, which means it is not overridden but it is hidden, a concept called method hiding.
- A obj = new B(); is upcasting, but with static methods, only the reference type matters, not the object.
- Since obj is of type A, obj.show() calls A's version, not B's.
- So the output is: A static.

135 Output-Based 29: Overriding: Accessing Child-Specific Method.

```
class A {
    void show() {
        System.out.println("A");
    }
}
class B extends A {
    void display() {
        System.out.println("B display");
    }
}
public class Test {
    public static void main(String[]
    args) { A obj = new B();
        obj.display();
    }
}
```

Explanation:

- A obj = new B(); here reference is of type A, object is of type B.
- We are trying to call obj.display();, but display() is not defined in class A.
- Even though the object is of type B, the compiler checks in class A for available methods because the reference type is A.
- Since A has no display() method, this results in a compile-time

error How you fix this compile time error?

Answer: Cast 'obj' to B class.

```
B b = (B) obj;
b.display();
```

136 Output-Based 30: String Concatenation.

```
class Test {
                public static void
            main(String[] args) { String s1 =
            "S" + 1 + 2;
                          String s2 =
            "S" + (1 + 2); String s3 =
            1 + 2 + "S";

                          System.out.println(s
            1); //S12 System.out.println(s2);
            //S3 System.out.println(s3); //3S
```

Explanation:

In s1 = "S" + 1 + 2, Java processes expressions left to right. Since "S" is a string, 1 is converted to "1" and concatenated, resulting in "S1". Then "S1" concatenates with 2 (converted to "2"), so "S12".

For s2 = "S" + (1 + 2), the parentheses force the addition 1 + 2 to happen first as integer arithmetic, gives 3. Then "S" concatenates with "3" to form "S3".

In s3 = 1 + 2 + "S", the addition 1 + 2 happens first because both are integers, resulting in 3. Then 3 is concatenated with "S" to get "3S".

137 Output-Based 31: Overriding: Type Casting 1.

```
class A {}
class B extends A {}
class C extends B {}
public class Test {
        public static void main(String[] args) {
                A a = new C();
                B b = (B) a;
                System.out.println("Cast successful");
        }
}
```

Explanation:

- C extends B, which extends A, so C is indirectly a subclass of A.
- A a = new C(); is valid because we assigning a subclass (C) to a superclass reference (A) , that is **upcasting**.
- B b = (B) a; is **downcasting**, means are converting 'a' reference to class B. It is safe here because 'a' is actually pointing to a C object and C is a subclass of B.
- Since the cast is valid at runtime and the output is: Cast successful.

138 Output-Based 32: Overriding: Type Casting 2.

```
class A {}
class B extends A {}
class C extends B {}
public class Test {
            public static void main(String[]
        args) { A a = new B();
                C c = (C) a;
                System.out.println("Cast successful");
        }
}
```

Explanation:

- A a = new B(); we are storing a B object in a reference of type A. This is **upcasting** and perfectly safe.
- Then we write: C c = (C) a; this is **downcasting**. We are telling Java, "Treat this object as a C."
- But here is the problem: the actual object in memory is a B, **not a C**. So, the cast is **logically wrong**, the compiler allows it because C is related to A.
- At runtime, Java checks the object and throws: ClassCastException

139 Output-Based 33: Overriding: Field Access.

```
class A {
        int val = 1;
}
class B extends A {
        int val = 2;
}
public class Test {
            public static void main(String[]
        args) { A obj = new B();
        System.out.println(obj.val);
        }
}
```

Explanation:

- Both class A and class B have a field named val. This is **variable hiding**, not overriding.
- A obj = new B(); we are creating a B object but storing it in an A reference.
- When we access obj.val, Java looks at the reference type, not the object type and since obj is of type A, it fetches A's val.
- So the output is 1.
- Point to remember: fields are resolved at compile-time by reference type, but methods resolve at run time which use runtime polymorphism.

140 Output-Based 34: Overriding: Private Method in Parent.

```
class A {
            private void show() {
        System.out.println("A");
        }
}
class B extends A {
            public void show() {
        System.out.println("B");
        }
}
public class Test {
            public static void main(String[]
        args) { A obj = new B();
                obj.show();
        }
}
```

Explanation:

- In class A, show() is marked private, which means it is not inherited by class B.

- In class B, we define a new public show() method, but it doesn't override A's method.
- A doesn't have a visible show() method (it is private), so we can't call it through an A reference..
- So obj.show(); causes a **compile-time error**

Point to remember: Private methods are not inherited, so no overriding happens.

New B().show() will work, it prints B. It treats show() as normal function

141 Output-Based 35: Overriding: Protected Method Access

```
class A {
            protected void test() {
        System.out.println("A test");
        }
}
class B extends A {
        void test() {



             System.out.println("D test");
        }
}
public class Test {
        public static void main(String[] args) {
                new B().test();
        }
}
```

Point to remember: In Java, when we override a method in a subclass, the overridden method (the method in the subclass) must have the same or more accessible (less restrictive) access modifier than the overriding method (the method in the parent class). **Keep same visibility or increase it**, but **never reduce it**

Explanation:

- A protected method can be accessed within the same package or by subclasses (even in different packages).
- A default (package-private) method is less accessible than protected because it is only accessible within the same package
- test() in class A is protected, means visible to subclasses and classes in the same package.
- test() in class B is default/package-private (since we didn't specify any modifier).
- So, we are reducing visibility, which is not allowed, even if the classes are in the same package.
- That is why the compiler throws error: Cannot reduce the visibility of the inherited method from A

Solution: Either make the method protected or public in class B:

142 Output-Based 36: Overriding: Parent public, child private

```
class A {
             public void test() {
        System.out.println("A test");
        }
}
class B extends A {
           private void test() {
      System.out.println("B test");
        }
}
public class Test {
        public static void main(String[] args) {
                new B().test();
        }
}
```

Point to remember: In Java, when we override a method in a subclass, the overridden method (the method in the subclass) must have the same or more accessible (less restrictive) access modifier than the method (the method in the parent class). **Keep same visibility or increase it**, but **never reduce it**

Explanation:

- Class A has a public method test() that prints "A test".
- Class B extends A and attempts to override test() but declares it as private.
- In Java, a subclass cannot reduce the visibility of an inherited method. This is a compilation error because private is more restrictive than public

Solution: make the method public in class B:

143 Output-Based 37: Overriding: Checked Exceptions

```
class Parent {
            void readFile() throws IOException
        { System.out.println("Reading file");
        }
}

    class Child extends
    Parent { @Override
            void readFile() throws Exception {
        System.out.println("Reading file in child");
        }
}
```

Point to Remember: In method overriding, the child class cannot throw broader or new checked exceptions than the parent method. It can throw the same or narrower checked exceptions.

Explanation:

- Parent.readFile() declares throws IOException, which is a checked exception.
- In Child, the overridden method declares throws Exception, which is more general than IOException.
- In Java, an overridden method can only throw the same or narrower checked exceptions , not a broader one.
- Since Exception is broader than IOException, the compiler throws an error

144 Output-Based 38: Overriding: Un-Checked Exceptions

```
class Parent {
            void connect() throws Exception{
        System.out.println("Connecting to server");
        }
}

class Child extends Parent {


        @Override
            void connect() throws NullPointerException {
        System.out.println("Connecting to child server");
        }
}
```

Explanation:

- Parent.connect() throws Exception, a checked exception.
- Child.connect() throws NullPointerException, which is an unchecked (runtime) exception.
- Point to Remember: In Java, Child class methods can throw unchecked exceptions freely, regardless of what the Parent declares

145 Output-Based 40: Overriding: Method Call in Constructor

```
class A {
    A() {
                show();
        }
        void show() {
                System.out.println("A show");
        }
}
    class B
    extends A { int
    x = 10; void
    show() {
                System.out.println("B show: " + x);
        }
}
public class Test {
        public static void main(String[] args) {
                new B();
        }
```

Explanation:

- new B(); starts object creation, which first calls A's constructor (because B extends A).
- Inside A() constructor, it calls show().
- Since show() is overridden in B, Java calls B's version , this is runtime polymorphism.
- But here is the catch: when A() is running, the B part of the object isn't fully initialized yet.
- So even though control goes to B.show(), the field 'x' in B hasn't been assigned 10 yet, 'x' holds the default value 0.
- That is why the output is: B show: 0.

146 Output-Based 41: Overriding: Method Call in Constructor with static

```
class A {
    A() {
                show();
        }
            static void show() {
        System.out.println("A show");
        }
}
    class B extends A {
    static int x = 10;
    static void show() {
                System.out.println("B show: " + x);
        }
}
public class Test {
        public static void main(String[] args) {
                new B().show();
        }
}
```

Explanation:

- new B() triggers object creation, which first calls the constructor of A, since B extends A.
- Inside A() constructor, it calls the static method show().
- Static methods are not overridden, they are hidden, so the version that gets called is based on the reference type at compile-time, not runtime.
- In this case, since show() is called from A's constructor, the method A.show() gets executed, not B.show().
- So, the line System.out.println("A show"); runs.
- After the object is fully created, new B().show() is executed in main(). Now the reference is of type B, so B.show() runs, and by this point, static variable x = 10 is initialized.
- Output is A show and B show: 10

147 Output-Based 42: Interface Default Method vs Class Method

```
class A {
            public void msg() {
                        System.out.println("Class");
            }
}
interface B {
            default void msg() {
                        System.out.println("Interface");
            }
}
```

```
class Test extends A implements B {
                    public static void
             main(String[] args) { Test m = new
             Test();
                          m.msg();
             }
```

Explanation:

- Class A has a regular instance method msg().
- Interface B has a default method msg().
- Class Test extends A and implements B.
- Point to remember: When msg() is called, Java gives priority to the class method over the default method from the interface.
- So, the output is: Class.

148 Output-Based 43: Overriding: Diamond Problem (Interface)

Explanation:

- Both Interfaces A and B have a default method with **same signature**: void show().
- When class C implements both A and B, and both interfaces have a default method with the same name, Java doesn't know which one to inherit, this is called Diamond problem.

```
interface A {
             default void show() {
        System.out.println("A");
         }
}
interface B {
             default void show() {
        System.out.println("B");
         }
}
class C implements A, B {
             public void
        show() {
        A.super.show();
                 B.super.show();
        System.out.println("C");
         }
}
public class Test {
         public static void main(String[] args) {
                 new C().show();
         }
```

- Class C implements both A and B, so Java forces it to override show() to resolve the conflict.
- Inside C's show() method, A.super.show() calls A's version and B.super.show() calls B's version
- The class can still access specific interface methods using InterfaceName.super.methodName()
- After calling both interface methods, System.out.println("C") prints "C".
- Final output is: A, B, C

149 Output-Based 44: Abstract Class: Main Method

```
abstract class Test {
            public static void main(String[] args) {
        System.out.println("Yes, abstract class can have main");
        }
}
```

Explanation:

- An abstract class can have a main method and run like any other class. Abstract class only prevents instantiation, not execution.

150 Output-Based 45: Abstract Class: Private Method

```
abstract class A {
        abstract private void run();
}
```

Explanation:

- An abstract method can't be a private method, it causes compile time error.

151 Output-Based 46: Abstract Class: Multiple abstract classes in chain

```
abstract class A {
        abstract void run();
}
abstract class B extends A {}
class C extends B {
        void run() {
                System.out.println("Running from C");
        }
}
public class Test {
            public static void main(String[]
        args) { A obj = new C();
                obj.run();
        }
}
```

Explanation:

- Class A defines an abstract method run(), and class B extends it without implementing run() so B is still abstract.
- Class C extends B and provides the actual implementation of run().
- In main(), A obj = new C(); it is a valid upcasting, and obj.run() calls C's version due to runtime polymorphism.
- Output is: Running from C.

152 Output-Based 47: Overloading: Type Promotion

```
class Test {
            void show(int a, long b) {
        System.out.println("int, long");
        }

            void show(long a, int b) {
        System.out.println("long, int");
        }

            public static void main(String[]
        args) { Test t = new Test();
                t.show(10, 20);
        }
}
```

Explanation:

- There are two overloaded methods: show(int, long) and show(long, int).
- We call t.show(10, 20), where both 10 and 20 are int.
- Java doesn't know which method to use, should it convert the first int to long or long to int?
- This casues compile time error.

153 Output-Based 48: Overloading: With Varargs vs Exact Match

```
class Test {
      void show(String s) {
    System.out.println("String");
    }

      void show(String... s) {
    System.out.println("Varargs");
    }

      public static void main(String[]
    args) { Test t = new Test();




    t.show("hello");
    }
}
```

Explanation:

- There are two overloaded show() methods: one takes a single String, the other takes a varargs (String...).
- We call t.show("hello") with a single String argument.
- Java gives **priority to the exact match** over varargs.
- So, show(String s) is chosen instead of show(String... s).
- Output: String.

154 Output-Based 49: Overloading: Autoboxing vs Widening

```
class Test {
            void show(Integer i) {
        System.out.println("Integer");
        }

            void show(long l) {
        System.out.println("long");
        }

            public static void main(String[]
        args) { Test t = new Test();
                t.show(10);
        }
}
```

Explanation:

- Two overloaded show() methods: one takes Integer, the other takes long.
- The call t.show(10) passes an int.
- Point to Remember: Java prefers **widening** over **boxing** during overload resolution.
- int to long is widening and int to Integer is boxing.
- So, java choose, long.

155 Output-Based 50: Overloading: Primitive Overloading

```
class Test {
            void show(String s) {
        System.out.println("String");
        }

            void show(Object o) {
        System.out.println("Object");



        }

            public static void main(String[]
        args) { Test t = new Test();
                t.show(null);
        t.show("CodingLyf");
        }
}
```

Explanation:

- Two overloaded show() methods: one takes String, the other takes Object.
- Point to remember: Overloading prefers more specific types when multiple matches are possible.
- t.show(null) passes null, which matches both, but String is more specific than Object
- "CodingLyf" is a String, so again show(String) is called.
- Output: String, String

156 Output-Based 51: Overloading and Inheritance

```
class A {}
class B extends A {}

class Test {
            void show(A a) {
        System.out.println("A");
        }

            void show(B b) {
        System.out.println("B");
        }

            public static void main(String[]
        args) { A obj = new B();
                Test t = new
        Test(); t.show(obj);
        }
}
```

Explanation:

- 'obj' is declared as type A but instantiated with new B().
- Point to Remember: Method overloading is resolved at compile-time based on reference type, not object type. Overloading depends on reference type, not runtime object type.
- Since obj is of type A, show(A a) is selected.
- Output: A.

157 Output-Based 52: Overloading: Reference vs Object Type

```
class Test {
            void show(Object o) {
        System.out.println("Object");
        }

            void show(String s) {
        System.out.println("String");
        }

            public static void main(String[]
        args) { Object obj = "Hello";
                Test t = new
        Test(); t.show(obj);
        }
}
```

This is different from Question No.50. Here we are passing Object.

Explanation:

- Two overloaded show() methods: one takes Object, one takes String.
- 'obj' is declared as Object but holds a String value.
- Point to remember: Overloading is based on reference type, not runtime value type.
- So, show(Object o) is chosen at compile-time.
- Output: Object

158 Output-Based 53: Super and This: Constructor

```
class A {
        A(String s) {
                System.out.println("A");
        }
}
    class B extends A
    { B() {
                System.out.println("B Default");
        }
}
public class Test {
        public static void main(String[] args) {
                new B();
        }
}
```

Explanation:

- Class A has only a parameterized constructor A(String s) , it has no default constructor.
- Class B extends A and its constructor must call super(...) explicitly if no default constructor exists in A.
- Point to remember: If the superclass does not contain a no-arg constructor, the subclass must explicitly call one of its available constructors.
- B() doesn't call super(...), so the compiler tries to insert super() but no-arg constructor doesn't exist in A.
- Result: Compile-time error (Implicit super constructor A() is undefined. Must explicitly invoke another constructor)

159 Output-Based 54: Super and This: Constructor

```
class A {
    A() {
                System.out.println("A");
        }
}
    class B extends A
    { B() {
                System.out.println("B");
        }
}
public class Test {
        public static void main(String[] args) {
                new B();
        }
}
```

Explanation:

Class A has a no-arg constructor that prints A.

Class B extends A and also has a no-arg constructor that prints B.

Point to remember: When creating a subclass object, the superclass constructor runs first.

In this example, Since A has no-arg constructor, when we call new B(), it first implicitly invoke the A's constructor.

Output: A, B

160 Output-Based 55: Super and This: Constructor Chaining

```
class A {
        A(int a) {
                System.out.println("A "+a);
        }
}
    class B extends A
    { B() {
                this(10);


                System.out.println("B Default");
        }
        B(int b) {
                super(b);
        System.out.println("B int");
        }
}
public class Test {
        public static void main(String[] args) {
                new B();
        }
}
```

Explanation:

- Class A only has a parameterized constructor A(int) that prints "A <value>".
- In B's constructor B(), the first statement is this(10), so it jumps to B(int) before executing "B Default".
- Point to remember: this() is used for constructor chaining within the same class, and it must be the first statement.
- Inside B(int), super(b) calls A's constructor A(int), which runs first because superclass constructors always execute before subclass constructors.
- Execution flow: A(int) -> print "A 10", back to B(int) -> print "B int", back to B() -> print "B Default".
- Output: A 10, B, B Default.

161 Output-Based 56: Super and This: Variable Access

```
class A {
        int x = 10;
}
class B extends A {
           void
      display() { x
      = 20;
                System.out.println(x);
        System.out.println(super.x);
        }
}
public class Test {
        public static void main(String[] args) {
                new B().display();
        }
```

Explanation:

new B().display();

Creates an object of B. Since B extends A, it also contains A's x variable.

Inside display()

- x = 20; changes the inherited x from 10 to 20.
- No data type is written here because we're not declaring a new variable, we're modifying an existing one inherited from A.
- Note: If we write int x = 20; that will create a new local variable inside display() and shadow the inherited one.

System.out.println(x); - Prints the x from the current object : 20.

System.out.println(super.x); - super.x also refers to the same x variable in A (since it wasn't shadowed in B). The value is now 20 because we modified it earlier.

```
@FunctionalInterface
interface MyFunc {
        void test();
            default void show() {
        System.out.println("Default");
        }
}

public class Test {
        public static void main(String[] args) {
                MyFunc f = () ->
        System.out.println("Lambda"); f.test();
                f.show();
        }
}
```

162 Output-Based 57: Functional Interface: Can a functional interface have default methods?

Answer: Yes, a functional interface can have any number of default or static methods but it must have exactly one abstract method.

Explanation:

1. @FunctionalInterface ensures the interface has only **one abstract method**.
2. MyFunc has one abstract method test() and one default method show().
3. Pointer to remember: Default methods don't break the functional interface rule because they have a body. Same rule applies even to static methods
4. A lambda expression is used to implement the single abstract method test().
5. Output: Lambda, Default

163 Output-Based 58: Functional Interface: Is Child class is a functional interface?

```
@FunctionalInterface
```

```
interface Parent {
        void baseMethod();
}
@FunctionalInterface
interface Child extends Parent {
        // No new abstract methods allowed except override
}
class Test {
        public static void main(String[] args) {
                Child c = () -> System.out.println("Child");
        c.baseMethod();
        }
}
```

Answer: Yes, Child class is a functional interface.

Explanation:

- Base is a functional interface with one abstract method baseMethod().
- Child extends Base but doesn't declare any **new** abstract methods , it just inherits baseMethod().
- Point to remember: A sub-interface remains a functional interface if it only inherits the single abstract method from its parent and doesn't declare any additional abstract methods.
- This means Child still has exactly one abstract method (baseMethod()), so it meets the functional interface rule.
- The lambda expression provides the implementation for that single abstract method, producing output: Child

164 Output-Based 59: Functional Interface: Will the code below compile?

```
@FunctionalInterface
interface MyFunc {
        void test();
}

public class Test {
        public static void main(String[] args) {
                int x = 10;
                MyFunc f = () ->
        System.out.println(x); x = 20;
                f.test();
        }
}
```

Answer: No, It won't compile.

Explanation:

- MyFunc is a functional interface with one abstract method test().
- In the lambda, x is used, so variables must be **final or effectively final**.
- Point to remember: Local variables captured in a lambda cannot be modified after they are used inside it.
- Since x is reassigned to 20, it is no longer effectively final.
- Result: Compile-time error

165 Output-Based 60: Functional Interface: Is this a valid Functional Interface?

```
@FunctionalInterface
    interface MyInterface
    { int calculate(int
    x); String
    toString();
}

class Demo {
            public static void main(String[]
        args) { MyInterface m = (a) -> a * a;
        System.out.println(m.calculate(5));
        }
```

Answer: Yes, it is a valid Functional Interface.

Explanation:

MyInterface declares one abstract method calculate(int) and overrides toString() from Object.

Methods from Object class are do **not** count as the abstract methods in a functional interface.

Point to remember: A functional interface can still have only one abstract method even if it overrides Object methods.

The lambda (a) -> a * a implements calculate(int).

Output: 25

166 Output-Based 61: Thread: Starting the thread twice?

```
public class Test implements Runnable {
            public void run() {
                        System.out.printf("CodingLyf");
            }
                  public static void main(String[] args) throws
            InterruptedException { Thread thread1 = new Thread(new Test());
                        thread1.start
            (); thread1.start();
                        System.out.println(thread1.getState());
            }
}
```

Answer: Exception, IllegalThreadStateException.

Explanation:

- A Thread object can be started only once , once it is started, it moves from NEW to other states.
- thread1.start() the first time works fine and runs run().
- The second thread1.start() call throws IllegalThreadStateException because the thread is no longer in NEW state.
- Point to remember**:** Once a thread has been started, it cannot be restarted; we must create a new Thread object.
- Output: First "CodingLyf" may print, then a runtime exception is thrown before printing

167 Output-Based 62: Thread: Call run() and start()?

```
public class Test extends Thread implements Runnable {
            public void run() {
                        System.out.printf("CodingLyf ");
            }
                  public static void main(String[] args) throws
            InterruptedException { Test obj = new Test();
                        obj.run();
                        obj.start();
            }
}
```

Explanation:

- Test extends Thread and implements Runnable, but Thread already implements Runnable, so the implements Runnable here is redundant.
- Calling obj.run() directly just executes it like a normal method in the main thread (no new thread is created).
- Calling obj.start() creates a new thread, which then calls run().
- Point to remember: run() is just a method; start() is what actually starts a new thread and then calls run() internally.
- Output: CodingLyf CodingLyf

168 Output-Based 63: Thread: Join ?

```
public class Test {
                  public static void main(String[] args) throws
            InterruptedException { Thread t1 = new Thread(() ->
            System.out.println("One"));

                        Thread t2 = new Thread(() -> {
                                    try {


                                                System.out.println("Two");
                                    } catch
                        (InterruptedException e) {
                        e.printStackTrace();
                                    }
                        });

                        t2.start();
                        t1.start();

                        t2.join();
            System.out.println("Three");
            }
```

Explanation:

Creating t1: A Thread is created with a lambda that simply prints "One". At this stage, the thread is in the **NEW** state and hasn't started yet.

Creating t2: Another Thread is created whose job is to first call t1.join() (meaning it will wait until t1 finishes) and then print "Two". It is also in the **NEW** state.

Starting t2 first: The main thread calls t2.start(). This moves t2 to the **RUNNABLE** state, and eventually the JVM schedules it. Inside t2's code, it reaches t1.join(). Since t1 hasn't started yet, t2

just waits for t1 to terminate in the future.

Starting t1: The main thread then calls t1.start(). Now t1 moves to **RUNNABLE**, the JVM picks it up, and it executes System.out.println("One"). Once it finishes, t1 moves to the **TERMINATED** state.

Unblocking t2: Because t1 is now terminated, t1.join() in t2 returns immediately. t2 resumes execution and prints "Two", then terminates.

Main thread waiting for t2: The main thread calls t2.join(). If t2 hasn't finished yet, the main thread will block until it does. In this case, it waits just until "Two" is printed.

Printing "Three": Once t2 has terminated, the main thread continues and prints "Three".

169 Output-Based 64: Thread: Create a deadlock

```
public class Test {
            public static void main(String[] args) throws
        InterruptedException { Thread mainThread = Thread.currentThread();

                    Thread t = new
            Thread(() -> {
            System.out.println("A");
            try {
                                mainThread.join();
                            } catch


                        }
                        System.out.println("B");
                });

                t.start();
                t.join();
        System.out.println("C");
        }
}
```

Explanation:

Thread.currentThread() returns the main thread object, stored in mainThread.

A new thread "t" is created with a task that prints "A", then tries mainThread.join() (waiting for the main thread to finish), and finally would print "B".

The main thread calls t.start(). This moves t to the **RUNNABLE** state, and the JVM schedules it.

As soon as t runs, it prints "A", then calls mainThread.join(). Now "t" is blocked, waiting for the main thread to finish execution.

Right after starting t, the main thread calls t.join(), which means the main thread is now blocked waiting for "t" to finish.

Deadlock: The main thread is waiting for t to finish, but "t" is waiting for the main thread to finish , neither can proceed. This is a **classic deadlock**.

Result: The only output is "A", after which the program hangs forever. "B" and "C" never print.

170 Output-Based 65: Instance vs Static ?

```
class Test {
        int a = 10;
        static int b = 20;

        public static void main(String[]
        args) { Test t1 = new Test();
                t1.a = 100;
                t1.b = 200;

                Test t2 = new Test();

                System.out.println("t1.a =" + t1.a + " t1.b ="
        + t1.b); System.out.println("t2.a =" + t2.a + " t2.b ="
        + t2.b);
        }
```

Answer:

t1.a =100 t1.b =200
t2.a =10 t2.b =200

Explanation:

- Instance variable a belongs to each object separately, so changing t1.a to 100 doesn't affect t2.a so it stays 10.
- Static variable b is shared by the class itself, not individual objects, so updating t1.b to 200 changes b for all instances.
- So, both t1.b and t2.b reflect the updated value 200.

171 Output-Based 66: Static variable declaration?

```
class Test {
            static int i = 1;

            public static void main(String[] args) {
                        static int i = 1;

            }
}
```

Answer: Compile-time Error

Explanation:

We cannot declare a local variable “I” as static inside a method; static is only allowed for class-level members.

172 Output-Based 67: Tricky question on runnable

```
class CodingLyf implements Runnable {
            public void run() {
                        System.out.println("Run");
            }
}

class Test {
                  public static void
            main(String[] args) { CodingLyf t
            = new CodingLyf(); t.start();
            System.out.println("Main");
            }
}
```

Answer: Compile-time Error

Explanation:

- CodingLyf implements Runnable, which means it only has the run() method.
- Runnable does not have a start() method.
- The start() method belongs to the Thread class.

Fix: Wrap the Runnable instance inside a Thread and call start() on that Thread:

```
CodingLyf t = new CodingLyf();
Thread thread = new Thread(t);
thread.start();
System.out.println("Main");
```

Java – Scenario Based and Custom Implementations

173 Java Scenario 1: If – Else Conditions or Switch.

If a customer is a "Premium" member and purchases more than 5 items, give 20% discount. If not premium but buys more than 10 items, give 10% discount. Otherwise, no discount.

```
public class DiscountCalculator {
        public static void main(String[] args) {
                String membershipType = "Premium"; // Change to "Regular" to test
                int itemsPurchased = 7;       // Change to test different cases

                int discount = switch (membershipType) {
                        case "Premium" -> (itemsPurchased > 5) ? 20 : 0;
                        default -> (itemsPurchased > 10) ? 10 : 0;
                };

                System.out.println("Membership: " + membershipType);
        System.out.println("Items purchased: " + itemsPurchased);
        System.out.println("Discount: " + discount + "%");
        }
}
```

Explanation:

- We can do with using if-else or switch. Above code is using switch expression from Java 17+.
- The switch directly evaluates membershipType.
- For "Premium", it checks if itemsPurchased > 5, else 0.
- For anything else (non-premium), it checks itemsPurchased > 10.

174 Java Scenario 2: ATM Operations - Switch.

Write logic to handle ATM operations:

- Ask user for operation type (withdraw, deposit, check_balance).
- If withdraw, check if the balance is sufficient; if yes, deduct amount, else show “Insufficient funds”.
- If deposit, add amount to balance.
- If check_balance, display balance.

Hint: Use a switch for operation type and if-else for balance validations.

Code link: JavaSamples/Scenario2_ATMOperations.java at main · CodingLyf-Fullstack/JavaSamples

175 Java Scenario 3: Traffic Light Action - Switch.

Given the current lightColor (RED, YELLOW, GREEN), determine the action:

- RED > "Stop"
- YELLOW > if pedestrianWaiting == true, print "Stop, pedestrian crossing"; else "Slow down".
- GREEN > if emergencyVehicle == true, print "Give way to emergency vehicle"; else "Go".

Hint: Use switch for light color, if-else inside each case for conditions.

Code link: JavaSamples/Scenario3_TrafficLightActions.java at main · CodingLyf-Fullstack/JavaSamples

176 Java Scenario 4: E-commerce Cart Total Calculation (For loop).

Given a list of cart items with price and quantity, loop through and:

- Calculate total bill
- Apply a 10% discount if total exceeds 5000
- Stop processing further items if total exceeds 10,000 (simulate "max cart limit")

Hint:

- Loops through each CartItem and adds price × quantity to the total.
- Stops the loop early if total > 10,000.
- Applies a 10% discount if the total is above 5,000

Code link: JavaSamples/Scenario4_CartBilling.java at main · CodingLyf-Fullstack/JavaSamples

177 Java Scenario 5: Password Strength Checker.

Given a string password, loop through characters and:

- Count uppercase, lowercase, digits, and special characters
- If all categories are present and length ≥ 8, print "Strong password" else "Weak password

Hint:

- Loops through each character.
- Uses Character methods for uppercase, lowercase, and digit detection.
- Treats anything else as a special character.
- Strength check: at least one from each category and length ≥ 8.

Code link: JavaSamples/Scenario5_PasswordChecker.java at main · CodingLyf-Fullstack/JavaSamples

178 Java Scenario 6: Printing Even and Odd Numbers with Two Threads

Two threads should print numbers from 1 to 20. One prints even numbers, the other prints odd numbers.

Hint:

- Use synchronization to ensure correct order

Code link: **JavaSamples/Scenario6_OddEvenPrinter.java at main · CodingLyf-Fullstack/JavaSamples**

Explanation:

1. Two threads start with different responsibilities

- Odd thread calls printOdd(1, 3, 5...)
- Even thread calls printEven(2, 4, 6...)

- Both share the same Printer object, so synchronization works on its monitor.

2. The isOdd flag controls whose turn it is
- Initially isOdd is false (meaning it is Odd thread turn).
- Odd thread should run first, Even thread must wait until isOdd becomes true.

3. Even thread hits printEven() and
   blocks while (!isOdd) { wait(); }
- Since isOdd is false initially, this condition is true.
- wait() is called, which releases the lock on Printer and puts Even thread into *waiting state*.
- It stays parked until someone calls notify() on the same monitor.

4. Odd thread gets the lock and runs
   printOdd() while (isOdd) { wait(); }
- Here, isOdd is false, so it skips waiting.
- Prints odd number, sets isOdd = true, calls notify() to wake up the Even thread.

5. Key point about wait() here
- Odd waits if it is not its turn (isOdd == true).
- Even waits if it is not its turn (isOdd == false).
- wait() is the mechanism that *releases the lock and pauses the thread* until notify() signals that it is safe to proceed.

Reference: **Print Even and Odd Numbers Using 2 Threads | Baeldung**

179 Java Scenario 7: Sequential Execution of Two Threads Printing Numbers (1,2,3…) and Letters(A,B,C,..) in Java. So that it prints (A,1B,2…)

Hint:

- Use synchronization to ensure correct order

Code link: JavaSamples/Scenario7_AplhaNumericSequence.java at main · CodingLyf-Fullstack/JavaSamples

Explanation:

Check the code, it contains all necessary explanation in the comments.

- We create two threads; one for letters, one for numbers.
- we create a shared lock “monitor” .
- We also keep a flag called isLetter. If it is false, it is the letter thread’s turn. If it is true, it is the number thread’s turn.
- Each thread checks this flag in a while loop. If it is not their turn, they go to sleep using monitor.wait() until the other thread wakes them up.

- When a thread does print its value, it flips the flag to give the turn to the other thread, then calls monitor.notify() to wake them up.
- We start with isLetter = false, so the letter thread goes first, prints “A”, and hands the turn to the number thread.
- This back-and-forth continues until we’ve gone through all 26 letters and numbers in sequence.

180 Java Scenario 8: Producer Consumer Problem

One thread produces integers into a shared queue, another consumes them. Implement using wait() and notify() for thread coordination.

Solution:

This can be done in two ways

1. Using Synchronized and inter process communication(wait() and notify())
2. Using BlockingQueue

Explanation:

a. Synchronized(wait() and notify()):

1. Create two threads t1 and t2, t1 is produce the value and t2 is to consume the value.
2. Create a ShareQueue class, In which we take a Queue or LinkedList. We need to how much capacity this Queue should be.
3. SharedQueue class contains two synchronized methods produce() and consume(). These methods are invoked by the two threads.
4. If Queue is full, make producer to wait, until consumer consumes the queue.
5. If Queue is empty, make consumer to wait, until producer pushes the values to the queue.
6. In this sample, we are producing 20 values and consuming till queue is empty. Once queue is empty thread stops execution.
7. Here is the code link:

b. Using BlockingQueue

A BlockingQueue is just a queue with built-in thread safety and blocking behavior. It is in java.util.concurrent and is mostly used when we have producer-consumer kind of situations.

How it works?

When we try to put something in and the queue is full -> the thread will wait until space is available. When we try to take something out and the queue is empty -> the thread will wait until there’s something to take.

This removes the headache of manually handling wait() and notify(); the queue does it for us.

Blocking Operations:

- put(E e): Inserts the specified element into this queue, waiting if necessary for space to become available.
- take(): Retrieves and removes the head of this queue, waiting if necessary until an element becomes available.

Implementations:

- ArrayBlockingQueue: A bounded blocking queue backed by an array.

- LinkedBlockingQueue: An optionally bounded blocking queue backed by a linked list.
- PriorityBlockingQueue: An unbounded blocking queue that orders elements according to their natural ordering or a custom Comparator.

Reference for producer consumer problem: Java BlockingQueue Example | DigitalOcean

Code link: JavaSamples/Scenario8_ProducerConsumer.java at main · CodingLyf-Fullstack/JavaSamples

181 Java Scenario 9: Simulating the deadlock

Create two threads, each trying to lock two resources in opposite order, causing a deadlock. Then show how to avoid it.

Code links:

Deadlock Example: JavaSamples/Scenario9_Deadlock.java at main · CodingLyf-Fullstack/JavaSamples

Explanation: In this example, thread1 acquires resource1 and then tries to acquire resource2.

Simultaneously, thread2 acquires resource2 and then tries to acquire resource1.

This creates a circular dependency where thread1 holds resource1 and waits for resource2 (held by thread2), and thread2 holds resource2 and waits for resource1 (held by thread1), leading to a deadlock

Deadlock Prevention Example: JavaSamples/Scenario9_DeadPrevention.java at main · CodingLyf-Fullstack/JavaSamples

In the corrected example, both thread1 and thread2 acquire resource1 before attempting to acquire resource2.

This consistent ordering prevents the circular wait condition, thereby avoiding deadlock.

If thread1 acquires resource1 first, thread2 will wait for resource1 to be released. Once thread1 releases both locks, thread2 can then acquire them in the correct order and proceed.

182 Java Scenario 10: Increment the shared counter with threads

Create 10 threads, each updating a shared counter. The goal is for the final count to be 10,000.

Solution: Use AtomicInteger for thread-safe increments.

We'll look at two examples:

1. Using a normal int (shows an incorrect result due to race conditions).
2. Using AtomicInteger (produces the correct result).

Code links:

With Integer: JavaSamples/Scenario10_SharedCounter_Int.java at main · CodingLyf-Fullstack/JavaSamples

AtomicInteger: JavaSamples/Scenario10_SharedCounter_Atomic.java at main · CodingLyf- Fullstack/JavaSamples

183 Java Scenario 11: Bank Account Withdrawal

Multiple threads withdraw from the same account. How do you make sure to avoid race conditions. What techniques you would use?

Solution: We should use synchronized blocks. Here is the complete sample.

Code link: JavaSamples/Scenario11_BankAccount.java at main · CodingLyf-Fullstack/JavaSamples

184 Java Scenario 12: CompletableFuture Scenario

Imagine you are calling two external services:

1. Fetch user details from Service A.
2. Fetch user's recent orders from Service B.

We don't want to wait for one to finish before starting the other. How will you implement it?

Solution:

Using CompletableFuture. It allows us to run tasks asynchronously and combine results when ready. It is Great for parallel tasks like API calls, DB queries, or expensive computations.

```
public class CompletableFutureExample {

    public static void main(String[] args) throws
ExecutionException, InterruptedException {

                // Async task 1
                CompletableFuture<String> userFuture = CompletableFuture.supplyAsync(()
        -> {
                        simulateDelay(1000);
                        return "User: CodingLyf";
                });

                // Async task 2
                CompletableFuture<String> ordersFuture =
        CompletableFuture.supplyAsync(() -> {
                        simulateDelay(1500);
                        return "Orders: 5";
                });

                // Combine both results
        CompletableFuture<String> combinedFuture = userFuture.thenCombine(ordersFuture,



                });

                System.out.println(combinedFuture.get()); // Waits for both tasks to
        complete
        }

        private static void simulateDelay(int millis) {
        try { Thread.sleep(millis); } catch (InterruptedException e) {
e.printStackTrace(); }
        }
```

CompletableFuture.supplyAsync() runs a task in a separate thread, thenCombine() helps to combine

the results of two independent futures, and the execution remains non-blocking until we explicitly call .get() or .join().

185 Java Scenario 13: Payment Gateway Integration

You need to build a payment API that must support multiple payment methods (UPI, Card, Wallet) and allow third parties to add new methods in the future.

What would you choose? Interface, Abstract Class, or Static Method?

Solution:

Static methods won't work here because the code is fixed. We will have to change it every time we add a new payment method, which makes it hard to grow in the future.

Abstract classes can give some default behavior, but they only allow inherit from one class. That can limit third parties if they already have another base class.

Interfaces are the best option. We can add new payment methods without touching the existing code.

```
//Payment method contract
interface PaymentMethod {
            void pay(double
      amount); String
      getName();
}

//Existing implementations
class UpiPayment implements PaymentMethod {
            //Implement the methods
}

class CardPayment implements PaymentMethod {
            //Implement the methods
```

Code link: JavaSamples/Scenario13_PaymentIntegration.java at main · CodingLyf-Fullstack/JavaSamples

186 Java Scenario 14: Notification Service

Different channels like Email, SMS, Push Notification must follow the same contract (sendMessage), but each channel is completely independent.

The app needs to send notifications through Email, SMS, and Push Notifications.

- Create a Notifier interface with sendNotification(String message, String to).
- Implement each type separately.

- Show how dependency injection can let you swap implementations.
-

```
interface Notifier {
        void sendNotification(String message, String to);
}

    class EmailNotifier implements Notifier {
    @Override
            public void sendNotification(String message, String to) {
        System.out.println("Sending EMAIL to " + to + ": " + message);
        }
}

    class SmsNotifier implements Notifier
    { @Override
            public void sendNotification(String message, String to)
        { System.out.println("Sending SMS to " + to + ": " +
        message);
        }
```

Solution:

Interfaces are the best choice because they let us add new payment methods without changing any existing code, keeping the design flexible and future-proof.

Code link: JavaSamples/Scenario14_NotificationService.java at main · CodingLyf-Fullstack/JavaSamples

187 Java Scenario 15: File Export System

You need to build a reporting system that can export data to PDF, Excel, or CSV.

- Create an Exporter interface with export(List<String> data, String fileName).
- Implement the interface for each format and allow users to choose export type at runtime.

Hint: Use Interfaces

188 Java Scenario 16: Authentication Strategies

Your app should allow Username/Password login, OAuth login, and Biometric login.

- Create an interface `Authenticator` with boolean authenticate(String user, String credential).
- Implement separate classes for each authentication type.

Hint: Use Interfaces.

189 Java Scenario 17: Employee Payroll System

All employees have name, salary, and a method calculateBonus(), but bonus calculation differs for FullTimeEmployee and ContractEmployee..

How do you design this, Using interface or abstracts class?

Solution:

An abstract class fits better here because:

- All employees have common fields (name, salary) ; abstract classes can store these, interfaces can't.
- We can keep shared code in one place and only make subclasses implement the different

part (calculateBonus()).

- calculateBonus() can be an abstract method so each type of employee defines its own formula.

In short: use an abstract class when all types share fields and some common methods, but each has its own version of part of the logic.

Code link: JavaSamples/Scenario17_EmployeePayroll.java at main · CodingLyf-Fullstack/JavaSamples

190 Java Scenario 18: Bank Account Types

In a banking app, savings and current accounts share logic like deposit() and calculateInterest() but have different interest rules.

```
//Abstract class with common fields and behavior
 abstract class Employee {
 protected String name;
 protected double salary;

  public Employee(String name, double salary) {
          this.name = name;
          this.salary = salary;
  }

  // Abstract method: subclasses must implement their own bonus logic
  public abstract double calculateBonus();

  public void displayInfo() {
          System.out.println("Name: " + name + ", Salary: " + salary);
  }
}
```

Hint:

- Create an abstract class BankAccount with deposit() and abstract calculateInterest() method.
- Implement for SavingsAccount and CurrentAccount.

191 Java Scenario 19: Custom Exceptions

When creating a custom exception, how do you decide whether to extend Exception or RuntimeException?

Explanation:

When creating a custom exception in Java, the choice between extending Exception or RuntimeException depends on whether we want to create a checked exception or an unchecked exception.

1. Extend Exception for Checked Exceptions:

If the exception represents a condition that a client (calling code) can be handle, we should create a checked exception.

Example 1: InsufficientBalanceException

Imagine a bank withdrawal method. If someone tries to take ot more than they have, we throw this exception. The ATM or mobile app can catch it and display, “Not enough funds.” The program

continues running happily.

Example 2: FileFormatException

The program only accepts .csv files for importing data. If someone uploads a .pdf, we throw this. The caller can say, “Wrong file format. Please upload a CSV.”

Example 3: OrderNotFoundException

In an e-commerce site, a customer checks the status of an order ID that doesn’t exist. We throw this so the UI can show “Order not found” instead of a system crash.

2. Extend RuntimeException for Unchecked Exceptions:

If the exception represents a programming error, a fundamental flaw in the application logic, or a condition that cannot be recovered by the client, we should create an unchecked exception.

Example 1: PaymentProcessingFailedException
The payment service has a serious error , maybe wrong credentials or the provider is down. The app can’t fix this during runtime.

```
import java.io.*;

  public class FileReadExample {
          public static void main(String[] args) {
                  // try-with-resources automatically closes the stream
          try (InputStream input = new FileInputStream("data.txt")) {
                          int data;
                              while ((data = input.read()) != -
                1) { System.out.print((char) data);
                          }
                      } catch (IOException e)
             { e.printStackTrace();
                  }
          }
  }
```

Example 2: DatabaseConnectionFailedException
The app can’t connect to the database at all. Without that, nothing works, so it should fail immediately.

192 Java Scenario 20: Closing resources

How do you make resources like Inputstream or files or DB connections close after you done with your work?

Explanation:

In Java, We need to use **try-with-resources** to close the resources like InputStream, Reader, or File handles.

How this works?

- Any class that implements **AutoCloseable** (like InputStream, OutputStream, Reader, Writer) will be closed automatically when the try block ends , even if an exception happens.
- This removes the need for a finally block.

Reference: Java's AutoCloseable Interface Explained | Medium

193 Java Scenario 21: Creating Custom Exception

You have an Interface EmailService with sendEmail(String to, String subject, String body) that throws InvalidEmailException or EmailServerDownException.

Implementations are SMTPEmailService and

SendGridEmailService. How do you design the custom

exceptions?

Explanation:

We should create 2 Exception classes that extends Exception.

```
//Checked exception for invalid email addresses
class InvalidEmailException extends Exception {
public InvalidEmailException(String message) {
          super(message);
  }
}

//Checked exception for server issues
class EmailServerDownException extends Exception {
  public EmailServerDownException(String message) {
          super(message);
}
```

Code link: JavaSamples/Scenario21_CustomException.java at main · CodingLyf-Fullstack/JavaSamples

194 Java Scenario 22: Global Math Operations

You need add, subtract, multiply, divide methods that same for the whole application and how do you design these methods?

Explanation:

We should create them as static methods as they won't depend on any object state.

```
                    return a / b;
            }
}
```

195 Java Scenario 23: Logging

Your system needs a log(message) function that can be called from anywhere without creating a logger object?

Hint:**Use Static.**

Explanation:

We should create them as static methods as they won't depeer any object state.

```
@FunctionalInterface
interface Calculator {
        int calculate(int a, int b);
}

public class Test {
            public static void main(String[]
        args) { Calculator add = (a, b) -> a +
        b; Calculator multiply = (a, b) -> a *
        b;



       System.out.println(multiply.calculate(5, 3)); // 15

                    }
  }
                                                    // 8
```

196 Java Scenario 24: Creating Custom Functional Interface

How do you create and use a functional interface? Write a simple interface (Calculator) and demonstrate how to implement it using a lambda expression.

197 Java Scenario 25: Collections - 1

You have a list of recently visited URLs. The order of visits matters, but a URL should only appear once. Which collection will you use and why?

Solution: Use a **LinkedHashSet** ; it preserves insertion order and prevents duplicates.

198 Java Scenario 26: Collections - 2

You are building a leaderboard where player names are keys and scores are values. You need constant- time lookup by name and to iterate over them in sorted order of names. Which collection type works here?

Solution: Use a **TreeMap<String, Integer>** ; it keeps keys sorted in natural or custom order while allowing O(log n) operations for insert, lookup, and remove.

199 Java Scenario 27: Collections - 3

You have a List<Integer> with 1 million numbers. You need to remove all numbers less than 100 while iterating.
What's the correct way to do this without hitting

ConcurrentModificationException? Solution: Use an **Iterator** and its remove()

method:

```
Iterator<Integer> it = list.iterator();
    while (it.hasNext()) {
            if (it.next() < 100) {
       it.remove();
          }
}
```

This avoids ConcurrentModificationException because remove() updates the iterator's internal state safely.

200 Java Scenario 28: Collections - 4

You have a Map<String, Integer> with employee names and their performance scores. You need the top 3 performers in descending order of score. How will you achieve this?

Solution: We can achieve this by Collections.sort and also we can use Streams.

```
public class Test {
            public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();
        scores.put("Mahesh", 85);
                scores.put("NTR", 95);
                scores.put("Charan", 92);
                scores.put("Prabhas", 88);

                List<Map.Entry<String, Integer>> list = new
        ArrayList<>(scores.entrySet());

                // Sort by value in descending order
                Collections.sort(list, (e1, e2) ->
        e2.getValue().compareTo(e1.getValue()));

                        System.out.println(e.getKey() + " -> " + e.getValue());
                }
        }
}
```

201 Java Scenario 29: Collections - 5

You have two lists of customer emails. Merge them into one, preserving order of first appearance, without duplicates. Which collection will you use?

Solution: Use a LinkedHashSet, it removes duplicates while keeping the insertion order. Add all emails from the first list, then from the second list.

202 Java Scenario 30: Collections - 6

You have a string and need to find the first character that appears only once. Which collection will help track occurrence and maintain original order efficiently?

Solution: Use a **LinkedHashMap<Character, Integer>**; it preserves insertion order and allows us to store character counts.

After populating it, iterate over the entries and return the first key with a count of 1.

```
public class Test {
	public static void main(String[] args) {
		String str = "swiss";

		Map<Character, Integer> map = new LinkedHashMap<>();

		// Count occurrences
		for (char c : str.toCharArray()) {
			map.put(c, map.getOrDefault(c, 0) + 1);
		}

		// Find first character with count 1
		for (Map.Entry<Character, Integer> entry : map.entrySet()) {
			if (entry.getValue() == 1) {
				System.out.println("First non-repeating character: " +
entry.getKey());

			}
		}
return;
```

```
            System.out.println("No unique character found");
        }
    }
```

Note: We can also use Streams to achieve this

```
public class Test {
        private static final int MAX_SIZE = 5;
        private Queue<String> logs = new ArrayDeque<>();

        public void addLog(String log) {
                if (logs.size() == MAX_SIZE) {
            logs.poll(); // remove oldest
            }
            logs.offer(log); // add newest
    }

        public void printLogs() {
        logs.forEach(System.out::println);
    }

       public static void main(String[] args) {
       Test logger = new Test();

                for (int i = 1; i <= 10; i++) {
            logger.addLog("Log " + i);
            }

            logger.printLogs(); // prints last 100
}
```

203 Java Scenario 31: Collections - 7

You are building a real-time logging system that stores only the last 100 log entries in memory. Old entries should automatically be removed as new ones are added. Which collection will you choose?

Solution: Use an **ArrayDeque** (or LinkedList) and enforce the size manually.
On each insert, if size exceeds 100, remove from the head (poll()), then add the new log to the tail (offer()).

Note: In the sample, we have taken Max Size as 5, when we try to insert 10 items, first 5 will be deleted.

204 Java Scenario 32: Collections – 8

Given a list of product IDs sold in a day, you want to find the product sold the most. Which collection will you use, and what will be your approach?

Solution: Use a **HashMap<Integer, Integer>** to store each product ID and its sale count.
Iterate through the list, update counts with getOrDefault(), then scan the map to find the entry with the highest value.

```
public class Main {
            public static void main(String[] args) {
                        List<Integer> sales = Arrays.asList(101, 102, 101, 103, 101,
            102, 103, 103,
```

```
                Map<Integer, Integer> countMap = new HashMap<>();

                // Count occurrences
                for (int id : sales) {
                        countMap.put(id, countMap.getOrDefault(id, 0) +
        1);
                }

                // Find product with max sales
                int maxProduct = -1, maxCount = 0;
                for (Map.Entry<Integer, Integer> entry : countMap.entrySet())
    {
                                if
                (entry.getValue()          >
                maxCount)  {  maxProduct  =
                entry.getKey();  maxCount  =
                entry.getValue();
                        }
                }
```

205 Java Scenario 33: LRU Cache Implementation

You are implementing a cache where the most recently used item should stay, and the least recently used should be evicted once capacity is reached. Which Java collection can do this with minimal custom code?

Solution: Use a **LinkedHashMap** with accessOrder = true, it maintains entries in the order they were last accessed.
Override removeEldestEntry() to automatically evict the least recently used when capacity is exceeded.

Code Link: JavaSamples/Scenario33_LRUCache_Implementation.java at main · CodingLyf-Fullstack/JavaSamples

206 Java Scenario 34: Task Scheduler by Priority

You need to schedule jobs where each job has a priority number. Highest priority should be executed first, lowest last.

Solution: Use PriorityQueue

```
/ Max-heap style: highest priority first
PriorityQueue<Job> queue = new PriorityQueue<>(
                        (j1, j2) -> Integer.compare(j2.priority, j1.priority)
);

queue.add(new Job("Email Report", 2));
queue.add(new Job("Database Backup", 5));


queue.add(new Job("UI Refresh", 3));

    while (!queue.isEmpty())
    { Job job =
    queue.poll();
        System.out.println("Executing: " + job);
```

PriorityQueue is normally a min-heap (smallest element first), so we provide a custom comparator to reverse it.

The loop executes jobs from highest priority to lowest.

Code Link: JavaSamples/Scenario34_TaskScheduler.java at main · CodingLyf-Fullstack/JavaSamples

207 Java Scenario 35: Detecting Duplicate Usernames

Given a stream of usernames during registration, detect immediately if the username is already taken without scanning the entire list. How will you use HashSet here?.

Solution:

Use a **HashSet<String>** to store all registered usernames.
Before adding a new one, check set.contains(username).
This gives **O(1)** average-time lookup without scanning the list.

```
public boolean register(String username) {
            if (usernames.contains(username)) {
        System.out.println("Username already taken: " + username);
        return false;
        }
        usernames.add(username);
    System.out.println("Registered: " + username);
    return true;
}
```

208 Java Scenario 36: Auto-Suggest in Search

You have a collection of product names. As the user types a prefix, show suggestions in sorted order starting from that prefix. Which collection do you use?

Solution:

We can use a **TreeSet<String>** since it stores elements in sorted order and supports range queries using subSet().

```
public class Test {
    public static void main(String[] args) {
        TreeSet<String> products = new TreeSet<>();

                products.add("apple");
        products.add("apricot");
        products.add("banana");
        products.add("blueberry");
        products.add("blackberry");
        products.add("cherry");

                String prefix = "bl";
                String end = prefix + Character.MAX_VALUE; // ensure all starting with
        prefix

                NavigableSet<String> suggestions = products.subSet(prefix, true, end,
        true); System.out.println("Suggestions for '" + prefix + "': " + suggestions);
        }
```

How it works:

- TreeSet keeps items in natural sorted order.
- subSet(prefix, true, end, true) efficiently fetches only items starting with that prefix , no full scan needed.

209 Java Scenario 37: Stack – Parentheses Balancer

You have a collection of product names. As the user types a prefix, show suggestions in sorted order starting from that prefix. Which collection do you use?

Solution:

We can use a **Stack<Character>**:

- **Push** opening brackets ((, {, [) onto the stack using push().
- **Pop** when we encounter a closing bracket and check if it matches the top.

```
public class Main {
	public static void main(String[] args) {

		System.err.println(isBalanced("{[()]}"));
		System.err.println(isBalanced("{[(])}"));
	}

	static boolean isBalanced(String str) {
		Stack<Character> stack = new Stack<>();
		for (Character ch : str.toCharArray()) {
			if (ch == '(' || ch == '{' || ch == '[') {
				stack.push(ch);
			} else if (ch == ')' || ch == '}' || ch == ']') {
			  {
				if (stack.isEmpty())
					return false;
				Character top = stack.pop();
				if (ch == ')' && top != '(' ||
				    ch == '}' && top != '{' ||
				    ch == ']' && top != '[') {
				   return false;
				}
			  }

			}
		}
		return stack.isEmpty();
	}
}
```

210 Java Scenario 38: Thread safe list

Multiple threads are adding elements to a list at the same time. Which collection will you choose?

Solution:

- Use **CopyOnWriteArrayList**.
- it allows concurrent reads without locking because it works on a snapshot of the array.
- Writes (add, remove) create a new copy of the array, so reads never block.

211 Java Scenario 39: Counting API Requests Per User

Multiple threads are recording the API hits for different users in real time. Which collection you would use to keep counts without explicit synchronization?

Solution:

Use ConcurrentHashMap<String, AtomicInteger> so increments are thread-safe without explicit locking.

```
public class ApiRequestCounter {
        private ConcurrentHashMap<String, AtomicInteger> counts = new
    ConcurrentHashMap<>();

        public void recordRequest(String user) {
                counts.computeIfAbsent(user, k -> new
        AtomicInteger()).incrementAndGet();
        }

        public int getCount(String user) {
                return counts.getOrDefault(user, new AtomicInteger()).get();
        }

            public static void main(String[] args) {
        ApiRequestCounter counter = new ApiRequestCounter();
        counter.recordRequest("Dhoni");
        counter.recordRequest("Dhoni");
        counter.recordRequest("Virat");



        System.out.println("Virat: " + counter.getCount("Virat"));      // 1
    }
}
```

212 Java Scenario 40: ArrayList vs LinkedList

You are storing the last 1 million stock price updates in memory.

- Every new price gets added **at the end**.
- Once you reach 1 million, you remove the **oldest price** (front of the list).
- You also frequently check the price at random positions in the

list. Which list implementation would you choose, and why?

Solution:

Use ArrayList for fast random access and for quick insertions at the end.

LinkedList: better for frequent middle insertions/deletions since no shifting of elements, just pointer updates.

ArrayList: better for fast random access and adding at end because it uses an array under the hood

213 Java Scenario 41: Large file handling

How would you read a large file efficiently without running out of memory?

Solution:

When dealing with large files, the key is to read the file in chunks rather than loading the entire file

into memory. This can be done efficiently using BufferedReader or FileInputStream in Java. By processing the file line-by-line or in smaller byte chunks.

```
public class LargeFileReader {
        public void readFile(String filePath) {
                    try (BufferedReader reader = new BufferedReader(new
            FileReader(filePath))) { String line;
                        while ((line = reader.readLine()) != null) {
                                // Process
                each line
                System.out.println(line);
                        }
                    } catch (IOException e) {
            e.printStackTrace();
                }
        }
```

214 Java Scenario 42: Memory Leaks

How would you detect and prevent memory leaks that cause OutOfMemory in a Java application??

Common causes of memory leaks

- Static fields holding onto large objects for the entire life of the program.
- Long-lived collections (like List or Map) that keep growing because items are never removed.
- Listeners, callbacks, or threads that are registered but never unregistered.
- Unclosed resources like streams, sockets, or database connections.

We would need to 2 steps to follow.

1. **Identify the memory leaks**
2. **Fix to avoid the leaks**

To detect memory leaks, I'd use profiling tools like VisualVM, YourKit, or JProfiler. These tools allow me to monitor heap usage and identify objects that are not being garbage collected.

Preventing memory leaks often involves practices like:

- Avoiding static references: Ensure that static fields don't hold onto objects longer than necessary. Properly closing resources:
- Use try-with-resources to ensure resources like streams and connections are closed automatically.
- Weak References: Use weak references for cache implementations to allow garbage collection when memory is needed.

Reference: Java Memory Leaks: Detection & Fix | Medium

215 Java Scenario 43: Optimize the GC

How would you optimize an application to reduce the impact of garbage collection?

To reduce the impact of garbage collection (GC) in an application, follow these steps:

1. **Reduce Object Creation**: Avoid creating unnecessary objects. Reuse objects when possible instead of creating new ones.
2. **Use Primitive Types:** Prefer int, double, etc., over their object versions (Integer, Double) to

avoid extra memory usage.

3. **Limit Large Collections:** Clear or reuse large lists, maps, or arrays instead of letting them grow endlessly.
4. **Avoid Memory Leaks:** Remove references to objects (like event listeners or caches) when they're no longer needed.
5. **Tune GC Settings:** Adjust JVM flags (like -Xmx, -Xms) to optimize heap size and GC behavior based on the app's needs.
6. **Use Object Pools:** For frequently created/destroyed objects (like database connections), use pooling (e.g., ArrayList pooling).
7. **Profile & Monitor:** Use tools (like VisualVM) to find memory hotspots and optimize them.

216 Java Scenario 44: Best coding practices

What are the best practices that you follow while developing java application?

1. Write Clean, Readable Code
   - Follow consistent naming conventions (camelCase for variables, PascalCase for classes).
   - Keep methods small and focused ; one responsibility per method.
   - Use meaningful names (instead of calc() write calculateTax()).
2. Follow SOLID Principles
   - Single Responsibility: Each class does one thing well.
   - Open/Closed: Open for extension, closed for modification.
   - Liskov Substitution: Derived classes should be usable as their base type.
   - Interface Segregation: Keep interfaces simple.
   - Dependency Inversion: Depend on abstractions, not concrete classes.
3. Handle Exceptions Properly
   - Don't swallow exceptions silently (catch (Exception e) {} is bad).
   - Use custom exceptions where it adds clarity.
4. Optimize Collections Usage
   - Choose the right type (ArrayList vs LinkedList, HashMap vs TreeMap).
   - Avoid unbounded collections for data that grows indefinitely.
   - Prefer Collections.unmodifiableList() when returning read-only data.
5. Manage Resources Safely
   - Use try-with-resources for streams, files, sockets, DB connections.
   - Close resources in the same scope we open them.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
        // work
}
```

6. Keep Concurrency Safe
   - Use concurrent collections (ConcurrentHashMap, CopyOnWriteArrayList) when needed.
   - Avoid unnecessary synchronization , it hurts performance.

- Use ExecutorService instead of creating raw threads.

7. Avoid Hardcoding
   - Move constants to config files or environment variables.
   - Use .properties or .yaml for application settings.
8. Write Unit & Integration Tests
   - Test edge cases, not just happy paths.
   - Use frameworks like JUnit, Mockito.
9. Memory Usage
   - Avoid creating unnecessary objects in loops.
   - Reuse immutable objects like String carefully.
   - Watch out for memory leaks from static references or listeners.
10. Follow Java Language Features Wisely
    - Prefer StringBuilder for concatenation in loops.
    - Use Optional to avoid null checks when it makes sense.
    - Use record for DTOs (Java 16+).
11. Use Logging Properly
    - Prefer a logging framework (SLF4J, Logback) over System.out.println.
    - Log only meaningful info.
    - Keep sensitive data out of logs.
12. Keep Build & Dependencies Clean
    - Avoid unused dependencies.
    - Keep dependencies up to date.
    - Use dependency management tools like Maven or Gradle consistently.

217 Java Scenario 45: Performance Optimization

What do you optimize the performance of a Java application?

1. **Profiling and Monitoring:** We can use profiling tools like JProfiler to identify performance bottleneck and to monitor the CPU usage, memory consumption, and response times.
2. **Database Optimization:** Data is very important part of every application, so we need to optimize the SQL queries. Optimize queries by adding appropriate indexes, rewriting complex joins, and reducing the number of queries by implementing batch processing.
3. **Code Refactoring:** Optimized loops and algorithms to reduce time complexity. I also removed unnecessary synchronization to improve thread performance (Check above question for coding practices).
4. **Caching:** Implemented caching strategies using tools like Ehcache to reduce the load on the database and improve response times for frequently accessed data.
5. **Garbage Collection Tuning:** Adjusted JVM garbage collection settings to reduce pause times and improve memory management.

218 Java Scenario 46: Implementing the Retry

How do you implement retry using Java.

Explanation:

We can implement retry using

1. Thread – A simple retry
2. Callable
3. Custom Interface

1. Thread – A simple retry

This code tries to run performOperation() up to 3 times, waiting 1 second between each retry if it fails. It simulates random failure with a 70% chance, and stops early if the operation succeeds.

If all retries fail, it prints "Retries exhausted**".**

Code Link: JavaSamples/Scenario46_SimpleRetry.java at main · CodingLyf-Fullstack/JavaSamples

2. Callable

This code defines a generic executeWithRetry method that runs a Callable and retries it up to a given number of times with a delay between attempts.

It stops and returns immediately if the operation succeeds, otherwise it keeps retrying until the max retries are reached.

If all attempts fail, it throws the last caught exception to the caller.

Code Link: JavaSamples/Scenario46_Retry_Callable.java at main · CodingLyf-Fullstack/JavaSamples

3. Custom Interface

Java — Retry Pattern. Ah, the joys of development! Dealing… | by Lucas Amado | Medium

219 Java Scenario 47: Implementing stack using arrays

How do you implement stack using arrays in java.

Explanation:

A stack is a fundamental data structure in computer science that follows the last-in, first-out (LIFO) principle. Imagine a stack of plates: the last plate we add is the first one we can remove. In programming, stacks are used to manage function calls, track undo/redo actions, and more.

Key Concepts:

- LIFO (Last-In, First-Out): The last element added to the stack is the first one to be removed.
- Push: Adds an element to the top of the stack.
- Pop: Removes and returns the top element from the stack.
- Peek: Returns the top element without removing it.
- isEmpty: Checks if the stack is empty.
- Size: Returns the number of elements in the stack.

Code Link: JavaSamples/Scenario47_StackWithArray.java at main · CodingLyf-Fullstack/JavaSamples

220 Java Scenario 48: Implementing stack using 2 Queues

How do you implement stack using queues in java.

Explanation:

In Java, a Queue is an interface within the Java Collections Framework that represents a collection designed for holding elements prior to processing, typically in a First-In, First-Out (FIFO) manner. This means the element added first to the queue is the first one to be removed.

Key Characteristics:

- **FIFO Ordering:** Elements are processed in the order they were inserted.
- **No Random Access:** Unlike lists, elements cannot be accessed directly by index.
- **Implementations:** Common implementations include LinkedList, PriorityQueue, ArrayDeque, and ArrayBlockingQueue.

Code Link: JavaSamples/Scenario48_Stack_Using_Queues.java at main · CodingLyf-Fullstack/JavaSamples

Code Explanation:

q1 (Main Queue): This queue always stores the elements in the order required for stack operations (LIFO). The front of q1 will always contain the element that should be popped next.

q2 (Helper Queue): This queue is used temporarily during the push operation to maintain the correct order in q1.

push(int x):

The new element x is first added to q2.

All elements from q1 are then moved to q2. This ensures that the new element x is now at the front of q2, followed by all previous elements in their original order.
Finally, the references of q1 and q2 are swapped. This effectively makes q2 (which now holds the correctly ordered elements) the new q1, and the old q1 becomes the new q2 (which is now empty).

pop(): Since q1 always has the most recently added element at its front, a simple poll() operation on q1 removes and returns the top element.

peek(): Similarly, peek() on q1 returns the top element without removing it.

isEmpty() and size(): These operations directly leverage the corresponding methods of q1.

Code Steps:

Initial state: q1: [ ] q2: [ ]

Step 1: push(A):

- q2.offer(A); // q2: [A]
- while (!q1.isEmpty()) { ... } // skipped, q1 is empty
- swap q1 and q2
- After swap: q1: [A] q2: [ ]

Step 2: push(B):

- q2.offer(B); // q2: [B]
- while (!q1.isEmpty()) {
  q2.offer(q1.poll()); // move A from q1 -> q2
- }
- Now q2: [B, A], q1: [ ]
- swap q1 and q2
- After swap: q1: [B, A] q2: [ ]

Final view: Top of stack -> B A

221 Java Scenario 49: Implement Queue using Array

A simple circular array-based queue.

In a **circular array queue**, the idea is that the array behaves like a circle rather than a straight line. When we reach the end of the array, we wrap around to the beginning instead of stopping. This allows us to **reuse empty spaces** at the front of the array once elements are dequeued.

How it works in code:

rear = (rear + 1) % capacity;

1. rear + 1 moves the rear pointer to the next position in the array.
2. % capacity ensures that if rear reaches the last index (capacity - 1), it comes back to 0.

Code link: JavaSamples/Scenario49_ArrayQueue.java at main · CodingLyf-Fullstack/JavaSamples

Explanation is there in the code itself.

222 Java Scenario 50: Implement Custom HashMap

Code Link: JavaSamples/Scenario50_CustomHashMap.java at main · CodingLyf-Fullstack/JavaSamples

Explanation there itself in the code: Check the logs to understand it better

Streams – Programming and Scenarios

223 Streams 1: Remove duplicates without distinct()

Explanation:

1. filter(seen::add) means: for every number, try adding it into the HashSet seen.
2. HashSet.add(x) returns **true** the first time a number is inserted, and **false** if that number was already in the set.
3. Because filter keeps only the elements where the condition is **true**, the stream passes the first time each number appears and drops later duplicates.

224 Streams 2: Get duplicates

```
                                .collect(Collectors.toList())
        ; System.out.println("Unique Numbers: " +
        uniqueNumbers);

        }
```

Explanation:

- filter(i -> !seen.add(i)) means: for every number, try adding it into the HashSet seen.
- HashSet.add(x) returns **true** the first time a number is inserted, and **false** if that number was already in the set.
- Because we use !seen.add(i), the filter keeps only the elements where add() returned **false**.

That means only duplicates pass through the streams, and first occurrences are dropped.

225 Streams 3: Sort in descending order

Explanation:

- distinct() removes duplicates, leaving only unique numbers.
- sorted(Comparator.reverseOrder()) then sorts those unique numbers in **descending order** (largest -> smallest).
- By default, sorted() uses **natural order** (ascending). When we pass Comparator.reverseOrder(), it flips that natural order.
- Behind the scenes, the stream collects elements, then applies the comparator to arrange them before building the final list.

For the input [11, 11, 1, 3, 5, 6, 5]:

After distinct() -> [11, 1, 3, 5, 6]

After sorted(reverseOrder()) -> [11, 6, 5, 3, 1]

226 Streams 4: Practice Questions on Sort

1. Given a list of strings, sort them according to increasing order of their length?

List<String> listOfStrings = Arrays.asList("Java", "Python", "C#", "HTML", "Kotlin", "C++", "COBOL", "C");

Use: .sorted(Comparator.comparing(String::length))

2. How do you sort the given list of decimals in reverse order?

List<Double> decimalList = Arrays.asList(12.45, 23.58, 17.13, 42.89, 33.78, 71.85, 56.98, 21.12);

227 Streams 5: Find Max Number in a list

```
public class SteamsSample {
            public static void main(String[] args) {
                        List<Integer> numbers = Arrays.asList(1, 2, 4, 41, 4);
                        int maxNumber = numbers.stream()
                                                .max(Comparator.naturalOrder())
                                    .orElse(0);
            System.out.println("Max Number: " + maxNumber);
            }
}
```

Explanation:

1. numbers.stream() : creates a stream from the list [1, 2, 4, 41, 4].
2. .max(Comparator.naturalOrder()) : finds the maximum element by comparing numbers in their **natural order** (ascending).
3. .orElse(0) : in case the stream is empty, it safely returns 0 instead of throwing an

exception. Another Variant:

Given a list of strings, find the longest string using Java streams**.**

List<String> strings = Arrays.asList("apple", "banana", "orange","grape",

"kiwi"); Use: .max((o1, o2) -> o1.length() - o2.length())

228 Streams 6: Check all numbers even or not

```
public class SteamsSample {
            public static void main(String[] args) {
                        List<Integer> numbers = Arrays.asList(2, 4, 6, 8, 10);
                        boolean allEven = numbers
                                                .stream()
                                    .allMatch(i -> i % 2 ==
            0); System.out.println("Result: " + allEven);
            }
}
```

Explanation:

1. allMatch(i -> i % 2 == 0) -> checks if **every element** in the stream satisfies the condition (even in this case).
2. Since all numbers are even, it returns true.

Other flavours

1. **anyMatch**

   Returns true if **at least one element** satisfies the condition.

   boolean hasEven = numbers.stream().anyMatch(i -> i % 2 == 0);

   // true, because even numbers exist

2. **noneMatch**

   Returns true if **no elements** satisfy the condition.

   boolean noOdd = numbers.stream().noneMatch(i -> i % 2 != 0);

   // true, because there are no odd numbers

Similar Question:

1. Check if Any String Starts with 'A'

List<String> nameList = Arrays.*asList*("Banana", "Apple", "Cat", "Andrew"); Use - anyMatch

229 Streams 7: Numbers starting with 1

```
public class SteamsSample {
            public static void main(String[] args) {
                        List<Integer> numbers = Arrays.asList(10, 12, 20, null, 19,
            30); List<Integer> startWith1 = numbers
                                                .stream()
                                                .filter(i ->
                              String.valueOf(i).startsWith("1"))
                                          .toList();
            System.err.println(startWith1);
            }
```

Explanation:

1. numbers.stream() -> creates a stream of [10, 12, 20, null, 19,

30]. 2 .filter(i -> String.valueOf(i).startsWith("1"))

- Converts each element into a string.
- Keeps only those that start with "1".
- Caution: when i is null, String.valueOf(null) returns the literal string "null", so it won't throw an exception (but it also won't match).

3. toList() -> collects results into a list.

230 Streams 8: Find Palindrome Strings

```
public class SteamsSample {
            public static void main(String[] args) {
                        List<String> palindromeNames = Arrays.asList("Telugu",
            "Tamil",
"Malayalam");
                        List<String> findPalindromeStrings = palindromeNames
                                                .stream()
                                                .filter(s -> {
                                                                        return
                                            s.toLowerCase()

                                                contentEquals(new
StringBuilder(s.toLowerCase()).reverse());
                        }).toList();
```

Explanation:

1.Arrays.asList(...) Creates a list: ["Telugu", "Tamil", "Malayalam"]

2.stream() Turns that list into a stream , basically, a way to process each word one by one

3..filter(...): Keeps only the words that satisfy a condition , in this case, words that read the same backward and forward (palindromes).

Inside the filter:

- s.toLowerCase() makes the word lowercase (to ignore case).
- new StringBuilder(s.toLowerCase()).reverse() reverses the word
- contentEquals(...) checks if the original word equals its reverse.
- .toList() Collects the remaining (filtered) words into a new

list. System.out.println(findPalindromeStrings)

sPrints the final list of palindrome words.

231 Streams 9: Find the longest word in a list

```
List<String> words = Arrays.asList("cat", "elephant", "dog", "giraffe", "zebra");
Optional<String> longestWord = words.stream().max(Comparator.comparingInt(String::length));

longestWord.ifPresent(word -> System.out.println("The longest word is: " + word));
```

232 Streams 10: List of the questions on filter (For Practice)

```
List<Integer> list = Arrays.asList(10,15,8,49,25,98,32);
```

1. Find out all the even numbers that exist in the list using Stream functions Hint: Use .filter

2. Find out the **FIRST** even numbers that exist in the list using Stream

```
public class SteamsSample {
		public static void main(String[] args) {
				int[] a = new int[] { 4, 2, 7, 1 };
				int[] b = new int[] { 8, 3, 9, 5 };
				int[] c = Stream.concat(
                                                Arrays.stream(a).boxed(),
                                 Arrays.stream(b).boxed())
                                                .sorted()
                                                .mapToInt(i -> i)
.toArray(); System.out.println(Arrays.toString(c));
		}
}
```

functions Hint: Use .filter and .findFirst

3. Java 8 program to perform cube on list elements and filter numbers greater than 50. Hint: Use .map and .filter

233 Streams 11: Merge two unsorted arrays into single sorted array

Explanation:

- Stream.concat(...) -> merges two streams (a and b) into one sequence.
- .boxed() -> converts int primitives into Integer objects so they can be handled in a regular Stream.
- .mapToInt(i -> i) -> converts back from Stream<Integer> to IntStream for the final array.

Variants can be asked:

- Distinct merge: First remove duplicates , hint: .distinct().sorted().toArray()
- Reverse order: .boxed().sorted(Comparator.reverseOrder()).mapToInt(i -> i).toArray()
- Find max after merge: .max().orElse(-1)

234 Streams 11: Get top 3 elements from the list

```
public class SteamsSample {
		public static void main(String[] args) {
				List<Integer> listOfIntegers = Arrays.asList(71, 18, 42, 21,
		67, 32, 95,
14, 56, 87);

				List<Integer> topThree = listOfIntegers
                                                .stream()


                                                .limit(3)
                                        .toList
		(); System.out.println(topThree);
		}
}
```

Explanation:

sorted(Comparator.reverseOrder()) : arranges the numbers in descending order (largest to smallest).

limit(3) : takes only the first 3 elements from that sorted stream (the top three numbers).

235 Streams 12: Get 3rd highest element from the list

```
public class SteamsSample {
		public static void main(String[] args) {
				List<Integer> listOfIntegers = Arrays.asList(71, 18, 42, 21,
		67, 32, 95,
14, 56, 87);

				Integer thirdHighest = listOfIntegers
							.stream()
							.sorted(Comparator.reverseOrder())
							.skip(2)
					.findFirst().orElse(-
		1); System.out.println(thirdHighest);
		}
```

Explanation:

skip(2): ignores the first 2 elements in the sorted stream (so it moves past the highest and second highest).

findFirst(): picks the very next element after skipping, which is the third highest here.

orElse(-1): if no element is found (e.g., empty list), it safely returns -1 instead of failing.

236 Streams 13: Check if two strings are anagrams or not

```
public class SteamsSample {
		public static void
		main(String[] args) { String s1 =
		"RaceCar";
				String s2 = "CarRace";

				s1 = Stream.of(s1.split(""))
							.map(String::toUpperCase)
							.sorted()
							.collect(Collectors.joining());

				s2 = Stream.of(s2.split(""))
							.map(String::toUpperCase)
							.sorted()
							.collect(Collectors.joining());

		if (s1.equals(s2)) {
			System.out.println("Two strings are anagrams");



					System.out.println("Two strings are not
			anagrams");
				}
		}
```

An anagram is when two words or phrases use the same letters with the same frequency, just in a different order.
Example: "listen" and "silent" are anagrams.

Explanation:

- s1.split("") -> map(String::toUpperCase) -> sorted() breaks the string into characters, makes them uppercase, and sorts them alphabetically.
- .collect(Collectors.joining()) puts the sorted characters back into a single string.
- If both sorted strings are equal, then they're anagrams (same letters, same count, just rearranged).

237 Streams 14: Sum of all digits in a number

```
public class SteamsSample {
        public static void main(String[] args) {
                int i = 15623;

         Integer sumOfDigits = Arrays.stream(String.valueOf(i).split(""))
                                        .collect(Collectors

                                                summingInt(Integer::parseInt));

                System.out.println(sumOfDigits);
        }
```

238 Streams 15: Common elements between two arrays

```
public class SteamsSample {
        public static void main(String[] args) {
                List<Integer> list1 = Arrays.asList(71, 21, 34, 89, 56, 28);
                List<Integer> list2 = Arrays.asList(12, 56, 17, 21, 94, 34);
        list1.stream().filter(list2::contains).forEach(System.out::println);
        }
}
```

239 Streams 16: Reverse each word of a string

```
public class SteamsSample {

              public static void main(String[]
        args) { String str = "Java Concept Of
        The Day";

                String reversed = Arrays.stream(str.split(" "))
                                        .map(s -> new
                StringBuilder(s).reverse())
                                        .collect(Collectors.joining(" "));

                System.out.println(reversed);
```

Explanation:

.map(s -> new StringBuilder(s).reverse()): reverses each word

.collect(Collectors.joining(" ")): joins them back with spaces.

240 Streams 17: Count the number of occurrences of a given String.

```
public class SteamsSample {
            public static void main(String[] args) {
                        List<String> strings = Arrays.asList("java scala ruby", "java
            react spring
java");
                        String word = "java";
                        long count = strings.stream()
                                                                  .flatMap(s ->
                              Arrays.stream(s.split(" ")))
                                                                  .peek(System.out::pri
                              ntln)
                                                                  .filter(w ->
                              w.equals(word))
```

Explanation:

flatMap(s -> Arrays.stream(s.split(" ")))

- Each element in strings is a sentence like "java scala ruby".
- s.split(" ") breaks it into words : ["java", "scala", "ruby"].
- Arrays.stream(...) turns that array into a stream of words.
- **flatMap** flattens all those small word-streams into one continuous stream of words across all sentences.
    - So instead of [["java","scala","ruby"], ["java","react","spring","java"]]
    - We get a single stream: ["java","scala","ruby","java","react","spring","java"].

241 Streams 18: Convert the list of sentences into unique words.

```
public class SteamsSample {
            public static void main(String[] args) {


                        List<String> sentences = List.of("java is cool", "cool code in
            java"); Set<String> words = sentences.stream()
                                    .flatMap(s -> Arrays.stream(s.split(" ")))
                                 .collect(Collect
            ors.toSet()); System.out.println(
            words );
            }
```

242 Streams 19: More Questions on flatMap.

1. You have List<List<Integer>>. How do you create a single List<Integer>? Input: List<List<Integer>> nums = List.of(List.of(1, 2), List.of(3, 4));
   Output: [1,2,3,4]
2. How do you get distinct characters from a list of words? Input: List<String> words = List.of("java", "scala"); Output: [j,a,v,s,c,l]

243 Streams 20: Find all the Longest words in a list

```
public class SteamsSample {
            public static void main(String[] args) {
            List<String> words = Arrays.asList("apple", "banana", "orange",
"pineapple", "blueberry");

                        int maxLength = words.stream()
                                                .max((o1, o2) -> o1.length() -
                        o2.length())
                                                .get()
                                                .length();

                        List<String> list = words
                                                .stream()
                                                .filter(s -> s.length() ==
                        maxLength)
                                    .toList();

            System.out.println("Longest Strings " + list);
```

Output:

Longest Strings [pineapple, blueberry]

244 Streams 21: Questions on reduce

1. Sum of numbers List<Integer> numbers = Arrays.asList(5, 10, 15, 20);

2. Find Maximum using reduce, List<Integer> numbers = Arrays.asList(7, 2, 10, 4);
3. List<String> words = Arrays.asList("cat", "elephant", "dog", "tiger");

245 Streams 22: Find the longest common prefix using Java streams:

```
public class SteamsSample {
            public static void main(String[] args) {

                        List<String> strings = Arrays.asList("flower", "flow",
            "flight"); String longestCommonPrefix = strings.stream().reduce((s1,
            s2) -> {
                                    int length = Math.min(s1.length(), s2.length());
                                    int i = 0;
                                          while (i < length &&
                        s1.charAt(i) == s2.charAt(i)) { i++;
                                    }
                                    return s1.substring(0, i);
                        }).orElse("");
                        System.out.println("Longest common prefix: " +
            longestCommonPrefix);
```

.reduce((s1, s2) -> {...})

- reduce takes two elements at a time (s1 and s2), applies a function, and returns a result.
- That result will then be compared with the next element in the stream.
- First compares "flower" and "flow" : gives "flow".
- Then compares "flow" and "flight" : gives "fl".

  Inside the lambda:

Find the minimum length of the two

strings. Start comparing characters one by
one.
Stop when characters differ or when we reach the end.
s1.substring(0, i) returns the prefix that matched.
Output: Longest common prefix: fl

246 Streams 23: Max Product in a given array

```
public class SteamsSample {
            public static void main(String[] args) {

                        int[] array = { 1, 4, 9, 6, 2, 7, 8 };
                        Integer maxProduct = IntStream.range(0, array.length)
                        .map(i -> IntStream.range(i + 1, array.length)
                                    .map(j -> array[i] * array[j])
                                                                                .max(

                                    )
                                                                                .orEl

                                    se(0))



                        System.err.println(maxProduct);
            }
}
```

The problem is about finding the maximum product that can be formed by multiplying any two different numbers in the array.

- Think of it like trying every pair, calculating their product, and then just picking the largest one.
- For example, in {1, 4, 9, 6, 2, 7, 8}, the biggest product is from 9 × 8 = 72.

Step by step

1. IntStream.range(0, array.length)
    - This creates a stream of all indices from 0 to array.length - 1.
    - Each index i represents the first number in a pair.
2. IntStream.range(i + 1, array.length)
    - For each i, this creates another stream of indices starting from i+1 to the end.
    - Why i+1? To avoid repeating pairs and avoid multiplying a number with itself.
    - So if i = 2 (say number 9), it pairs 9 with every number after it.
3. .map(j -> array[i] * array[j])
    - For each valid pair (i, j), it multiplies the two numbers.
    - Example: if i=2 (9) and j=6 (8), result is 72.
4. Inner .max().orElse(0)
    - Among all products generated for a fixed i, this picks the largest one.
    - Example: if i=2 (9), the products are 9×6=54, 9×2=18, 9×7=63, 9×8=72. The max here is 72.
5. Outer .max().orElse(0)
    - Now it compares the “best product” for each i and keeps the overall maximum.
    - That gives us the largest product across the entire array.

247 Streams 23: First non-repeating character in a String

```java
public class SteamsSample {
        public static void main(String[] args) {

                String input = "aabbcdeffg";
                String firstNonRepeating = Arrays.stream(input.split(""))
                                .collect(Collectors
                                                .groupingBy(
                                                                Function.identity(),
                                                                LinkedHashMap::new,
                                                                Collectors.counting()))
                                .entrySet()
                                .stream()
                                .filter(entry -> entry.getValue() == 1)
                                .map(Map.Entry::getKey)
                                .findFirst()
                                .orElse(null);
                System.out.println("First Non-Repeating Character: " + firstNonRepeating);
        }
}
```

248 Streams 24: Most Repeated Number in a given list.

```java
public class SteamsSample {
                public static void main(String[] args) {
                        List<Integer> numbers = Arrays.asList(10, 20, 5, 8,
                30, 25, 30); Integer mostRepeated = numbers.stream()
                                                                .collect(Collectors
                                                                                                .grouping
                                                By(Function.identity(),
                                                                ollectors.counting()))
                                                                .entrySet()
                                                                .stream()
                                                                .max(Map.Entry.comparingByValue())
                                                                .map(Map.Entry::getKey)
                                                                .orElse(null);
                                System.out.println("Most Repeated Number: " + mostRepeated);
                }
}
```

249 Streams 25: Frequency of words or count of each word in a String

```
public class SteamsSample {
            public static void main(String[] args) {
                        String sentence = "Java is fun and java is
                    powerful"; Map<String, Long> frequencySplit = Arrays
                                                .stream(sentence.toLowerCase().split
                        ("\\s"))
                                                .collect(Collectors
                                                                        .grouping
                                            By(Function.identity(), Collectors.counting()));
                        System.err.println("Result " + frequencySplit);
            }
```

Explanation:

- sentence.toLowerCase() : converts everything to lowercase so "Java" and "java" are treated the same.
  Result: "java is fun and java is powerful"
- .split("\\s") : splits the string by whitespace into words. Result: ["java", "is", "fun", "and", "java", "is", "powerful"]
- Arrays.stream(...) -> turns that array into a stream.
- .collect(Collectors.groupingBy(Function.identity(),

  Collectors.counting())) groupingBy(Function.identity()) : groups by

  the word itself.

  Collectors.counting() : counts how many times each word appears.

  Result {java=2, is=2, fun=1, and=1, powerful=1}

250 Streams 26-35: Scenario based questions on Student Data

We have Student records with Firstname, lastname, city, grade, age and department

Questions:

1. Find students from Hyderabad with a grade greater than 8.0
2. Find the student with the highest grade
3. Count the number of students in each department
4. Find the average grade per department
5. List students sorted by age and then by grade
6. Create a comma-separated list of student names
7. Check if all students are above 18
8. Find the department with the most students
9. Divide students into those who have grades above 8.0 and below
10. Find the student with the longest full name

Code link: StreamsScenarios/StreamsOnStudentData.java at main · CodingLyf-Fullstack/StreamsScenarios

251 Streams 36-45: Scenario based questions on Employee Data

We have Employee data with Name, Department, Age, Gender

```
EmployeeDto employee1 = new EmployeeDto("SRK","ECE",31,"Male");

 List<Student> students = Arrays.asList(
         new Student("Rahul", "Sharma", "Hyderabad", 8.38, 19, "Civil"),
         new Student("Amit", "Verma", "Delhi", 8.4, 21, "IT"),
         new Student("Suresh", "Reddy", "Chennai", 7.5, 20, "Civil"),
EmployeeDto employee2 = new EmployeeDto("Salman","CS",44,"Male");
EmployeeDto employee3 = new EmployeeDto("Katrina","ECE",21,"Female");
EmployeeDto employee4 = new EmployeeDto("Kareena","CS",34,"Female");
EmployeeDto employee5 = new EmployeeDto("Hrithik","EEE",30,"Male");
EmployeeDto employee6 = new EmployeeDto("Aish","EEE",25,"Female");
```

Questions:

1.Find the names of all Employees in the CS department, sorted by age in descending order

2. Group Employees by department and count how many Employees are in each department

3.Find the youngest female Employee.

4. Create a map of department -> list of Employee names.

5. Find the average age of Employees in each department.

6. Get a list of unique departments Employees belong to

7. Partition Employees into male and female groups, then list their names.

8. Group employees by department, then within each department find the oldest employee

9. Build a map of gender with average age of employees sorted by average age descending

10. For each department, find the youngest employee, but instead of returning the employee object, return only their name in uppercase.

11. Return a map where keys will be first letter of the name and value will the set of names starting with that letter, no solution provided, try on your own.

Code link: StreamsScenarios/StreamsOnEmployeeData.java at main · CodingLyf-Fullstack/StreamsScenarios

252 Stream 46-50: Scenario based questions on City Data

We have City data with name, population.

```
List<City> cities = Arrays.asList(
                        new City("Delhi", 12000),
                        new City("Mumbai", 800000),
                        new City("Bangalore", 450000),
                        new City("Hyderabad", 1200000),
                        new City("Chennai", 60000)
                );
```

Questions:

1. City with the second highest population

2. Group by first character of name, then max population in each group

3. Average population of top 3 most populated cities.

4. Map of population range -> city names.

5. Using reduce: String of cities ordered by population.

Expected output: Hyderabad(1200000) > Mumbai(800000) > Bangalore(450000) > Chennai(60000) > Delhi(12000)

Code link: StreamsScenarios/StreamsOnCityData.java at main · CodingLyf-Fullstack/StreamsScenarios

Coding Questions for Practice

307 Array Coding Problems

1. Reverse the array

Given **int**[] arr = {2,5,6,4,1,3}; reverse: [3, 1, 4, 6, 5, 2]

2. Find Second Smallest and Second Largest element in an array

Given int[] arr = {2, 5, 6, 4, 1, 3, 10, 8, 7}; Second largest is 8 and second smallest is 2

Code link: https://github.com/CodingLyf-Fullstack/Arrays/edit/main/SecondSmallestAndLargest.java

3. Find the repeating elements / duplicates from an array

Given **int**[] arr = { 2, 5, 6, 2, 1, 3, 1, 8, 3 }; In this array {2,1,3} are duplicates Write a function that should return {2,1,3}

Code link: https://github.com/CodingLyf-Fullstack/Arrays/edit/main/FindDuplicatesInArray.java

4. Given two arrays, merge them into a single sorted array. Input: **int** arr1[] = {2,3,5,4,1}; **int** arr2[] = {8,7,6,10,9}; Excepted output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Code link: Arrays/MergeArraysIntoSinlgeSortedArray.java at main · CodingLyf-Fullstack/Arrays

5. Count of number of occurrences of each number in an array.

Given **int** arr[] = {9, 8, 4, 9, 5, 4, 9, 1}. Here 9 repeated 3 times and 4 repeated 2 times etc. Expected output: {1=1, 4=2, 5=1, 8=1, 9=3}

Code Link: Arrays/CountNumberofOccurences.java at main · CodingLyf-Fullstack/Arrays

6. Find all non-repeating or unique elements in an array.

Given int[] arr = {4, 5, 2, 4,, 2, 1}; 5 and 1 and not repeated. Excpected output : {5,1}.

7. Check if Array is a subset of another array or not

Input1:

int[] arr1 = {1, 2, 3, 4, 5};
int[] arr2 = {2, 4, 5};
Here 2,4,5 are subset of arr1. Function should return true

Input1:

int[] arr1 = {1, 2, 3, 4, 5};

int[] arr2 = {2, 6};

Here 2,6 are subset of arr1. Function should return

Code Link: Editing Arrays/FindArrayisSubset.java at main · CodingLyf-Fullstack/Arrays

8. Find majority element in an array.

A majority element in an array is an element that appears more than n/2 times, where n is the size of the array.

Given **int**[] nums = { 2, 2, 1, 1, 1, 2, 2 }; where 2 is majority element

Code Link: Arrays/FindMajorityElement.java at main · CodingLyf-Fullstack/Arrays

9. Move all zeroes to the end of the array

Given **int**[] arr = { 0, 2, 3, 0, 0, 1, 4, 0, 0 }; Expected output: {2,

3, 1, 4, 0, 0, 0, 0, 0} Code link: Arrays/MoveallZerostoEnd.java

at main · CodingLyf-Fullstack/Arrays

We can also use two pointer technique to solve this problem. Refer question no. 21

10. Find Leaders in an Array

A leader in an array is an element that is greater than all the elements to its right.

- The rightmost element is always a leader because there are no elements to its right.
- You need to check each element and see if it is bigger than all the elements that come after it in the array.

Input [16, 17, 4, 3, 5, 2] and the leaders are [2,5,17]

Code link: https://github.com/CodingLyf-Fullstack/Arrays/edit/main/FindLeadersInArray.java

11. Merge Two Sorted Arrays Using streams

Given **int**[] arr1 = { 2, 4, 6, 7, 9 }; **int**[] arr2 = {

5, 8, 10, 12 }; Hint: Use Streams.concat()

then .sorted()

12. Find Subarray with Given Sum

Given an array of integers and a target sum: int arr[] = {1, 2, 3, 7, 5}; int target = 12.

The task is to find a continuous part of the array (subarray) whose elements add up exactly to the given sum.

Important: Subarray means the elements must be contiguous (next to each other in the array). It is not just picking any random elements.

Key point: We need use Sliding window with variable window size.

Reference: Sliding Window in 7 minutes | LeetCode Pattern

Code link: Editing Arrays/FindSubArrayWithGivenSum.java at main · CodingLyf-Fullstack/Arrays

13. Find number of subarrays with Sum.

In this problem, we need to find number of sub array, in the above only one sub array will be there. But we can have multiple sub arrays .

Given an array of integers and a target sum: int arr[] =

{1,2,2,3,1}; int target = 5. We can see two sub arrays [1,2,2],

[2,3]. So the output should be 2.

Code link: Editing Arrays/FindNumberofSubArrayswithGivenSum.java at main · CodingLyf- Fullstack/Arrays

14. Find Maximum Subarray Product

Given an integer array, find the contiguous subarray (containing at least one number) which has the largest product.

Input: **int**[] arr = { 2, 3, -2, 4 } , Expected output is 6 which maximum

product of a sub array. Code Link:

Similar question: Find Maximum Subarray Sum (Kadane's algorithm)

Code link: Arrays/MaxSubarrayProduct.java at main · CodingLyf-Fullstack/Arrays

15. Find Maximum sub array whose length is K

Key point: We need use Sliding window with fixed window size(K).

Reference: Sliding Window in 7 minutes | LeetCode Pattern

Given an array **int**[] arr = {2, 1, 5, 1, 3, 2}; and **int** k = 3; Means. Given a fixed sized. We need to get maximum sum of a sub array with length 3.

Example:

List all subarrays of length 3

- [2, 1, 5] > sum = 8
- [1, 5, 1] > sum = 7
- [5, 1, 3] > sum = 9 (Maximum)
- [1, 3, 2] > sum = 6

Code Link: Arrays/MaxSubArrayWhoseLengthisK.java at main · CodingLyf-Fullstack/Arrays

16. Sort the Map by values.

Suppose you have a HashMap<String, Integer> that stores names and their scores, and you need to sort it by values.

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Mahesh ", 45);
scores.put("NTR", 30);
scores.put("Chiru", 75);
scores.put("Balu", 60);
```

Code link: Arrays/SortMapByValues.java at main ·

CodingLyf-Fullstack/Arrays Try to solve the same problem

Java 8

17. Rotate the Array by K steps.

Given an array int arr[] = { 1, 2, 3, 4, 5, 6, 7 } and an integer k = 3. Rotate the array to the right by k steps.

That means, each element moves k steps to the right, and if it goes past the end, it comes back around to the start.

Example:

Input int arr = [1, 2, 3, 4, 5, 6, 7] and k = 3

Ouput [5, 6, 7, 1, 2, 3, 4]

Code link: Arrays/RotateArraybyKSteps.java at main · CodingLyf-Fullstack/Arrays

18. Move all zeros to the left.

In question 9 we have seen moving all zeros to the end of the array, now it is move to the left means start of an array

Given **int**[] arr = { 0, 2, 3, 0, 0, 1, 4, 0, 0 }; Expected output: {0,

0, 0, 0, 0, 2, 3, 1, 4} Code link: Arrays/MoveAllZerosToLeft.java

at main · CodingLyf-Fullstack/Arrays

Using Two pointer: Arrays/MoveZerosToLeftUsingTwoPointer.java at main · CodingLyf-Fullstack/Arrays

19. Top K frequent elements

Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Code link: Arrays/TopKFrequentElements.java at main · CodingLyf-Fullstack/Arrays

20. Check if a Pair with Given Sum Exists in Array (Two Sum Problem)

You are given an array of integers and a target sum. You need to check if there exists any pair of numbers in the array whose sum is equal to the target.

Input: arr = [8, 4, 1, 6], target = 10

Output: true because there is pair 4 and 6 whose sum is 10

Code Link: Arrays/CheckPairWithGivenSumExistsInArray.java at main · CodingLyf-Fullstack/Arrays Additional Question: Same question can be asked to the pair instead of true/false.

Hint: **return new int**[]{numToIndex.get(complement), i};

Note: we can also solve with two pointer technique, but we need to sort arrray (Refer Question 21)

21. Triplet with Zero Sum – 3 Sum Problem

Given an **unsorted array**, check if there exists a triplet (a, b, c) such that a + b + c = 0.

Input: arr = [-1, 0, 1, 2, -1, -4]; there are two triplets with sum 0 [(-1, -1, 2), (-1, 0, 1)]. So our function should return true.

Approach:

Here we use Two pointer technique. Reference to know more about two pointer: Two Pointers in 7 minutes | LeetCode Pattern

Code link: Arrays/TripletWithSumZero.java at main · CodingLyf-Fullstack/Arrays

22. Remove Duplicates from Sorted Array – Two pointer

You are given a sorted array. You need to remove duplicates *in-place* so that each unique element appears only once.

Input: arr = [1, 1, 2] output [1,2]

Code link: Arrays/RemoveDuplicates.java at main · CodingLyf-Fullstack/Arrays

23: Longest Consecutive Sequence

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

Input: nums = [100,4,200,1,3,2]

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4].

Therefore, its length is 4. Code link: Arrays/LongestConsecutiveSequence.java at main · CodingLyf-Fullstack/Arrays

24. Create a function that takes input as 7 and returns 11. if we give input 11, it should return 7. Don't use if-else conditions.

```
public class Main {

            public static void main(String[] args) {

                        System.out.println(t
oggle(7));
System.out.println(toggle(11));
}
public int              tog
          gle(int n) {
    }
}
```

25.Print all subset of a given array – Recursion

You are given an array, and you want to **print all possible**

- Subsets can be of any size (0 to n).
- Example: for [1,2] and subsets are: [], [1], [2], [1,2].

Code Link Arrays/SubsetsRecursive.java at main · CodingLyf-Fullstack/Arrays

Additional problems for practice

1. Sum of natural numbers using recursion

2. Factorial of a number using recursion
3. Find out the Sum of Digits of a Number.
4. Find Common elements between two arrays int[] arr1 = {1, 2, 4, 5, 6}; int[] arr2 = {2, 3, 5, 7};
5. Add and substract two matrices

308 String Coding Problems

1. Check two strings anagrams or not

An anagram means both strings contain the same characters with the same frequency, but possibly in a different order.

Example:

"silent" and "listen"

are anagrams.

"hello" and "world"

are not.

Code link: Strings/CheckAnagrams.java at main · CodingLyf-Fullstack/Strings

2. Check if String has all unique characters Input: "silent"

- Characters: s, i, l, e, n, t
- All are different return **true**

Input: "hello"

- Characters: h, e, l, l, o
- l repeats return **false**

Code Link: Strings/CheckAllUniqueChars.java at main · CodingLyf-Fullstack/Strings

3. Find all Unique Characters

Given a String you need to fetch all the all-unique characters means characters that appear only once. Input: “success”, Unique characters are u, e.

Code Link: Strings/FindAllUniqueCharacters.java at main · CodingLyf-Fullstack/Strings

4. Find the First Non-Repeating Character

Input: "swiss”, First non-repeating character is w.

Code Link: Strings/FirstNonRepeatingCharacter.java at main · CodingLyf-Fullstack/Strings

5. String to Camel Case

Input: “hello from ”; output: “Hello From ”.

Code Link: Strings/StringtoCamelCase.java at main · CodingLyf-Fullstack/Strings

6. Check if one String is rotation of another String in java

You need to check if one string is a **rotation**

of another. Example:

- s1 = "abcd"
- s2 = "cdab"

Here s2 is a rotation of s1 because if you rotate "abcd" left by 2, you get "cdab". Code Link: Strings/CheckStringRotation.java at main · CodingLyf-Fullstack/Strings

7. Count Number of Words in String without using split(). Input: “This is ”;

Output: 4

Code Link: Strings/CountNumberOfWords.java at main · CodingLyf-Fullstack/Strings

8. Remove duplicates from the String.

Given a String remove all the duplicate characters from the given String.**Example 1: Input:** s = "bcabc"

Output: “bca"

Code Link: Strings/RemoveDuplicates.java at main · CodingLyf-Fullstack/Strings

9. Find the Max repeating character

Input: str = “apple”

Output: p

Explanation: The character 'p' have the maximum occurrence i.e 2.

Code Link: Strings/FindMaxRepeatingCharacter.java at main · CodingLyf-Fullstack/Strings

10. Create a new String by removing all occurrences of given character Input: "banana", remove 'a' so output: bnn.

Code Link: Strings/RemoveChars.java at main · CodingLyf-Fullstack/Strings

11. Find the largest word in a given string

Input: “Java makes coding enjoyable and challenging”; Output: “challenging” Code Link: Strings/FindTheLargestWord.java at main · CodingLyf-Fullstack/Strings

12: Remove characters from first string that are present in the second string You are given two strings:

- **First string**: the base string

- **Second string**: contains characters that need to be removed

from the first string First string = "computer"

Second string = "cat"

Remove ‘c’ and ‘t’ from first String

Code link: Strings/RemoveCharactersFromFirstString.java at main · CodingLyf-Fullstack/Strings

13. Implement a method to compare two Strings for equality

(don’t use .equals) Code Link: Strings/StringEqualityCheck.java

at main · CodingLyf-Fullstack/Strings

14. Isogram

An isogram is a word or phrase where no letter repeats. Every character

appears only once. Example 1:

Word: machine

- Letters: m, a, c, h, i, n, e
- None of them

repeat > isogram.

Example 2:

Word: programming

- Letters: p, r, o, g, r, a, m, m, i, n, g
- Here r, m, and g repeat > Not an isogram.

Code Link: Strings/Isogram.java at main · CodingLyf-Fullstack/Strings

15. Replace all instances of a Substring with another sub string.

– Sliding Window String input = "abc123abc456abc";

String target = "abc";

String replacement = "XYZ";

Then all the “abc” should be replaced with “XYZ”

Code Link: Strings/ReplaceAllInstances.java at main · CodingLyf-Fullstack/Strings

16. Merge Two Strings Alternatively

Input: String s1 = "Coding";

String s2 = "Lyf"; Output:

CLoydfing

Code Link: Strings/MergeTwoStringsAlternatively.java at main · CodingLyf-Fullstack/Strings

17. Check if a String is a subsequence of another string – Two pointer

A string s1 is a subsequence of string s2 if all characters of s1 appear in s2 **in the same order**, but not necessarily contiguously.

s1 = "abc", s2 = "axbycz" > subsequence, "abc"

is a subsequence. s1 = "abc", s2 = "acb" > Not

subsequence, order breaks.

18. Count the number of times one string occurs in another - Sliding Window

Given two strings, s1 (the pattern) and s2 (the text), count how many times s1 occurs contiguously in s2.

- Contiguously means the characters of s1 appear together in order, without any gaps.

Example 1:

s1 = "abc"; s2 =

"abcabcabc"; Output: 3

Explanation:

- "abc" appears starting at index
- Total occurrences = 3

Example 2:

s1 = "xyz"; s2 = "xyabcz";

Output: 0 Explanation:

- " xyz" is not there “xyabcz”.

Code Link: Strings/CheckSubsequence.java at main · CodingLyf-Fullstack/Strings

19. Find the longest substring without repeating the characters – Siding Window

Given a string s, find the length of the longest substring and substring without duplicate characters.

Input: s = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and

not a substring. Code Link: Strings/LongestSubString.java at main · CodingLyf-Fullstack/Strings

20. Find the longest common prefix

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an

empty string "". Example:

Input: strs =

["flower","flow","flight"]

Output: "fl"

Code Link: Strings/LongestCommenPrefix.java at main · CodingLyf-Fullstack/Strings

21. Reverse individual words

Given a string containing multiple words, reverse each word individually without changing their order in the sentence.

Inpu
t:
"Hell
o
Worl
d"
Outp
ut:
"olle
H
dlro
W"

Code Link: Strings/ReverseIndividualWords.java at main · CodingLyf-Fullstack/Strings

22. Reverse words in String

Given a string containing multiple words, reverse the order of the words, but keep the characters in each word intact.

- Input: "Hello World from Java"
- Output: "Java from World Hello"

Hint: Split the sting with space " ". Iterate the array and apply reverse logic.

23. Rotate String both left

and right by K steps. Rotate a

string left or right by k

positions.

- Left Rotation: Move the first k characters to the end.
- Right Rotation: Move the last k characters

to the beginning. Example:

Input String: "abcdef"

- Left rotation by 2: "cdefab"
    - "ab" moves from the start to the end.

- Right rotation by 2: "efabcd"
    - "ef" moves from the end to the start.

Code Link: Strings/RotateStringLeftAndRight.java at main · CodingLyf-Fullstack/Strings Note: You can also do with substring. Give a try.

24. String Compression

Given a string, compress it by replacing consecutive repeated characters with the character followed by its count.

- If the compressed string is not shorter than the original, you can

keep the original. Example 1:

Input:
"aaabbc"
Compression Steps:

- 'a' repeats 3 times, "a3"
- 'b' repeats 2 time , "b2"
- 'c'

repeats 1

time, "c"

Output:

"a3b2c"

Original:
"aaabbc":
length 6
Compressed:
"a3b2c":
length 5

Code Link: Strings/StringCompression.java at main · CodingLyf-Fullstack/Strings

25: Find all permutations of a String – Recursive

Given String “ABC”, Print all the permutations ABC, ACB, BAC,

BCA, CAB, CBA Code Link: Strings/FindAllPermutations.java at

main · CodingLyf-Fullstack/Strings Reference: L12. Print all

Permutations of a String/Array | Recursion | Approach - 1

Additional questions:

26. Write a program to sort characters in a string - Use any sorting technique like Bubble sort

27. Write a program to find a substring within a string. If found display its starting position - Sliding Window ()

Hint: Refer question 18. Write a function and instead of incrementing the count. return i and break the loop.

if(i == s1.length) >

index not found

else index found at i

28. Change case of each character in a string

29. String Palindrome

30. Reverse the String

Questions by Experience Level

Please Note: These questions are segregated based on fundamental knowledge but in real time its highly depends on company, role and interview process. So please don't stick to these it is just for an idea.

309 Junior Level Developer

Core Java Fundamentals

1. Explain about JDK, JRE, and JVM.
2. What are the core features of Java that make it object-oriented?
3. Can you list all primitive data types in Java and their sizes?
4. How does autoboxing/unboxing work? Can you give an example where it might cause a bug?
5. Explain access modifiers (private, protected, public, default) with simple code.
6. What is a package in Java, and why do we use it?
7. How would you explain the static keyword (variables, methods, blocks)?
8. What is a constructor in Java? What are its types?

OOP Concepts

9. What is inheritance? Can you explain with real-world examples?
10. How do you explain polymorphism to a non-technical person? Give both compile-time and runtime examples.
11. What's the difference between abstraction and encapsulation?
12. Method overloading vs overriding, when do you use which?
13. Difference **interface** vs **abstract class**?
14. What is a Marker Interface and example?
15. Difference between final, finally, and finalize()?

Strings

16. What's the difference between String, StringBuilder, and StringBuffer?
17. Why is String immutable?
18. How does the String Pool work internally? Why do we need it?

19. Difference between creating a string using new and using a literal?
20. == vs .equals() ?

Collections

21. What's the difference between Array and ArrayList?
22. ArrayList vs LinkedList , which one would you choose for frequent insertions?
23. Explain HashMap and its common operations.
24. Difference between HashMap and TreeMap.
25. What's the difference between HashSet and TreeSet?
26. why use List over Set, or vice versa?
27. What is an Iterator? Can you show how to use it?
28. What's the difference between break and continue?
29. Fail safe vs Fail fast?
30. Difference between Vector and ArrayList.

Exception Handling

31. What is an exception? Checked vs unchecked?
32. Explain try-catch-finally. Can you use try without catch?
33. What is the super class of Exception?
34. What is the difference between throw and throws?
35. Why would you create a custom exception?
36. What are the exceptions you have got in your project? How do you prevent them?
37. Difference between Error and Exception.

Multithreading & Concurrency

38. Difference between Thread and Runnable.
39. Lifecycle of a thread in Java.
40. Difference between wait() and sleep().
41. What is the role of synchronized keyword?

Streams

42. What is the Stream API in Java 8?
43. Can you explain map, filter, and forEach with a small example?
44. Why are streams preferred over loops in some cases?
45. Differences Intermediate and terminal operators.
46. Practice the coding questions on filter, map, groupigBy, max, sort
47. Difference between sequential and parallel streams.
48. How does reduce() work?
49. Explain Collectors.groupingBy().
50. How does lazy evaluation work in streams?
51. Difference between map() and peek().
52. How to remove duplicates using streams?
53. How to join a list of strings into one string with delimiter?
54. How do streams work with primitive types (IntStream, LongStream)?

310 Mid-Level Developer

1. Collections

1. When you override equals(), why must you override hashCode() as well?
2. How does HashMap work internally? Explain hashing and collisions.
3. What happens when two keys have the same hash?
4. What is load factor and resizing in HashMap?
5. Difference between HashMap, LinkedHashMap, and TreeMap.
6. Difference between HashMap and ConcurrentHashMap (thread safety).
7. What's the difference between ArrayList, LinkedList, Vector? Which one would you use for frequent random access?
8. Difference between fail-fast and fail-safe iterators (with example).
9. How do you prevent ConcurrentModificationException?
10. How does Arrays.asList() work?
11. What are WeakHashMap and its use cases?
12. How do you synchronize a collection?

Java 8+ Features in Practice

13. What is a Functional Interface? Can you give examples from Java libraries?
14. Difference between Lambda expressions and Anonymous classes.
15. Stream API ; explain intermediate vs terminal operations.
16. map() vs flatMap() , when do you use each?
17. How would you find the 2nd highest salary in a list of employees using Streams?
18. Optional, what problems does it solve? When should you NOT use it?
19. Stream pipelines, how do they work under the hood?
20. Parallel Streams, when should you use them, and when should you avoid them?
21. What is flatMap() used for in real scenarios?
22. Explain how stream pipeline works internally.

OOPs

23. What are Sealed Classes (Java 17)? Why are they introduced?
24. Shallow copy vs Deep copy , when do you need each?
25. How does cloning work (Cloneable)? Why is it tricky?
26. Stack vs Heap memory , what goes where?
27. What is Garbage Collection? Can you explain different types of GCs (briefly)?
28. How do you handle OutOfMemoryError in production?

Multithreading & Concurrency Basics

29. What is the role of synchronized keyword?
30. Thread-local variables, where do they help in real-world projects?
31. Explain volatile vs synchronized.
32. Difference between Callable and Runnable.
33. How does ExecutorService work?
34. Explain producer-consumer problem.
35. What are deadlocks and how to prevent them?
36. What is a thread pool?
37. How do you stop a thread safely?
38. Difference between Future and CompletableFuture.
39. How do you achieve inter-thread communication in Java?

311 Senior Developer 7+

1. JVM Internals & Performance

1. Explain JVM memory areas (Heap, Stack, Metaspace, Code Cache).
2. Different GC algorithms (Serial, Parallel, G1, ZGC). When would you choose one over the other?
3. What is a memory leak in Java despite having GC? How would you detect and fix it?
4. How would you design a thread-safe Singleton?
5. Explain Liskov Substitution Principle with an example.
6. What are anti-patterns you've seen in OOP design?
7. How do you handle large-scale refactoring in legacy OOP systems?
8. How does polymorphism help in designing extensible systems?
9. How do you design an immutable class for multi-threaded environment?
10. Explain Dependency Inversion Principle in real-world code.
11. How do you decide between inheritance and interfaces in large projects?
12. How would you apply design patterns (Factory, Strategy, Observer) in real-world apps?

Advanced Concurrency & Multithreading

13. What is the Executor Framework? Why use it instead of manually creating threads?
14. Explain Future, CompletableFuture, and use cases for async programming.
15. Difference between synchronized and ReentrantLock. When would you choose one?
16. volatile vs atomic variables, when to use?
17. Explain the Producer-Consumer problem and solve it with BlockingQueue.
18. What is ForkJoinPool? How is it different from normal thread pools?
19. What is the Java Memory Model (JMM)? How does it affect visibility of shared variables?
20. How do you make a singleton thread-safe in Java?

Java Cheat Sheet

These cheat sheets helpful for quick reference before interviews.

312 Java Cheat Sheet

1. Fundamentals

- **Variables & Data Types:** Primitives (int, double, boolean, char) store raw values; References
  (String, arrays, objects) store memory addresses. Default values differ for primitives and objects (null for references).
- **Operators**: Arithmetic (+,-,*,/,%), Relational (<,>,==), Logical (&&,||,!), Assignment (=,+=), Unary (++, --), Ternary (condition ? x : y), Bitwise (& ,|,^, >>, <<).
- **Conditionals**: if-else handles branching; switch-case supports multi-path execution (supports String & enum since Java 7).
- **Loops**: for, while, do-while for repetition; Enhanced-for (for-each) for collections/arrays.
- **Type Casting**: Widening (automatic, safe: int to long), Narrowing (manual, may lose data: double to int).

- **Keywords**: Reserved words like final, static, this, super, transient, volatile, synchronized.

2. Object-Oriented Programming (OOP)

- **Classes & Objects:** Class = blueprint with fields + methods, Object = runtime instance of the class.
- **Constructors**: Used to initialize objects; can be overloaded; if none defined, Java provides a default.
- **Inheritance**: extends lets child reuse parent behaviour. Single inheritance for classes, multiple inheritance via interfaces.
- **Polymorphism**: Overloading = same method name with different signatures; Overriding = subclass redefines parent method (runtime dispatch).
- **Encapsulation**: Hide variables with private and expose via getters/setters. Helps maintain control & validation.
- **Abstraction**: Abstract classes (partial abstraction) and Interfaces (full abstraction till Java 7; default methods allow behaviour from Java 8).
- **this & super**: this: reference current object, super: call parent's methods/constructors.
- **final**: Used to create constants, prevent inheritance, prevent method overriding.
- **static**: Belongs to class, not object. Used for utility methods, static blocks, static nested classes.

3. Core Concepts

- **Methods**: Modularize code; have parameters, return types, and may throw exceptions.
- **Overloading & Overriding**: Overloading = compile-time resolution; Overriding = runtime resolution (polymorphism).
- **Arrays**: Fixed-length container for same-type elements; multi-dimensional arrays for matrices.
- **Strings**: Immutable (String), mutable alternatives are StringBuilder (not thread-safe, fast) and StringBuffer (thread-safe).
- **Wrapper Classes**: Provide object representation of primitives (useful for collections, autoboxing/unboxing).
- **Enums**: Represent constant sets with methods and fields (can have constructors too).

4. Collections Framework

- **List**: Ordered, duplicates allowed (ArrayList = dynamic array, LinkedList = doubly linked).
- **Set**: No duplicates (HashSet = unordered, LinkedHashSet = insertion order, TreeSet = sorted).
- **Map**: Key-value pairs (HashMap = unordered, LinkedHashMap = ordered, TreeMap = sorted keys).
- **Queue/Deque**: FIFO (Queue), double-ended (Deque, e.g., ArrayDeque). PriorityQueue orders by comparator.
- **Collections Utility**: Helper methods like sort, binarySearch, max, unmodifiableList.
- **Stream API**: Declarative functional style with map, filter, reduce, collect. Parallel streams enable concurrency.

5. Java Streams & Functional Programming

- Functional Interfaces: Predicate, Function, Consumer, Supplier.
- Lambda expressions
- Method references (Class::method)
- Stream API: intermediate operators (map, filter, sorted) and terminal operators (collect, reduce, forEach).
- Parallel streams.
- Optional: of, empty, ofNullable, orElse, ifPresent.

6. Input/Output (I/O)

- **Console I/O**: Read with Scanner, BufferedReader; output with System.out.println().
- **File I/O**: Classes: FileReader, FileWriter, BufferedReader, BufferedWriter. Support line-by-line processing.

Serialization: Persist object state to a file/network via Serializable. Mark sensitive fields transient.

- **NIO (New I/O)**: Channels, Buffers, Selectors for non-blocking, scalable I/O operations.

7. Exception Handling

- **Checked vs Unchecked**: Checked must be declared or handled (IOException); Unchecked are runtime (NullPointerException).
- **try-catch-finally:** Catch exceptions gracefully; finally always executes (commonly for resource cleanup).
- **throw & throws:** throw creates an exception; throws declares that method may pass exceptions up.
- **Custom Exceptions:** Extend Exception or RuntimeException for business-specific errors.

8. Multithreading & Concurrency

- Thread Creation: Extend Thread or implement Runnable/Callable.
- Synchronization: synchronized methods/blocks ensure only one thread accesses resource. Locks/ReentrantLocks give more control.
- Volatile: Guarantees visibility of changes to variables across threads (not atomicity).
- Executor Framework: High-level API to manage pools of threads (ExecutorService, ScheduledExecutorService).
- Future & CompletableFuture: Handle async tasks and callbacks (CompletableFuture = non- blocking, chainable).
- Atomic Classes: Thread-safe operations (AtomicInteger, AtomicBoolean).
- Parallel Streams: Divide stream operations into multiple threads automatically.

9. Java Memory & Performance

- Stack vs Heap: Stack = method calls, local vars; Heap = object instances, GC-managed.
- Garbage Collection: JVM automatically reclaims memory. Different algorithms: Serial, Parallel, G1, ZGC.
- Memory Leaks: Commonly due to unclosed streams, static collections holding

references.

- JVM Tuning: Configure heap size (-Xmx), GC behavior, thread stack size for optimization.

10. Java 8+ Features

- Lambdas: Concise anonymous functions (x -> x*x).
- Functional Interfaces: Interfaces with one abstract method (Supplier, Consumer, Predicate).
- Streams: Declarative style for processing collections with intermediate ops (map, filter) and terminal ops (collect, reduce).
- Optional: Container to avoid NullPointerException (orElse, ifPresent).
- Default & Static Methods in Interfaces: Provide implementations in interfaces without breaking old code.
- New Date/Time API: Immutable, thread-safe API (LocalDate, Period, ZonedDateTime).

11. Advanced

- Generics: Add type safety (List<String> vs List<Object>). Avoids casting errors.
- Annotations: Metadata used at compile or runtime (@Override, custom annotations, @Retention).
- Reflection API: Inspect and modify classes, fields, methods dynamically (used in frameworks).
- JDBC: API for DB connectivity using SQL queries (Connection, PreparedStatement, ResultSet).
- JPA/Hibernate: Object Relational Mapping (ORM) to simplify DB interaction via entities.
- Spring Integration: Use with Java for dependency injection, AOP, REST APIs, microservices.

12. Testing in Java

- JUnit (annotations: @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll)
- Mockito: mocks, spies, stubbing
- TestContainers (integration testing with DBs.)

13. Performance & Best Practices

- StringBuilder vs String concatenation
- Use primitives when possible
- Object pooling (where needed)
- Avoid unnecessary synchronization
- Caching results (Memoization)
- JVM tuning (Xms, Xmx, GC tuning)

313 November

1.Difference between

JVM vs JRE vs JDK JVM

(Java Virtual Machine)

JVM is the heart of Java. When we write Java code, it does not directly run on our computer's operating system. Instead, it runs inside the JVM.

JVM reads the **.**class file (which has bytecode) and converts it into machine code that our computer can understand.
This is what makes Java **platform independent**, the same Java code can run on Windows, Mac, or Linux, as long as a JVM is installed.

JRE (Java Runtime Environment)

JRE provides everything that JVM needs to run a program. It has the JVM, plus libraries and other files required during execution.

If the JVM is the engine, the JRE is the car that holds the engine and the fuel. We use JRE when we only want to run Java programs, not write them.

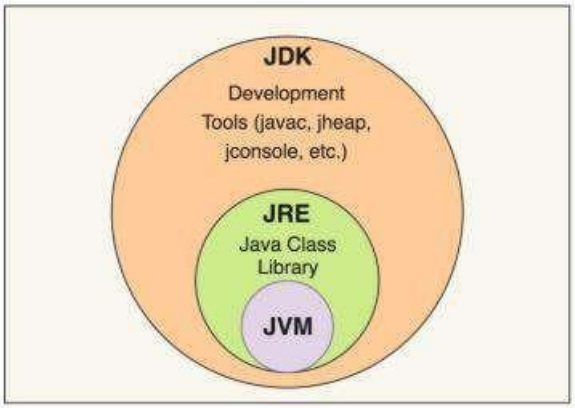
JDK (Java Development Kit)

JDK is the complete package for developing Java applications. It includes the JRE, the JVM, and also tools to write, compile, and debug Java code, like the javac compiler.

We install JDK when we want to create Java programs.
When we compile a file like MyProgram.java, the JDK converts it into MyProgram.class (bytecode), which can then be run by the JVM.

Example:

1. We write a program: MyProgram.java
2. JDK compiler (javac) compiles it into bytecode: MyProgram.class
3. JVM runs that bytecode inside the JRE.



- **JVM** runs the code.
- **JRE** provides the environment to run the code.
- **JDK** provides tools to write and compile the code.

2. What is throwable in Java?

In Java, Throwable is the parent class for everything that can be thrown as an error

or exception during program execution. It has two main types: **Exception** and **Error**.

Exception represents problems that can be handled, like dividing by zero or accessing a file that does not exist.
Error represents serious problems that usually cannot be fixed, like running out of memory.

```
try {
      int a = 5 / 0;
   } catch (Exception e)
   {
   System.out.println(e)
```

Here, Exception is a type of Throwable that the program catches and handles.

3. What is the default access modifier if none is specified?

If no access modifier is specified in Java, the default access level is called **package-private**.

It means the class, method, or variable is accessible only within the same package. It cannot be used by classes from a different package.

Example:

```
class MyClass {
         void showMessage() {
      System.out.println("Hello");
       }
}
```

Here, both MyClass and showMessage() have **default access**.
They can be used by other classes inside the same package but not from outside packages.

4. Will the below code compile and why?

```
void test() {

}

String test() {
             return "";
}
```

Explanation:

No, it will not compile. Both methods have the same name and same parameters, but only return type is different.
In Java, **method overloading** requires a either different number of parameters or different type of parameters, not just the return type.

5. Will the below code work?

```
class Student implements
Serializable { File file; // not
serialized
```

No, this code will not work if we try to serialize the Student object.

Since File does not implement Serializable, Java will throw a NotSerializableException at runtime when we attempt to serialize it.

Point to remember:

If a Serializable class has a member that is not serializable, the program will throw a NotSerializableException at runtime when we try to serialize that object.

This happens because every member inside a Serializable class must also be serializable.

To fix this, we can mark that non-serializable member as transient. This tells Java to skip that field during serialization.

Fix:

```
        class Student implements Serializable {
        transient File file; // not serialized
}
```

6. what is the diamond operator in java

The diamond operator (<>) in Java helps write cleaner code when working with generics. It was introduced in Java 7 to remove the need for repeating type information on both sides of an assignment.

Example: `List<String> names = new ArrayList<>();`

Here, we use <> instead of writing new ArrayList<String>(). The compiler automatically understands that this list stores String values.

7. What are static imports and any example from Java SDK?

static import in Java allows us to use static members of a class directly without writing the class name every time. It helps make the code shorter, cleaner, and easier to read when the same static
methods or constants are used often.

Explanation:

When an ArrayList grows, it keeps creating new in

```
import static java.lang.Math.*;
class Test {
   void show() {
      System.out.println(sqrt(16));     // instead of Math.sqrt(16)
      System.out.println(max(10, 20)); // instead of Math.max(10, 20)
   }
}
```

Here, we use static import from the Math class. Without static import, we would have to write Math.sqrt and Math.max. It is useful when we call many static methods or constants repeatedly in our program.

8. Java Scenario 51

Imagine we have an application that keeps processing data in batches.

```
List<String> records = new ArrayList<>();

while (hasMoreBatches()) {
   records.clear();                    // reuse the same list
   records.addAll(fetchNextBatch());   // add around 1000 records
   process(records);                   // process the data
}
```

Question: How can we decide the right initial capacity for an ArrayList in this case?

ternal arrays as elements are added, which costs memory and time. It is better to set

an initial capacity close to the average number of elements it usually holds.

```
List<String> names = new ArrayList<>(100);
```

If the list generally stores around 100 items, this avoids repeated resizing. It improves performance when the same list is used many times.

Follow-up question:

What will happen, if we set 1000 items without initial capacity?

If we create an ArrayList without specifying an initial capacity, it starts with a default capacity of 10. Now, when we keep adding elements:

- Once it reaches 10, it resizes to make room.
- Then again at 15, 22, 33, and so on (roughly 1.5 times each time).
- By the time we add 1000 items, it will have resized many times internally.

Each resize creates a new array, copies all existing elements to it, and discards the old one.

The list will still work and hold 1000 items, but it will perform much slower compared to a list created with an initial capacity of 1000.